



bcd8906b-7787-47...

PDF

I'm working on a research project for the Lossfunk Autoresearch challenge. I have already written my research proposal, and I want to use AI heavily during the project, but I still want to be actively involved in the research rather than just letting AI decide everything.

I'm planning to use Codex for the actual coding, repository work and eventually running the experiments. I want to use you more as a research partner while I supervise what Codex is doing.

I'll share my proposal with you first. Can you read it and help me understand what I should be careful about before I start?

Worked for 12s

Yes. I read the proposal carefully. The core idea is **strong and researchable**, but there are several things I would lock down *before letting Codex build the experiment*. Your biggest risk is not coding—it is accidentally creating an experiment where the hierarchical method gets an unfair advantage and then interpreting the result as evidence for hierarchy.

Your proposal already identifies the right central comparison:

hierarchical planning vs. **flat receding-horizon planning**, while controlling inference computation and increasing task horizon.

bcd8906b-7787-479b-a0e8-68ccff5...

That is the part I would protect very carefully.

1. First: what your actual research question is

I would keep reminding yourself that your project is **not**:

"Can hierarchical planning improve PushT?"

That would be too easy and much less interesting.

Your actual question is:

Does explicit hierarchical subgoal representation provide an advantage beyond what ordinary receding-horizon replanning already provides, when computation is controlled, and does that advantage change with horizon?

Your proposal explicitly frames the killer alternative this way: perhaps hierarchy only appears better because it converts one long rollout into multiple shorter rollouts, something a flat receding-horizon planner can also do.

bcd8906b-7787-479b-a0e8-68ccff5...

This distinction should become the **central invariant of the whole project**.

Whenever Codex proposes an implementation change, ask:

"Does this change make the hierarchical planner better because of hierarchy, or because I accidentally gave it something else?"

2. The biggest thing to be careful about: "compute matched"

This is currently the most important methodological issue.

Your proposal says you will match inference computation and measure:

- world-model forward passes
- planner evaluations
- parameter count
- planning iterations
- wall-clock inference time
- environment interactions.

bcd8906b-7787-479b-a0e8-68ccff5...

That's good.

But **"same number of forward passes" does not automatically mean "same computation."**

For example:

Flat RH

```
current state
  ↓
world model
  ↓
candidate actions
  ↓
choose action
  ↓
execute
  ↓
new observation
  ↓
repeat
```

Hierarchical

```
current state
  ↓
HIGH-LEVEL PLANNER
  ↓
subgoal
  ↓
LOW-LEVEL PLANNER
  ↓
actions
  ↓
execute
```

The hierarchical system may have:

- an additional network
- additional candidate subgoals
- additional evaluations
- additional search
- additional latent-state processing

So if both systems get "100 world-model evaluations," they may still have very different computational workloads.

Before coding, define exactly what counts as computation.

I would create a formal budget such as:

$$C = N_{WM} + \alpha N_{HP} + \beta N_{LP}$$

or whatever accounting scheme actually fits your implementation.

You don't necessarily need a complicated formula. The important thing is that **Codex must not invent the definition later.**

Your proposal already says exact measured budgets should be reported.

bcd8906b-7787-479b-a0e8-68ccff5...

I'd make that a hard requirement.

3. "Task horizon" needs an extremely precise definition

This is another major issue.

Your hypothesis says:

$$\Delta(H, C) = S_{\text{hier}}(H, C) - S_{\text{flat-RH}}(H, C)$$

and asks whether the difference changes with H .

bcd8906b-7787-479b-a0e8-68ccff5...

That's excellent conceptually.

But **what exactly is H ?**

Is it:

- number of environment actions?
- number of action chunks?
- number of world-model prediction steps?
- number of planning steps?
- number of control decisions?
- number of subgoals?
- temporal length of the task?

These are not necessarily equivalent.

You need one precise definition.

For example:

H = number of primitive environment action steps required/evaluated in the planning episode.

Then keep that definition fixed.

Otherwise you could accidentally create a situation where "longer horizon" also means "more computation," "more replanning," or "more subgoals," making interpretation difficult.

4. Your three methods must have information parity

Your three experimental conditions are:

- 1. Flat open-loop
- 2. Flat receding-horizon
- 3. Hierarchical planning

bcd8906b-7787-479b-a0e8-68ccff5...

But there is a deeper question:

What information does each planner get?

For example, suppose hierarchical planning receives:

```
current image
+
history
+
goal
+
extra latent representation
```

while flat RH receives only:

```
current image
+
goal
```

Then the result isn't cleanly attributable to hierarchy.

Likewise, if the hierarchical planner sees the environment more frequently, that's a confound.

You should create an **information-access table** before implementation:

Property	Open-loop	Flat RH	Hierarchical
Same initial observation	✓	✓	✓
Same goal	✓	✓	✓
Same world model	?	?	?
Same training data	?	?	?

Property	Open-loop	Flat RH	Hierarchical
Same observation frequency	?	?	?
Same environment interaction	?	?	?
Same model backbone	?	?	?
Additional high-level network	—	—	✓
Replanning	No	Yes	Yes
Explicit subgoal	No	No	Yes

Do not let Codex decide these questions.

These are research-design decisions.

5. Model capacity is a hidden danger

Your proposal explicitly recognizes this:

other explanations include greater model capacity, more inference computation, or a more favorable training setup.

bcd8906b-7787-479b-a0e8-68ccff5...

Suppose:

Flat RH

100M parameters

Hierarchical

100M world model

- 30M high-level planner
- 20M low-level planner

Then hierarchy has 150M parameters.

If it wins, you cannot confidently say:

"hierarchy caused the improvement."

It might simply have more capacity.

So decide whether your primary experiment will use:

Option A — Shared backbone

Same world model with different planning mechanisms.

This is probably the cleanest conceptual comparison.

Option B — Parameter-matched systems

Adjust architectures so total trainable parameters are approximately matched.

Option C — Report capacity explicitly and treat it as a limitation

Possible, but weaker.

The important thing is that this decision happens **before seeing results**.

6. Your hierarchical planner needs a very precise definition

Right now the proposal says:

"a high-level planner to predict intermediate subgoals and a low-level planner to reach them."

bcd8906b-7787-479b-a0e8-68ccff5...

That's conceptually clear, but implementation-wise it leaves a lot open.

You need to define:

What is a subgoal?

For example:

g_t = latent state at time $t + k$

or

g_t = predicted future observation

or

$g_t = \text{object position/configuration}$

or some learned latent representation.

This choice could substantially affect the research.

How many subgoals?

One?

$$s_0 \rightarrow g_1 \rightarrow s_T$$

Or:

$$s_0 \rightarrow g_1 \rightarrow g_2 \rightarrow g_3 \rightarrow s_T$$

How far apart are subgoals?

Fixed interval?

Adaptive?

How are subgoals selected?

Sampling?

Optimization?

Search?

Learned prediction?

These aren't merely coding details.

They define what "hierarchy" means in your experiment.

7. Be very careful with the oracle-subgoal experiment

Your proposal has a very useful ablation:

oracle subgoals as an upper-bound diagnostic.

bcd8906b-7787-479b-a0e8-68ccff5...

This can tell you something extremely interesting.

Imagine:

Flat RH = 70%
Hierarchical = 72%
Oracle = 95%

That tells you something very different from:

Flat RH = 70%
Hierarchical = 72%
Oracle = 73%

The first suggests subgoal **selection/prediction quality** may be a major bottleneck.

The second suggests hierarchy itself may have little headroom.

But there's a critical rule:

Oracle information must never accidentally enter the main comparison.

You already state that it is a diagnostic rather than a deployable method. Keep that separation very strict.

bcd8906b-7787-479b-a0e8-68ccff5...

8. Your reachability idea is particularly interesting

I think this is one of the strongest parts of the proposal.

You distinguish:

subgoal prediction error

from

subgoal reachability

from

subgoal completion

Your proposal explicitly treats these as different quantities.

bcd8906b-7787-479b-a0e8-68ccff5...

That is important.

Because a subgoal can be:

Accurate but unreachable

```
predicted g ≈ desired g
      ↓
low-level controller
      ↓
cannot reach
```

Or:

Inaccurate but reachable

```
predicted g ≠ ideal g
      ↓
but controller can reach it
      ↓
task continues successfully
```

Therefore:

prediction error $\neq$ execution difficulty

This gives you a potential **mechanistic explanation**, rather than merely saying "method A got 5% better."

Protect this part of the experiment.

9. Don't let Codex choose the experiment after seeing results

This is probably the biggest issue given your plan to use AI heavily.

Your proposal currently says the horizons will be:

selected after baseline reproduction and then kept fixed for the main comparison.

bcd8906b-7787-479b-a0e8-68ccff5...

That's reasonable, but it creates a potential researcher/agent degree-of-freedom.

Imagine Codex runs:

H=10
H=20
H=30
H=40
H=50

and discovers hierarchy only wins at H=40.

Then it says:

"Let's focus on H=40-60."

That's dangerous.

Instead, establish a rule like:

After baseline reproduction, select horizons according to a predefined difficulty/feasibility criterion, record them, and freeze them before the main comparison.

Then:

no changing horizons because the results look interesting.

Same applies to:

- seeds
- evaluation episodes
- computation budgets
- subgoal spacing
- replanning frequency
- thresholds
- success criteria
- statistical tests.

10. Your PushT success criterion must be frozen

Your proposal correctly says you will inspect the benchmark implementation and then fix the success criterion before comparisons.

bcd8906b-7787-479b-a0e8-68ccff5...

Do exactly that.

Don't do:

```
run experiment
↓
look at results
↓
choose success threshold
```

Do:

```
inspect benchmark
↓
define criterion
↓
document criterion
↓
freeze it
↓
run experiments
```

This is a small decision that can have a huge effect on scientific credibility.

11. Statistical analysis needs one more layer of thought

You say:

multiple random seeds, mean performance, variation across seeds, uncertainty across evaluation episodes, confidence intervals.

bcd8906b-7787-479b-a0e8-68ccff5...

Good.

But decide **what your statistical unit is**.

Suppose:

```
3 seeds
×
500 episodes
```

You should not casually treat this as:

$$n = 1500$$

independent experiments.

The episodes within a single training seed share the same learned model.

So conceptually you have:

Seed 1 → 500 episodes

Seed 2 → 500 episodes

Seed 3 → 500 episodes

not:

1500 independent models

This matters especially when you eventually claim:

"hierarchy significantly improves performance."

Decide the statistical procedure before the main experiments.

12. Don't optimize solely for "significance"

Your proposal is actually stronger than that because it asks about:

$$\Delta(H, C)$$

not merely whether:

$$S_{\text{hier}} > S_{\text{flat}}$$

I would preserve that.

A useful result could be:

H	Flat RH	Hierarchical	Δ
10	82%	82%	0%
20	77%	78%	+1%
40	59%	68%	+9%
60	31%	48%	+17%

The scientific story isn't simply:

"hierarchical is better."

It is:

the incremental benefit appears to increase with horizon under the matched computation regime.

That's much closer to your actual hypothesis.

13. The negative result is genuinely valuable

One thing I like about your proposal is that you've already protected against confirmation bias.

You explicitly say that if hierarchical and flat RH perform similarly, that itself tells you something. bcd8906b-7787-479b-a0e8-68ccff5...

And later:

If hierarchy does not outperform a compute-matched receding-horizon baseline, explicit subgoal representations may not be necessary in the tested setting. bcd8906b-7787-479b-a0e8-68ccff5...

Keep this mindset.

Your goal isn't:

Make hierarchy win.

Your goal is:

Find out whether hierarchy adds something that receding-horizon correction doesn't already provide.

That distinction will make your research much stronger.

14. What I would NOT let Codex decide

Since you're specifically planning to use Codex as the coding/experiment agent, I would divide responsibilities like this.

You decide

- research question
- hypothesis
- primary comparison
- definitions
- success criterion
- horizon definition

- compute budget definition
- fairness rules
- seeds
- statistical methodology
- primary metrics
- interpretation
- whether an unexpected result changes the research question

Codex decides/handles

- repository structure
- implementation
- boilerplate
- environment setup
- logging
- configuration files
- test writing
- debugging
- experiment execution
- plotting
- aggregation
- reproducibility scripts

Me + you

Use me as your **research discussion layer**:

```
You
↓
research decision
↓
Me
↓
critique / alternative explanations / experiment design
↓
Codex
↓
implementation
↓
results
↓
Me + you
↓
interpretation
```

Not:

You

↓

Codex

↓

"here's the result"

↓

paper

15. I would create a "Research Constitution" before coding

This is probably the single most useful thing we can do next.

A short document—maybe 2–4 pages—that says:

Research question

$$\Delta(H, C) = S_H(H, C) - S_{RH}(H, C)$$

Primary hypothesis

Hierarchy provides additional reliability at sufficiently long horizons.

Primary comparison

Hierarchical vs flat receding-horizon.

Secondary comparison

Open-loop vs flat RH vs hierarchical.

Fixed definitions

- task horizon H
- compute budget C
- task success
- subgoal
- subgoal reachability
- subgoal completion

Fixed experimental rules

- seeds
- evaluation episodes
- horizon-selection procedure
- compute matching

- training data
- model capacity
- observation frequency
- replanning frequency

Primary metrics

- task success
- success vs horizon
- success vs compute
- $\Delta(H, C)$

Secondary diagnostics

- subgoal error
- reachability
- completion
- model prediction error
- planning failure
- controller failure

Forbidden post-hoc changes

For example:

Do not change the primary metric, horizons, seeds, success threshold, or comparison protocol because of observed results without explicitly labeling the change as a new exploratory experiment.

That document becomes your **scientific guardrail against both yourself and Codex**.

16. One more important point: don't start with the full experiment

Your proposal itself has the correct staged structure:

1. reproduce flat baseline
2. implement flat RH and hierarchical
3. run horizon/compute/subgoal experiments
4. analyze failures and replicate if possible.

bcd8906b-7787-479b-a0e8-68ccff5...

I would make this even stricter:

Stage 0 — Research specification

No serious implementation yet.

Stage 1 — Environment + baseline reproduction

Only answer:

Can we reproduce the baseline?

Stage 2 — Measurement infrastructure

Before hierarchy, make sure we can accurately record:

```
seed
horizon
world-model evaluations
planner evaluations
wall-clock time
environment steps
success
failure category
```

Stage 3 — Flat RH

Get this working independently.

Stage 4 — Hierarchical planner

Only then introduce the new mechanism.

Stage 5 — Pilot

Small number of seeds/episodes to find implementation bugs.

Stage 6 — Freeze protocol

Lock the actual experiment.

Stage 7 — Main run

No methodological improvisation.

Stage 8 — Exploratory analysis

Only after the primary result is complete.

17. Your biggest risks, ranked

If I were supervising this project, these would be my watchlist:

Risk	Why it matters
Compute mismatch	Hierarchy may win simply because it gets more computation
Information mismatch	Hierarchy may receive more useful information
Model-capacity mismatch	Hierarchy may have more parameters
Horizon definition	"Longer horizon" could change several variables simultaneously
Replanning mismatch	Hierarchy may effectively receive more correction opportunities
Subgoal definition	Different definitions can make hierarchy much easier/harder
Post-hoc tuning	Results can become unintentionally cherry-picked
Statistical pseudoreplication	Episodes aren't independent learned models
Oracle leakage	Privileged information could accidentally enter the primary method
Compute accounting for reachability	Diagnostic rollouts can quietly inflate the experimental budget
Baseline reproduction	If baseline is weak, hierarchy can appear artificially strong
Implementation complexity	More components create more opportunities for hidden differences

The first six are the ones I would worry about most.

18. Most importantly: your proposal is already pointing toward the right scientific contribution

The proposal's contribution is not really "I built a hierarchical world model."

It's:

I isolated the incremental value of explicit hierarchy from shorter rollouts and repeated local correction.

Your proposal explicitly describes that as the contribution.

bcd8906b-7787-479b-a0e8-68ccff5...

That is a much more interesting research question.

And your failure analysis is also unusually useful: you plan to distinguish world-model prediction errors, subgoal-selection failures, reachability failures, low-level controller failures, planning-search failures, and compounding errors.

bcd8906b-7787-479b-a0e8-68ccff5...

I would protect that diagnostic structure rather than letting the project collapse into a single "success rate" number.

How I suggest we work together

Since you specifically want to use AI heavily **without giving AI ownership of the research**, I think we should establish a very clear workflow:

You = research lead

Me = research partner / critical reviewer

Codex = implementation + experiment engineer

Before you ask Codex to build anything substantial, **bring its proposed implementation/design to me**. We can ask:

- 1.** Is this faithful to the research question?
- 2.** Is the comparison fair?
- 3.** What confound did we introduce?
- 4.** What alternative explanation remains?
- 5.** What should be measured/logged?
- 6.** What should be frozen before running?
- 7.** What result would actually falsify the hypothesis?

Then Codex implements it.

And when Codex produces results, **don't immediately ask it to interpret them**. Bring the raw results here first. We can separate:

what happened → what could explain it → what experiment distinguishes the explanations → what conclusion is justified.

That will keep you genuinely involved in the research rather than becoming a manager of an AI-generated experiment.

My recommendation for the immediate next step

Before touching the repository, let's turn this proposal into a **Research Protocol v1.0** with the exact definitions for:

- H
- C
- compute matching
- observation/replanning frequency
- subgoal definition
- model-capacity matching
- PushT success criterion
- seeds
- evaluation protocol
- primary statistical analysis
- what Codex is and isn't allowed to change

Once those are frozen, **then** we can design the Codex implementation plan around them.

That is the point where I would be comfortable letting Codex start coding.

Sources

That makes sense. Before we lock anything down, I want to understand which parts of the plan actually need a decision from me and which parts we can leave open for the AI to explore. Can we go through that first?

Absolutely. I think this is the **right step before locking the protocol**.

The useful distinction is not simply "human decisions vs AI decisions." It is:

Which decisions determine what scientific claim your experiment can support?

Those should belong to you.

Which decisions are implementation choices whose alternatives can

be explored without changing the claim?

Those can be delegated to AI.

Your proposal already gives us a good starting boundary because it explicitly says that coding agents can implement baselines, automate experiments, debug, and generate plots, while the researcher should decide fair compute-matched comparisons, necessary baselines, and interpretation of unexpected results.

bcd8906b-7787-479b-a0e8-68ccff5...

I'd divide the project into **four levels of decisions**.

1. Decisions you should personally own

These are the decisions where changing the choice can change the *scientific question*.

A. The central research question — YOU

This should stay fixed:

Does explicit hierarchical subgoal representation provide an additional reliability advantage over flat receding-horizon planning when computation is controlled, and how does that advantage scale with horizon?

That's the core of the project.

bcd8906b-7787-479b-a0e8-68ccff5...

AI can help sharpen wording, but shouldn't decide to replace it with:

"Let's compare Mamba vs Transformer"

or

"Let's see whether hierarchy improves PushT."

Those would be different projects.

B. What counts as the primary comparison — YOU

Your proposal identifies:

Hierarchical vs flat receding-horizon

as the most important comparison.

bcd8906b-7787-479b-a0e8-68ccff5...

The open-loop planner is useful as a reference point, but the scientific test is really:

Hierarchy *vs* Flat RH

AI can suggest additional baselines.

But AI shouldn't decide:

"The open-loop baseline is easier, so let's make that the main comparison."

That changes the scientific argument.

C. What "fair comparison" means — YOU + ME

This is a human research judgment.

For example:

Should both methods receive the same world-model?

Should parameter counts be matched?

Should wall-clock time be part of the primary budget?

Should observation frequency be identical?

These aren't merely engineering questions because different answers affect what your result means.

Your proposal already recognizes this issue by planning to report multiple computational quantities rather than relying on one definition of computation.

bcd8906b-7787-479b-a0e8-68ccff5...

We can have AI **propose possible fairness rules**, but you should approve the final rule.

2. Decisions AI can explore, but you should approve

This is probably the largest category.

AI can investigate alternatives and make recommendations, but you shouldn't blindly accept the choice.

A. Exact architecture

For example:

- which world model
- latent representation
- planner architecture
- high-level network
- low-level controller
- number of layers
- hidden dimension
- optimizer
- learning rate

These can largely be explored by Codex/AI.

You don't need to personally decide:

"Use 256 hidden dimensions instead of 384."

That's engineering.

But there is an important boundary:

Architecture differences that affect fairness require your approval.

For example:

```
Flat RH:  
World Model → Planner  
  
Hierarchical:  
World Model → High-Level Planner  
               → Low-Level Planner
```

If the hierarchical system gets a substantially larger model, that's no longer merely an implementation detail.

So:

AI proposes → you approve the fairness implication.

B. Subgoal representation

This is more interesting.

Your proposal currently says hierarchical planning uses a high-level planner to predict intermediate subgoals and a low-level planner to reach them.

bcd8906b-7787-479b-a0e8-68ccff5...

But there are multiple ways to implement that.

AI could explore:

latent subgoal

versus

future state

versus

object position

versus

future observation

This is an excellent place for AI exploration.

But we need to distinguish:

Exploratory question

Which representation is easiest/stablest to implement?

AI can explore.

Scientific definition

What exactly counts as "explicit hierarchical subgoal representation" in this study?

You should approve.

3. Decisions AI should mostly own

These are things where you don't gain much research value by manually deciding.

Coding

Codex can decide:

- file organization
- classes
- functions
- configuration system
- logging implementation
- checkpoint format
- experiment scripts
- unit tests
- error handling

You don't need to micromanage this.

Debugging

Let Codex investigate:

```
CUDA error
↓
tensor shape mismatch
↓
incorrect environment API
↓
fix
```

That's exactly what coding agents are good at.

You should only intervene when the proposed "fix" changes the experiment.

Performance engineering

AI can explore:

- batching
- caching
- multiprocessing
- GPU utilization

- mixed precision
- memory optimization
- parallel evaluation

Again, these are implementation details **unless they change the computation being compared.**

Plotting

AI can generate:

- success vs horizon
- success vs computation
- $\Delta(H, C)$
- subgoal reachability
- failure breakdown
- confidence intervals

You can review the plots, but don't need to specify every plotting detail beforehand.

4. Decisions that should remain deliberately OPEN

This is important because I don't want us to over-protocolize the project.

Some uncertainty is actually **research opportunity.**

Your proposal has several secondary experiments:

- oracle subgoals
- noisy subgoals
- constrained vs unconstrained high-level search
- reachability analysis
- second environment.

bcd8906b-7787-479b-a0e8-68ccff5...

I would **not lock these down yet.**

Instead:

Primary experiment

Strictly controlled.

Secondary experiments

Exploratory.

For example, after the primary experiment we might discover:

"Hierarchy only fails because predicted subgoals have poor reachability."

Then AI can investigate:

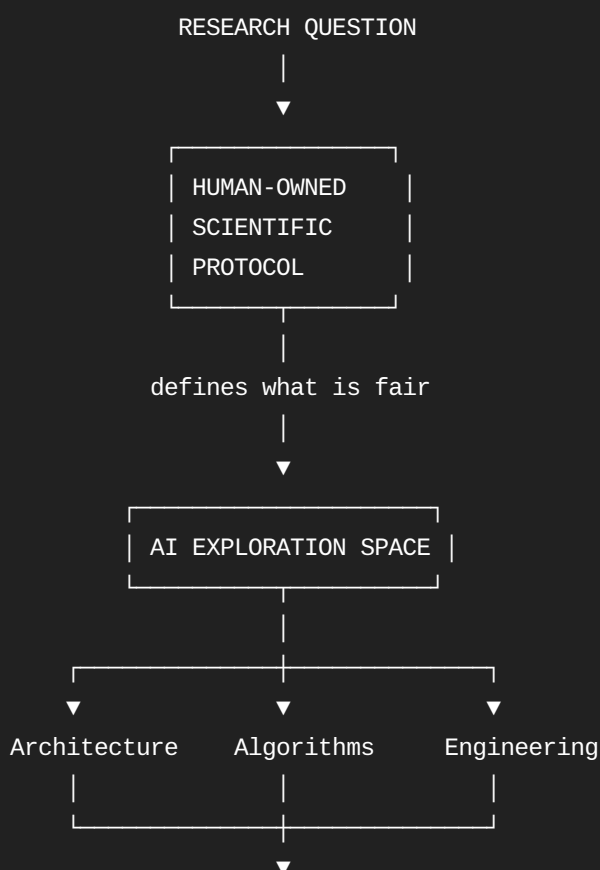
```
noise level
  ↓
reachability
  ↓
task success
```

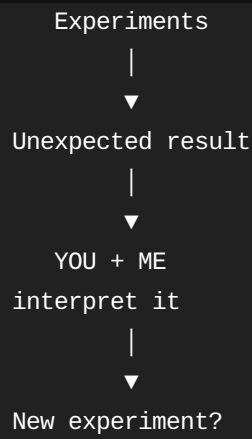
and suggest additional experiments.

That's exactly where you want AI creativity.

5. A very useful distinction: "protocol" vs "search space"

I'd structure your research like this:





This is much better than trying to pre-decide every detail.

6. Here's how I'd classify your actual proposal

Decision	Who should own it?	Why
Research question	You	Defines entire study
Main hypothesis	You	Determines falsification
Primary comparison	You	Defines causal comparison
PushT as primary environment	You	Defines scope
Definition of horizon H	You + me	Affects interpretation
Definition of computation C	You + me	Central fairness issue
Success criterion	You	Prevents post-hoc tuning
Seeds	You	Statistical protocol
Main horizons	You + me	Avoids cherry-picking
Number of evaluation episodes	You + me	Statistical validity
Statistical test	You + me	Determines evidence
World-model architecture	AI explores	Engineering choice
Planner architecture	AI explores	Engineering choice

Decision	Who should own it?	Why
Subgoal implementation	AI explores + you approve	Can affect scientific meaning
Hyperparameters	AI explores	Optimization
Code architecture	Codex	Engineering
Debugging	Codex	Engineering
GPU optimization	Codex	Engineering
Logging	Codex	Engineering
Plot generation	Codex	Engineering
Failure analysis	AI explores + you interpret	Scientific reasoning
Oracle experiment	Open initially	Secondary diagnostic
Noisy subgoal experiment	Open initially	Secondary diagnostic
Second environment	Open	Only after main result
New hypothesis from results	You + me	Scientific interpretation
Whether result supports claim	You + me	Cannot delegate interpretation

7. There is one especially important category: AI may propose, but cannot silently change

Let's call these "**red-line variables**."

Codex is allowed to experiment with them during development, but once the primary protocol is frozen, changing them requires your explicit approval.

I'd put these on the list:

H , C , success criterion, observation access, replanning frequency, training data
--

And probably:

subgoal definition

because that's central to your treatment.

If Codex says:

"I changed the replanning frequency because performance was better."

That is **not a normal code change**.

It is a research decision.

Codex should stop and tell you.

8. Conversely, don't unnecessarily restrict AI

Here's an important part of your goal.

You said you want to use AI heavily while remaining actively involved.

That doesn't mean:

"I personally decide everything."

That would waste the advantage of AI.

For example, suppose Codex discovers:

"There are three reasonable ways of computing the inference budget."

That's useful.

Let it investigate all three.

Then bring the evidence to us:

Budget definition	Advantage	Problem
WM forward passes	Simple	Ignores planner cost
Total network evaluations	More complete	Different network costs

Budget definition

Advantage

Problem

Wall-clock

Real-world cost

Hardware dependent

Then **you choose the scientific convention.**

That's genuine human-in-the-loop research.

9. I would actually encourage Codex to challenge your assumptions

This is where your setup could become much stronger.

Don't instruct Codex:

"Implement my hypothesis."

Instead:

"Implement the experiment while actively checking whether the proposed comparison is confounded."

For example, Codex could report:

"The hierarchical planner currently receives two additional model evaluations per control step. This makes the proposed compute match invalid."

That's exactly the behavior we want.

Or:

"The high-level planner sees the ground-truth future state during training, while the flat planner does not."

Excellent catch.

Or:

"The hierarchical planner's replanning interval is effectively shorter."

Again, useful.

AI shouldn't be the **scientific authority.**

It should be a very aggressive **experimental auditor**.

10. The ideal workflow for your project

I would use this cycle repeatedly:

Step 1 — You define the scientific question

↓

Step 2 — I challenge the reasoning

I look for:

- confounds
- alternative explanations
- missing controls
- statistical problems

↓

Step 3 — Codex proposes implementation

↓

Step 4 — Codex audits its own implementation

↓

Step 5 — You approve the experiment

↓

Step 6 — Codex runs it

↓

Step 7 — Raw results come back

↓

Step 8 — We interpret them

Not:

"AI says hierarchy won, therefore hierarchy works."

Instead:

"Here is the result. What explanations are consistent with it?"



Step 9 — Design discriminating experiments

This is where the research becomes interesting.

11. One thing I would deliberately NOT freeze right now

I wouldn't decide yet:

"What exact hierarchical architecture are we going to use?"

We don't know enough yet.

Your proposal establishes the conceptual treatment, but it doesn't require one particular architecture.

The same applies to:

- exact subgoal representation
- high-level search algorithm
- low-level planner
- network size
- optimizer
- training schedule
- secondary ablations.

Let Codex/AI investigate those.

What we *should* eventually freeze is the **scientific interface**:

What information goes in → what constitutes a subgoal → what computation is counted → what information the flat baseline receives → what constitutes success.

Then the implementation can evolve underneath that interface.

So I think our next discussion should be narrower

Rather than writing the Research Constitution immediately, let's go through the proposal **decision by decision**.

I'd suggest we start with the **5 decisions that most affect the scientific validity**:

1. **What exactly is a task horizon H ?**
2. **What exactly counts as inference computation C ?**
3. **What information must be identical across the three planners?**
4. **What exactly constitutes a hierarchical subgoal?**
5. **What must be frozen before we see the main results?**

For each one, I'll give you:

What the proposal currently says → what is genuinely a research decision → what AI can explore → what I think we should leave open → what you ultimately need to decide.

That way you're not just signing off on a protocol I designed—you'll actually understand **why each decision matters** before you make it.

Sources

Okay, let's go through those five one by one. Start with task horizon H , and explain what the proposal currently says, what choices we actually have, and what I need to decide.

Yes. Let's take **only Task Horizon H** for now and not jump ahead to computation or subgoals.

1. What your proposal currently says

Your proposal makes horizon a central independent variable.

You say you will:

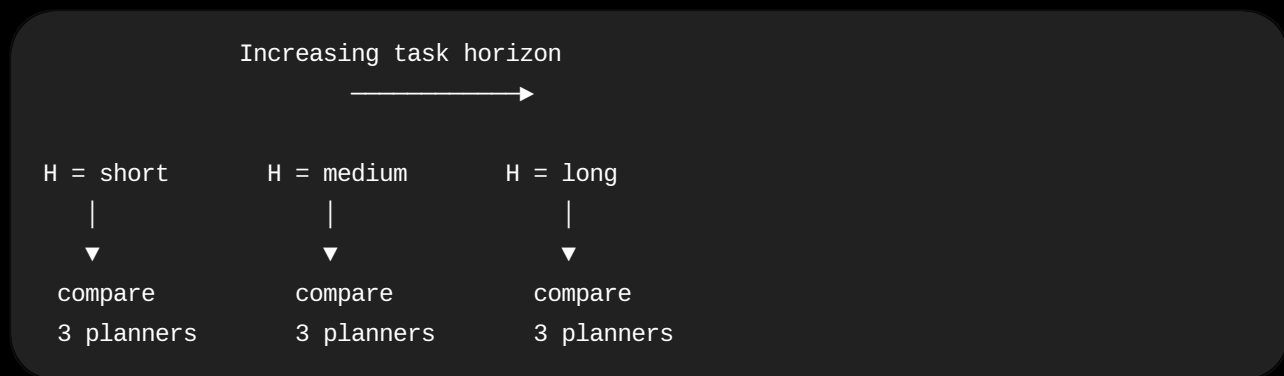
- evaluate the three planners over **increasing task horizons**;
- choose a predetermined set of short, medium, and long horizons after baseline reproduction;
- keep those horizons fixed for the main comparison;
- examine whether the hierarchical advantage

$$\Delta(H, C) = S_{\text{hier}}(H, C) - S_{\text{flat-RH}}(H, C)$$

changes as H increases.

bcd8906b-7787-479b-a0e8-68ccff5...

So conceptually you want something like:



The scientific question is then:

Does hierarchy become increasingly useful as the task becomes longer?

That's a good question.

But there is a gap

Your proposal **does not yet define exactly what H numerically means.**

It says "task horizon," "planning horizon," and "long-horizon" in several places, but it doesn't give us a precise operational definition such as:

$H = 50$ means 50 primitive environment action steps.

So this is genuinely something we need to decide.

2. Why the definition of H matters so much

Imagine we say:

$$H = 10, 20, 40, 80$$

What does "80" mean?

It could mean:

Choice A — Primitive action horizon

The agent has to execute 80 environment actions.

$s_0 \rightarrow a_0 \rightarrow s_1 \rightarrow a_1 \rightarrow \dots \rightarrow a_{79} \rightarrow s_{80}$

This is probably the most intuitive interpretation of **task horizon**.

Choice B — World-model prediction horizon

$H = 80$ means the world model predicts 80 future states.

But if your planner executes only part of those predictions before replanning, then the relationship between H and actual task difficulty becomes less direct.

Choice C — Number of planning decisions

For example:

$$H = 10$$

could mean the agent makes 10 high-level planning decisions.

But a hierarchical agent and a flat RH agent might have very different numbers of decisions for the same physical task.

That could make the comparison problematic.

Choice D — Action chunks

Suppose one decision produces 5 primitive actions.

Then:

$$H = 20$$

might actually correspond to:

$$20 \times 5 = 100$$

environment actions.

Again, the meaning changes.

3. There is an even deeper issue

Your experiment has **three different notions of horizon** hiding inside it.

This is important.

① Task horizon

How long the actual task is.

② Planning horizon

How far ahead the planner looks.

③ Replanning interval

How much of the plan gets executed before planning again.

For example:

Task:

$H = 100$ actions

Flat RH:

—plan—execute—replan—execute—replan—
h=20

Hierarchical:

—high-level—subgoal—subgoal—
| |
└low-level┐

These are **not the same thing**.

And your research question specifically wants to distinguish hierarchy from the benefits of shorter rollouts and frequent replanning.

bcd8906b-7787-479b-a0e8-68cff5...

Therefore, we have to be especially careful not to accidentally use "horizon" to mean different things for different planners.

4. What choices do we actually have?

I'd separate them into **two decisions**.

Decision A: What does H measure?

Possible choices:

Definition	Meaning	Main advantage	Main concern
Primitive environment steps	Number of actual actions in task	Very intuitive	Task difficulty may vary nonlinearly
World-model prediction steps	Number of predicted future steps	Directly tied to model rollout	Can confuse task horizon with planning horizon
Planning decisions	Number of planner decisions	Natural for planning	Hierarchy and flat RH aren't directly comparable
Action chunks	Number of temporal action blocks	Useful for chunked control	Depends on chunk definition

My current inclination is **primitive environment steps**, because your question is about how reliability changes as the *task itself* becomes longer.

But I don't want to silently decide that for you—the proposal hasn't specified it.

5. Decision B: How do we create different horizons?

Suppose we choose primitive environment steps.

We could then create:

Option 1 — Fixed set

For example:

$$H \in \{20, 40, 60, 80\}$$

Simple and easy to reproduce.

Option 2 — Log-spaced horizons

For example:

$$H \in \{10, 20, 40, 80\}$$

This gives more resolution across a large range.

Option 3 — Environment-defined difficulty

Instead of arbitrarily saying "80 is long," define horizons based on how many actions are needed for different task configurations.

Option 4 — Maximum feasible horizons

Start with the shortest feasible task and extend until performance becomes extremely low or computation becomes impractical.

This is useful for exploration, but **I wouldn't use it to choose the final horizons after seeing which method wins.**

6. Your proposal already gives us one important constraint

You wrote:

"I will evaluate a predetermined set of short, medium, and long horizons, selected after baseline reproduction and kept fixed for the main comparison."

bcd8906b-7787-479b-a0e8-68ccff5...

I actually like this structure.

It means we don't need to decide the exact numerical values **today**.

Instead:

Before baseline

We don't know what horizons are computationally feasible.

Baseline reproduction

We learn the practical range.

Then

We choose the horizon set using a predefined rule.

Then freeze it.

So there are really two things:

Rule for selecting horizons → should be decided before baseline exploration.

Actual numerical horizons → can be determined after baseline reproduction.

That's an important distinction.

7. What can AI explore?

Quite a lot.

I would absolutely let Codex explore the environment before we freeze the actual horizon values.

For example, it can investigate:

```
H = 10
H = 20
H = 30
H = 40
H = 60
H = 80
H = 100
```

and report:

H	Success	Avg episode length	Compute	Feasible?
10	...	...	...	✓
20	...	...	...	✓
40	...	...	...	✓
80	...	...	...	✓
100	...	...	...	?

This is **exploration**, not yet the main experiment.

AI can determine:

- what horizons the environment supports;
- where the baseline becomes difficult;
- where computation becomes unreasonable;
- whether some horizon values behave strangely;
- whether the benchmark has natural task lengths;

- what range gives meaningful variation in success.

That's exactly the kind of work I'd delegate.

8. What AI should NOT do

Suppose exploratory results look like:

H=10	hierarchy = 81%
H=20	hierarchy = 82%
H=30	hierarchy = 84%
H=40	hierarchy = 86%
H=50	hierarchy = 91%
H=60	hierarchy = 93%

Codex should **not** say:

"Let's use 40–60 because that's where hierarchy starts winning."

That would be selecting the experimental range based on the outcome.

Instead, the selection should be based on something independent, such as:

Choose four horizons spanning the feasible range of task lengths, with short/medium/long categories defined from baseline feasibility.

Then whatever happens at those horizons is the result.

9. There's another choice: should H be the same across all methods?

I strongly think this should be **yes** for the primary comparison.

Suppose:

Flat RH:
H = 100
Hierarchy:
H = 100

Both are solving the same task horizon.

Their **planning mechanisms** can differ.

That's what we want.

But we should not do:

Flat RH:

$H = 100$

Hierarchy:

$H = 100$, but broken into 4×25 -step subgoals

and then call those different horizons.

The overall task horizon remains:

$$H = 100$$

The **subgoal interval** is another variable.

That distinction will become very important when we get to your fourth question about subgoals.

10. I would therefore separate these variables

Eventually our experiment might look conceptually like:

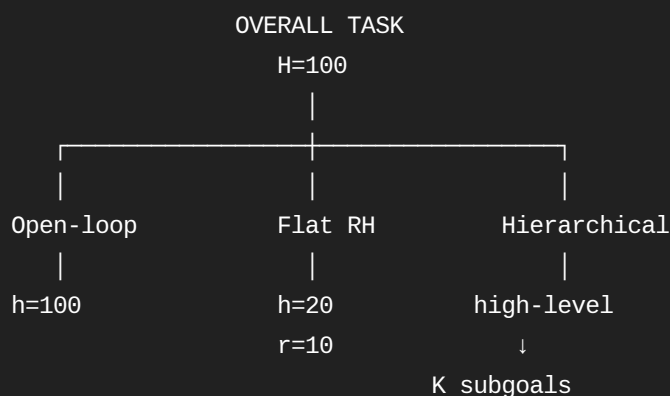
H = overall task horizon

h = local planning horizon

r = replanning interval

K = number of hierarchical subgoals

So:



This separation is **very important** for your research question.

Otherwise, when hierarchy wins, we won't know whether it was because of:

- hierarchy,
- shorter planning horizon,
- more frequent replanning,
- more intermediate decisions.

Your proposal already recognizes this exact problem conceptually.

bcd8906b-7787-479b-a0e8-68ccff5...

11. So what do YOU actually need to decide?

For H , I think you only need to make **three conceptual decisions now**.

Decision 1 — What does H represent?

My suggested candidate:

H = number of primitive environment action steps constituting the overall task horizon.

But this is a proposal, not something I want to impose on you.

Decision 2 — Should H be identical across methods?

I'd recommend defining:

Yes. H describes the same overall task for all three methods.

The methods differ in how they plan/control that task, not in what task length they are being evaluated on.

Decision 3 — How will we choose the actual H values?

I would **not choose the numbers yet**.

Instead, decide the rule:

After reproducing the baseline, select a fixed set of short/medium/long horizons spanning the feasible task range, using a criterion determined before looking at the hierarchical-vs-RH result. Freeze them before the primary comparison.

Then AI can do the baseline exploration needed to determine the actual numbers.

12. What I would leave open

For now, I'd leave these open:

- exact numerical values of H ;
- number of horizon levels;
- exact spacing between them;
- whether 3, 4, 5, etc. levels are practical;
- exact PushT task configurations;
- whether a second environment needs a different horizon convention.

Codex can help us discover the feasible range.

13. My proposed "decision card" for you

Before we move to C , I want **you** to answer these three:

H1 — Definition

Which do you want H to represent?

- A. Primitive environment action steps
- B. World-model prediction steps
- C. Planning decisions
- D. Something else

H2 — Cross-method consistency

Should the same overall H represent the same task for all three planners?

I would say **yes**, but you make the call.

H3 — Horizon selection

Are you comfortable with:

Explore feasible horizons during baseline reproduction → choose short/medium/long using a predefined rule → freeze them → never choose/change them based on the hierarchy-vs-RH result?

Again, that's what I'd recommend, but this is your research protocol.

Once you've answered **H1-H3**, we'll have genuinely made the first research decision together. Then we'll move to **#2: what exactly counts as inference computation C** .

Sources

For H1, I think primitive environment action steps make the most sense, since I want H to represent the actual task horizon rather than the planner's internal horizon. For H2, yes, the same overall H should represent the same task for all three methods. For H3, I'm fine with exploring the feasible range during baseline reproduction, then choosing and freezing the horizons using a rule that doesn't depend on which method performs better. Does that seem scientifically sound?

Yes. **Scientifically, those three decisions are sound**, and more importantly, they are well aligned with the actual question in your proposal.

I would lock them in, with one small refinement to H3.

H1 — Definition of task horizon

Decision:

H = number of primitive environment action steps in the task

I agree with your reasoning. This keeps H conceptually separate from the planner's internal horizon.

That distinction is important because your research question is about whether hierarchy helps with **longer tasks**, not whether one planner happens to look farther ahead internally.

So later we should be able to say:

"At task horizon $H = 80$, the agent must complete an 80-step task."

while separately reporting things like:

- local planning horizon,

- replanning interval,
- number of subgoals.

Those become **mechanisms**, not definitions of H .

H2 — Same task horizon across methods

Decision: Yes.

For a given experimental condition:

$$H_{\text{open}} = H_{\text{RH}} = H_{\text{hier}}$$

The three methods are therefore solving the same overall task.

For example:

$H = 80$ primitive actions

Open-loop:

[----- 80 -----]

Flat RH:

[--20--][--20--][--20--][--20--]

Hierarchical:

[-----subgoal-----][-----subgoal-----]...

The internal decomposition can differ, but the **overall task horizon remains 80**.

That is exactly the separation we want.

H3 — How we choose the actual horizons

Your proposed procedure is also sound:

Explore first → determine feasible range → apply a predetermined selection rule → freeze horizons → run the primary comparison.

The important word is **predetermined**.

We should eventually write something like:

"After baseline reproduction, we will characterize the feasible task-horizon range. We will then select a fixed set of short, medium, and long horizons according to a criterion defined independently of comparative performance. These horizons will remain fixed for the primary experiment."

Your proposal already points in this direction by saying the horizons will be selected after baseline reproduction and then kept fixed.

bcd8906b-7787-479b-a0e8-68ccff5...

One refinement I recommend

There's one subtle issue we haven't completely resolved:

What exactly does "feasible range" mean?

We shouldn't let Codex decide:

"H=80 is feasible because it ran."

That could accidentally make feasibility depend on arbitrary things like:

- one particular GPU,
- one timeout,
- one random seed,
- one planner,
- or whether a method happens to fail early.

Instead, when we get to baseline exploration, we'll define **feasibility independently of which method is better**.

For example, we might establish that the selected horizons must:

1. be valid under the PushT task/environment definition;
2. be computationally executable within a predetermined resource limit;
3. span meaningful short/medium/long task durations;
4. be sufficiently separated to test horizon scaling.

We don't need to decide those exact thresholds today.

So I would record H as follows

Task Horizon Protocol — v0.1

Definition

H = number of primitive environment action steps comprising the overall task

Cross-method consistency

The same H represents the same overall task horizon for open-loop, flat receding-horizon, and hierarchical planners.

Separation of variables

H will not be used to represent:

- local planning horizon,
- replanning interval,
- number of subgoals,
- or high-level planning steps.

Those will be treated as separate experimental variables.

Horizon selection

The feasible task-horizon range will be explored during baseline reproduction. The final horizon set will then be selected according to a criterion that does **not depend on comparative performance between the three methods**, and will be frozen before the primary comparison.

Post-freeze rule

Changing the primary horizon set because one method performs unexpectedly well or poorly will not be permitted without explicitly labeling the resulting experiment as exploratory.

One thing I would *not* decide yet

I would **not** choose something like:

$$H = \{20, 40, 80\}$$

yet.

We should let the baseline tell us what range is practical. Then we can make the selection rule concrete.

So yes: **H1, H2, and H3 can now be considered settled at the conceptual level.**

And this is a good example of the human/AI division we discussed: **you made the scientific decisions; later Codex can investigate the feasible numerical range without deciding what the research means.**

Next, we can move to **#2: what exactly counts as inference computation** *C*. That's probably the trickiest decision in the entire protocol because your central claim depends on the phrase "**compute-matched.**"

Sources

Okay, let's move to C. I want to understand what should count as inference computation, what options we have, and why the choice could affect the scientific claim. Don't decide it for me yet—let's examine the alternatives first.

Yes. And I agree with your instinct not to decide this yet.

Your proposal actually recognizes the problem quite well: it proposes using **world-model forward passes and planner evaluations as the primary budget**, while separately reporting parameter count, planning iterations, wall-clock inference time, and environment interactions.

bcd8906b-7787-479b-a0e8-68cff5...

The question is whether that is enough to support the phrase "**compute-matched.**"

1. What are we trying to control?

Suppose we compare:

Hierarchical *vs* Flat RH

and hierarchy achieves:

$$S_{\text{hier}} = 80\%$$

while flat RH achieves:

$$S_{\text{RH}} = 70\%.$$

We want to say something like:

"Hierarchy provides an additional advantage under matched inference computation."

For that statement to be meaningful, we need to know:

$$C_{\text{hier}} \approx C_{\text{RH}}$$

But **what exactly is C ?**

That's the question.

2. Option A — World-model forward passes

This is closest to what your proposal currently calls the **primary budget**.

For example:

$$C = N_{\text{WM}}$$

where N_{WM} is the number of world-model forward passes.

Imagine:

Method	WM evaluations
Flat RH	10,000
Hierarchical	10,000

Then we call them compute-matched.

Why this is attractive

Your project is fundamentally about **world-model planning**, so world-model evaluations are a meaningful computational resource.

It's also:

- easy to count;
- reproducible;
- independent of GPU hardware;
- directly available from the experiment logs.

And it fits your proposal's current formulation.

bcd8906b-7787-479b-a0e8-68ccff5...

But here's the problem

One world-model forward pass isn't necessarily the same amount of computation as one planner evaluation.

Suppose:

Flat RH

World model

↓

simple planner

while:

Hierarchical

World model

↓

high-level planner

↓

subgoal search

↓

low-level planner

Both could use exactly:

10,000

world-model evaluations.

But the hierarchical system might perform substantially more computation elsewhere.

So:

Equal WM evaluations $\neq$ necessarily equal total inference computation.

3. Option B — Count all model/network evaluations

We could define:

$$C = N_{\text{WM}} + N_{\text{high-level}} + N_{\text{low-level}} + N_{\text{other}}$$

Then we'd count essentially every relevant inference call.

Example:

	Flat RH	Hierarchical
World model	8,000	6,000
Planner	2,000	1,500
High-level	—	1,000
Low-level	—	1,500
Total	10,000	10,000

Now the methods have equal numbers of inference operations.

Advantage

More comprehensive than counting only world-model calls.

Problem

A forward pass through one tiny network and a forward pass through a huge network are not equivalent.

For example:

Network A: 1 million parameters
Network B: 100 million parameters

Counting:

$$1 + 1 = 2$$

doesn't mean they consumed equal computation.

So this gives us:

evaluation-count matching

rather than true computational matching.

That's an important distinction.

4. Option C — FLOPs / estimated computational operations

Another possibility is to estimate:

$$C = \text{FLOPs}$$

or some related hardware-independent operation count.

Then instead of saying:

both methods made 10,000 evaluations,

we could say:

$$C_{\text{hier}} \approx C_{\text{RH}} = 10^{12} \text{ FLOPs.}$$

Advantage

This gets closer to actual computational workload.

It accounts for the fact that different networks have different sizes.

But there are complications

FLOPs depend on implementation details.

For example:

- batch size;
- sequence length;
- attention complexity;
- precision;
- conditional computation;
- different hardware kernels.

And FLOPs aren't necessarily proportional to actual runtime.

So:

same FLOPs $\nRightarrow$ same wall-clock time.

5. Option D — Wall-clock inference time

Another possibility:

$$C = \text{actual inference time}$$

For example:

Method	Inference time
Flat RH	2.0 sec
Hierarchical	2.0 sec

Then we can say they had the same **latency budget**.

This has a very appealing interpretation

It answers:

"If I give both systems the same amount of real computational time, which is more reliable?"

That's arguably very practical.

But it introduces hardware dependence

Suppose:

GPU A → hierarchical = 2 sec
GPU B → hierarchical = 3 sec

Your result can become dependent on:

- GPU model;
- software stack;
- kernel implementation;
- parallelization;
- batching;
- CPU/GPU transfer;
- background load.

So wall-clock time is experimentally meaningful, but less clean as the **sole scientific definition** of computation.

Your proposal wisely says it will report wall-clock inference time separately rather than making it the only budget.

bcd8906b-7787-479b-a0e8-68ccff5...

6. Option E — Parameter count

We could also think about:

$C \approx$ number of parameters.

But this is really a **capacity measure**, not an inference-computation measure.

For example:

Method	Parameters
Flat RH	100M
Hierarchical	100M

This tells us that model capacity is matched.

But if one model evaluates its parameters 100 times and the other 10 times:

$$100M \times 100$$

versus

$$100M \times 10,$$

their inference costs are obviously different.

So parameter count is useful for answering:

"Does hierarchy simply have a larger model?"

but not:

"Did both methods receive the same inference computation?"

Your proposal correctly treats parameter count as something to **report separately**.

bcd8906b-7787-479b-a0e8-68ccff5...

7. Option F — A weighted computational budget

We could build something more sophisticated.

For example:

$$C = \alpha N_{WM} + \beta N_{HP} + \gamma N_{LP}$$

where:

- N_{WM} = world-model evaluations
- N_{HP} = high-level planner evaluations
- N_{LP} = low-level planner evaluations
- α, β, γ = computational costs.

This would attempt to approximate actual computation more accurately.

For example, if one WM evaluation costs approximately twice one planner evaluation:

$$C = 2N_{\text{WM}} + N_{\text{planner}}.$$

Interesting—but dangerous

Where do α, β, γ come from?

We'd need to measure or estimate them.

If we choose them arbitrarily, we've simply hidden an assumption inside the definition of C .

So this could become a useful **secondary analysis**, but I wouldn't casually make it the primary definition without justification.

8. Option G — Multi-budget reporting rather than one definition

This is probably the most interesting alternative.

Instead of pretending that there is one perfect definition of computation, we could report several:

Primary

$$C_{\text{WM}} = \text{world-model evaluations}$$

Secondary

$$C_{\text{all}} = \text{total network/planner evaluations}$$

Also report

- FLOPs, if reliably measurable;
- wall-clock inference time;
- parameter count;
- planning iterations.

Then we could ask:

Does the hierarchical advantage survive under multiple reasonable notions of computational budget?

This is conceptually powerful.

Suppose we get:

Budget	Hierarchical advantage
WM evaluations	+8%
Total evaluations	+7%
FLOPs	+6%
Wall-clock	+5%

That is much harder to explain away as an artifact of one particular budget definition.

Conversely:

Budget	Advantage
WM evaluations	+8%
Total evaluations	+1%
FLOPs	−2%
Wall-clock	−4%

Now we'd have learned something very different:

The apparent hierarchical advantage depends strongly on how computation is accounted for.

That itself could be scientifically interesting.

9. But there's a deeper issue: environment interaction

Your proposal also says you'll report **environment interactions**.

bcd8906b-7787-479b-a0e8-68ccff5...

This is related, but it isn't quite inference computation.

Consider:

Flat RH

```
observe
↓
plan
↓
execute 10 actions
↓
observe
↓
plan
```

Hierarchical

```
observe
↓
high-level subgoal
↓
low-level control
↓
execute 10 actions
↓
observe
```

Both might interact with the environment the same number of times.

But environment interactions measure something closer to:

How much real interaction did the agent receive?

rather than:

How much computation did the agent perform?

So I would keep environment interactions as a **separate resource dimension**, not merge them automatically into C .

10. Why this choice changes your scientific claim

This is the really important part.

Imagine hierarchy wins.

If we define compute as WM evaluations

Your claim becomes approximately:

"Hierarchy improves reliability when world-model usage is matched."

That's a fairly specific claim.

If we define compute as wall-clock time

The claim becomes:

"Hierarchy improves reliability under an equal real-time inference budget."

Different claim.

If we define compute as FLOPs

The claim becomes:

"Hierarchy improves reliability for approximately equal computational operation budgets."

Again different.

If we define compute as total model evaluations

The claim becomes:

"Hierarchy improves reliability when inference-call counts are matched."

Different again.

So the budget isn't merely a technical implementation detail.

It determines exactly what your result allows you to conclude.

11. There's another complication specific to your project

Consider this:

Flat RH

WM → planner → action

WM → planner → action

WM → planner → action

versus:

Hierarchical

WM → high-level → subgoal

↓

low-level

↓

action

↓

low-level

↓

action

Suppose hierarchy needs additional high-level computation.

If we give both methods equal **WM evaluations**, hierarchy gets additional computational work.

That's potentially okay **if your scientific question is specifically:**

"What happens when world-model usage is controlled, even though the architecture has additional planning computation?"

But it's not okay if we then claim:

"Hierarchy is better at equal total computation."

Those are different claims.

This is why we need to be precise.

12. I see three fundamentally different interpretations

This may be the cleanest way to think about the alternatives.

Interpretation 1 — Equal world-model opportunity

$$N_{WM}^{hier} = N_{WM}^{RH}$$

Question:

Does hierarchy help when both systems receive the same opportunity to use the world model?

Interpretation 2 — Equal total inference computation

$$C_{hier} = C_{RH}$$

Question:

Does hierarchy help when neither system receives more computational resources overall?

Interpretation 3 — Equal real-time budget

$$T_{hier} = T_{RH}$$

Question:

Does hierarchy help when both systems have the same amount of time to make decisions?

These are **three legitimate research questions**, but they are not identical.

Your proposal currently leans toward **#1 as the primary budget**, while reporting information relevant to #2 and #3.

bcd8906b-7787-479b-a0e8-68ccff5...

13. One thing I would avoid

I would avoid defining:

C = whatever makes the methods look equally expensive.

In other words, don't calculate the budget *after seeing the architecture and results* and then choose the accounting method that favors the desired

interpretation.

The definition needs to be justified independently.

14. There's actually a nice experimental possibility here

We don't necessarily have to force ourselves into:

one magical definition of compute.

We could potentially distinguish:

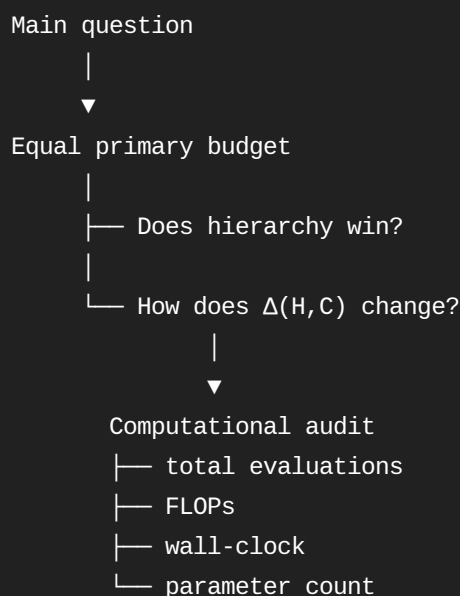
Primary computational budget

One clearly defined, reproducible measure.

Computational audit

Report several additional measures.

Then your results could have a structure like:



That seems much more intellectually honest than pretending any single number perfectly captures "computation."

15. What AI can explore here

This is a great area for Codex to help.

It can measure:

- exact number of world-model calls;
- exact planner calls;
- high-level calls;
- low-level calls;
- parameter counts;
- FLOPs where practical;
- actual inference time;
- GPU memory;
- environment interactions.

It can then produce a **computation audit** for every experiment.

For example:

Experiment: H=80, seed=1

	Flat RH	Hierarchical

WM calls	10,000	10,000
Planner calls	2,000	1,500
HL calls	0	1,000
LL calls	0	1,500
Parameters	80M	92M
FLOPs	...	...
Time	...	...
Env steps	80	80

That's exactly the kind of measurement infrastructure I'd want AI to build.

But the interpretation of those measurements—what constitutes the primary C —is a research decision for you.

16. So we haven't decided C yet

And I think that's correct.

At this point, the alternatives we've identified are:

C_1	World-model forward passes
C_2	All inference/model evaluations
C_3	FLOPs
C_4	Wall-clock inference time
C_5	Parameter count — capacity, not really compute
C_6	Weighted computational cost
C_7	Primary budget + multi-dimensional computational audit

And there is an important distinction:

C_5 shouldn't really be considered a substitute for inference compute. It is better viewed as a control/reporting variable for model capacity.

Likewise, environment interactions should probably remain a separate resource dimension.

The question I want you to think about

Before choosing, I'd ask you one conceptual question:

When you say "compute-matched," what do you most want your eventual scientific claim to mean?

A.

"Both methods get the same number of opportunities to use the world model."

B.

"Both methods receive approximately the same total inference computation, regardless of where that computation occurs."

C.

"Both methods get approximately the same real-world inference time."

D.

"I don't want to reduce computation to one imperfect number; I want a primary budget plus a broader computational audit."

You don't need to choose immediately. **D is not automatically better**, and A/B/C each answers a somewhat different scientific question.

I'd like to hear which interpretation feels closest to the claim **you actually want to make**, and then we can examine the consequences of that choice before locking anything down.

Sources

Before I choose between A-D, can you explain which of these is most consistent with the exact wording and intent of my original proposal, and what scientific claim each choice would allow me to make? I want to preserve the proposal's intent rather than accidentally changing the research question.

Yes. Looking **strictly at your original proposal**, I would distinguish between what it *explicitly says* and what we might choose to strengthen later.

1. What your proposal actually intends by "compute-matched"

Your central question is:

"When controlling for local replanning and inference computation, does explicit hierarchical subgoal representation provide an additional reliability advantage over flat receding-horizon world-model planning...?"

bcd8906b-7787-479b-a0e8-68cff5...

And the proposal repeatedly says the key comparison is:

hierarchical planning **versus flat receding-horizon planning under a matched total inference-computation budget**.

bcd8906b-7787-479b-a0e8-68cff5...

Then, when you explain how you intend to control computation, you say:

"The primary budget will count world-model forward passes and planner evaluations."

bcd8906b-7787-479b-a0e8-68cff5...

And you explicitly say you'll additionally report:

- parameter count,
- planning iterations,

- GPU/wall-clock inference time,
- environment interactions.

bcd8906b-7787-479b-a0e8-68ccff5...

So there is actually a pretty clear hierarchy in your proposal:

Primary

World-model forward passes + planner evaluations

Secondary audit

Parameters + planning iterations + wall-clock time + environment interactions

That means your original proposal is **closest to A**, but with an important qualification.

2. Why it isn't exactly A as I phrased A earlier

Earlier I described A as:

"Both methods get the same number of opportunities to use the world model."

But your proposal says:

world-model forward passes AND planner evaluations

So the more faithful representation is:

$$C_{\text{primary}} = N_{\text{WM}} + N_{\text{planner}}$$

or, more precisely, a matched accounting of those two categories.

Therefore, if we are trying to **preserve the original proposal**, I would not redefine your project as simply:

$$C = N_{\text{WM}}.$$

That would actually make the proposal *narrower* than what you wrote.

3. Let's compare A-D against your original intent

Here's the important part.

Choice	Match to original proposal	Scientific claim it supports
A — same WM opportunity	Closest, but incomplete	Hierarchy helps when WM usage is controlled
B — equal total inference computation	Very close to stated intent, but requires defining "total"	Hierarchy helps beyond RH under approximately equal overall inference computation
C — equal wall-clock time	Secondary in proposal	Hierarchy is more reliable under equal real-time inference budget
D — primary budget + audit	Closest to the proposal as written	Primary result under the defined budget, checked for robustness across other computational measures

So I would actually revise my earlier framing slightly:

Your original proposal is not simply A. It is fundamentally a combination of B-like intent and D-like methodology, with world-model/planner evaluations as the proposed primary operationalization of computation.

That's an important distinction.

4. Why B matches the *wording* of your research question

Your proposal says:

"matched total inference-computation budget."

bcd8906b-7787-479b-a0e8-68ccff5...

The word **total** matters.

If we used only:

$$C = N_{WM},$$

then a hierarchical system could potentially receive substantially more computation in its high-level and low-level planners while still being declared "compute matched."

That would be difficult to reconcile with the strongest reading of:

"total inference-computation budget."

So the *scientific intent* of your wording is closer to:

$$C_{\text{hier}} \approx C_{\text{flat-RH}}.$$

The unresolved question is:

How should we operationalize C ?

That's exactly the decision we're examining now.

5. What would B allow you to claim?

Suppose we successfully define a defensible total inference-computation budget:

$$C_{\text{hier}} \approx C_{\text{RH}}.$$

Then your result could support something like:

Under approximately matched inference computation and the same task horizon, hierarchical planning provides an incremental reliability advantage over flat receding-horizon planning.

That's extremely close to your intended contribution.

Your proposal explicitly describes the contribution as a:

"compute- and replanning-controlled analysis of the incremental value of explicit hierarchical representations."

bcd8906b-7787-479b-a0e8-68ccff5...

So B captures the **scientific claim** particularly well.

6. But B creates a problem you should know about

The phrase "total inference computation" sounds precise, but it isn't automatically precise.

Suppose we have:

Flat RH

- 10,000 world-model evaluations
- 2,000 planner evaluations

Hierarchical

- 7,000 world-model evaluations
- 1,500 low-level evaluations
- 1,000 high-level evaluations

How do we know:

$$10,000 + 2,000$$

is equivalent to:

$$7,000 + 1,500 + 1,000?$$

A forward pass through different networks isn't necessarily equivalent computational work.

So B gives you the **right scientific target**, but we still need a defensible operational definition.

That's why I wouldn't simply say:

"Let's count every forward pass."

We need to think carefully about what "total inference computation" means in this particular architecture.

7. What about C?

C would give you a different scientific question:

$$T_{\text{hier}} \approx T_{\text{RH}}.$$

Then you could claim:

Given the same inference time, hierarchy produces greater reliability.

That's a legitimate and potentially useful result.

But it is **not the central question your proposal was written around**.

Why?

Because your proposal mentions wall-clock time as something you will:

"also report"

bcd8906b-7787-479b-a0e8-68ccff5...

not as the primary definition of the budget.

So making wall-clock time the primary C would change the emphasis of the project.

8. What about D?

D is interesting because it is essentially what your proposal already does structurally.

You have:

Primary budget

World-model forward passes + planner evaluations.

Additional measurements

- parameter count
- planning iterations
- wall-clock inference time
- environment interactions.

bcd8906b-7787-479b-a0e8-68ccff5...

So D would preserve the proposal almost exactly.

The claim would be:

Under the proposal's primary inference-computation budget, hierarchy provides an incremental advantage over flat RH, while the result is evaluated against other computational measures to determine whether the conclusion depends on the accounting convention.

That's probably the most faithful representation of the document **as currently written**.

9. There's an important distinction between "primary claim" and "robustness"

Imagine your experiment produces:

Primary budget

$$C_{WM+planner}$$

Hierarchy wins.

Then:

Wall-clock

Hierarchy also wins.

FLOPs

Hierarchy also wins.

Parameters

Approximately matched.

That's a very strong computational story.

But imagine instead:

Primary budget	→ hierarchy +8%
Wall-clock	→ hierarchy +1%
FLOPs	→ hierarchy -3%

Then the conclusion needs to be more cautious.

You couldn't simply say:

"Hierarchy is more computationally efficient."

You could say something closer to:

"Hierarchy improves success under the primary inference-budget definition, although the advantage is not robust to all measures of computational cost."

That's actually valuable information.

10. What I think your original proposal is *trying* to avoid

The proposal's central concern is not really:

"Which architecture runs faster?"

It's this:

Hierarchy appears better

- because hierarchy is genuinely useful
- because it uses shorter rollouts
- because it replans more frequently
- because it gets more computation
- because it has more capacity

Your experiment is trying to eliminate the alternative explanations.

That's why you wrote:

"The important question is therefore whether hierarchy provides an additional advantage after local replanning and computation are controlled."

bcd8906b-7787-479b-a0e8-68ccff5...

So the **spirit** of the experiment is much closer to B/D than to a narrow A.

11. And this is why I don't think we should choose B blindly

There is a subtle danger in saying:

"Fine, let's just make total FLOPs equal."

That could actually move us away from the original experimental design.

Your proposal explicitly says:

"The primary budget will count world-model forward passes and planner evaluations."

bcd8906b-7787-479b-a0e8-68ccff5...

So if we suddenly replace that with a sophisticated FLOP-normalized budget, we'd be **redesigning the methodology**, not merely implementing your proposal.

That's something we should only do if we conclude that the original operationalization is scientifically inadequate.

12. My reading of the proposal, without choosing for you

If I had to map your original document onto A-D:

A — Partial match

It captures the importance of controlling world-model usage, but it leaves out the proposal's explicit inclusion of planner evaluations.

B — Best match to the intended scientific claim

"Hierarchy should provide additional benefit when total inference computation is controlled."

But B still requires us to decide what "total" means operationally.

C — Not the intended primary question

Wall-clock is explicitly secondary in your proposal.

D — Best match to the actual methodology written in the proposal

You define a primary budget and separately report other resource measures.

So I'd summarize it as:

$$\boxed{\text{Scientific intent} \approx B}$$

while

$$\boxed{\text{Current proposed methodology} \approx D}$$

with the primary budget operationalized using **world-model forward passes + planner evaluations**.

That distinction is, I think, the most important thing for us to recognize before making the next decision.

One question remains before we choose

We now need to answer:

Is "world-model forward passes + planner evaluations" a sufficiently defensible operational definition of the total inference computation for *this particular experiment*?

There are two possibilities:

- 1. Yes:** We keep the proposal's original primary budget and make its exact counting rules precise.
- 2. No:** We preserve the scientific intent but modify the computational budget definition before implementation.

I would **not change it yet**. The next useful step is to examine a concrete hypothetical hierarchical implementation and flat-RH implementation and see exactly what gets counted under your proposal's current definition.

That will tell us whether the original budget is actually adequate, rather than deciding abstractly.

Sources

Yes, let's do that. Give me a concrete hypothetical example of flat RH and hierarchical planning, and show exactly how world-model forward passes and planner evaluations would be counted for each. Then let's see whether that gives us a fair enough compute match or whether something is missing.

Yes. This is the right way to test whether the proposal's current budget is actually adequate.

I'll use a **toy implementation**, not claim this is how PushT or the eventual code must work. The purpose is to expose what the accounting would and wouldn't capture.

Your proposal says the primary budget should count **world-model forward passes and planner evaluations**, with other quantities reported separately.

bcd8906b-7787-479b-a0e8-68ccff5...

1. Hypothetical flat receding-horizon planner

Suppose the overall task horizon is:

$$H = 80$$

meaning 80 primitive environment actions, as we just decided.

The flat RH planner uses:

- local planning horizon = 20 actions
- executes 10 actions
- then replans
- therefore replans 8 times over the 80-step task.

Imagine each planning cycle does:

```
Current observation
    ↓
World model
    ↓
Generate 100 candidate action sequences
    ↓
Evaluate each candidate
    ↓
Select best sequence
    ↓
Execute 10 actions
    ↓
New observation
    ↓
Repeat
```

Let's define one **planner evaluation** as:

evaluating one candidate action sequence using the world model over the specified planning horizon.

That's a reasonable operational definition, but notice that we'd eventually need to make this exact.

2. Count the flat RH computation

Suppose each replanning event evaluates:

$$100$$

candidate action sequences.

There are:

$$8$$

replanning events.

Therefore:

$$N_{\text{planner-evals}} = 8 \times 100 = 800$$

Now suppose every candidate sequence requires 20 world-model steps.

Then:

$$N_{\text{WM}} = 800 \times 20 = 16,000$$

So we'd record:

Flat RH	Count
Replanning events	8
Candidate sequences	800
World-model forward passes	16,000
Planner evaluations	800

Under your proposal's current accounting, the primary computational budget could therefore be represented as:

$$C_{\text{RH}} = (16,000, 800)$$

rather than necessarily collapsing the two into one arbitrary number.

3. Now construct the hierarchical planner

Keep exactly the same overall task:

$$H = 80$$

Suppose the hierarchical planner predicts a subgoal every 20 primitive actions.

So:

$$80/20 = 4$$

high-level stages.

At each stage:

```
Current state
  ↓
High-level planner
  ↓
predict/select subgoal
  ↓
Low-level planner
  ↓
produce 20 primitive actions
  ↓
execute
  ↓
next subgoal
```

Suppose the high-level planner evaluates:

$$50$$

candidate subgoals at each stage.

And the low-level planner evaluates:

$$50$$

candidate action sequences for each selected subgoal.

Then:

High-level planner evaluations

$$4 \times 50 = 200$$

Low-level planner evaluations

$$4 \times 50 = 200$$

Total:

$$N_{\text{planner-evals}} = 400$$

Now suppose each low-level candidate requires 20 world-model steps:

$$200 \times 20 = 4,000$$

world-model evaluations.

And suppose each high-level candidate requires 20 world-model steps as well:

$$200 \times 20 = 4,000.$$

So:

$$N_{\text{WM}} = 8,000.$$

We'd get:

Hierarchical	Count
High-level stages	4
High-level candidate subgoals	200
Low-level candidate sequences	200
Planner evaluations	400
World-model forward passes	8,000

4. Now compare them

We get:

	Flat RH	Hierarchical
Task horizon	80	80
Replanning/stages	8	4
Planner evaluations	800	400
WM forward passes	16,000	8,000

If we use your proposal's current budget, hierarchical planning has received **half the computational budget**.

So this comparison is obviously **not compute-matched**.

This example immediately shows why simply saying:

"We count WM forward passes and planner evaluations"

isn't sufficient.

We need a **budget allocation rule**.

5. We could make them equal

Suppose we increase the hierarchical search.

For example, make it evaluate 100 high-level candidates and 100 low-level candidates at each of the four stages.

Then:

$$N_{\text{planner}} = 4(100 + 100) = 800.$$

If each requires 20 world-model steps:

$$N_{\text{WM}} = 800 \times 20 = 16,000.$$

Now:

	Flat RH	Hierarchical
Planner evaluations	800	800
WM evaluations	16,000	16,000

On the surface, we have an excellent match.

But now we encounter the deeper problem.

6. What exactly is a "planner evaluation"?

The flat planner's candidate evaluation might be:

candidate action sequence
↓
20 WM predictions

↓
score

The hierarchical planner's high-level evaluation might be:

```
candidate subgoal
  ↓
predict whether subgoal is useful
  ↓
perhaps invoke another network
  ↓
score
```

Those aren't necessarily the same computation.

And the low-level evaluation might be:

```
candidate actions
  ↓
20 WM predictions
  ↓
score
```

So our table says:

$$800 = 800$$

planner evaluations.

But the actual workloads could be quite different.

7. Here's the hidden computation

Imagine:

Flat RH

Each planner evaluation costs:

1.0 arbitrary compute units

Hierarchical

High-level evaluation costs:

3.0

because it uses a high-level network/search procedure.

Low-level evaluation costs:

$$1.0.$$

Then the hierarchical planner's 800 evaluations might actually cost:

$$400(3.0) + 400(1.0) = 1600$$

units.

Flat RH:

$$800(1.0) = 800.$$

So we'd have:

$$N_{\text{planner-evals}}^{\text{hier}} = N_{\text{planner-evals}}^{\text{RH}}$$

but

$$C_{\text{hier}} \neq C_{\text{RH}}.$$

This is the first major thing missing from a simple evaluation-count budget.

8. The world-model count has a similar problem

Suppose both use the same world model.

Then:

$$16,000$$

world-model evaluations are quite meaningful.

But what if the hierarchical high-level planner also performs:

- latent transformations;
- candidate subgoal scoring;
- additional network inference;
- search over subgoals?

Those operations aren't captured by:

$$N_{\text{WM}}.$$

So the proposal's separate measurement of planner evaluations is important.

But even:

$$(N_{\text{WM}}, N_{\text{planner}})$$

is still a **two-dimensional proxy** for computation rather than a complete computational equivalence.

9. There is an even more important issue: search structure

Consider these two systems.

Flat RH

```
100 candidates
  ×
20-step WM rollout
  =
2,000 WM steps
```

Hierarchical

```
100 subgoals
  ×
100 low-level actions
```

Maybe hierarchy does:

```
100 high-level candidates
      ↓
select 10 promising subgoals
      ↓
100 low-level candidates each
```

Now the number of evaluations and branching structure becomes more complicated.

Two systems can have the same:

$$N_{\text{WM}}$$

while performing fundamentally different searches.

That's not necessarily a problem—the whole point of your experiment is that **different planning structures may achieve different reliability at the same computational resource.**

But we need to be able to say what resource we actually matched.

10. This leads to an important distinction

I think there are actually **two questions hiding inside "compute matching."**

Question 1

Did we give the methods approximately equal amounts of the **specific computational resource we chose to control**?

For example:

$$N_{\text{WM}}$$

and

$$N_{\text{planner}}.$$

Question 2

Did we give them exactly equal **total computational work**?

For example:

FLOPs.

Your original proposal clearly commits to Question 1 as the practical primary budget and says additional measurements will be reported.

bcd8906b-7787-479b-a0e8-68cff5...

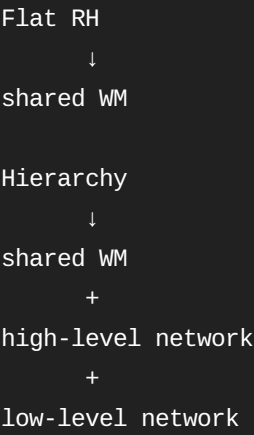
It does **not** establish that exact FLOP equality is required.

That's important because we shouldn't accidentally turn the project into a different experiment.

11. There's another hidden issue: parameter sharing

Suppose both planners use the same world model.

Then:



Now imagine:

	Flat RH	Hierarchical
WM parameters	100M	100M
Planner parameters	10M	10M
High-level parameters	—	20M
Low-level parameters	—	10M

Hierarchy has:

$$140M$$

parameters versus:

$$110M.$$

Even if we perfectly match:

$$N_{WM}$$

and

$$N_{planner},$$

the models don't have equal capacity.

That's why your proposal separately says parameter counts should be reported.

bcd8906b-7787-479b-a0e8-68ccff5...

Again:

parameter count isn't computation, but it's a potential alternative explanation.

12. What happens if we instead match wall-clock time?

Suppose we run both for exactly:

2 seconds

of inference.

Then perhaps:

	Flat RH	Hierarchical
WM calls	20,000	12,000
Planner calls	1,000	700
Wall-clock	2.0 s	2.0 s

Now hierarchy gets fewer WM evaluations because its additional high-level machinery consumes some of the available time.

That experiment answers:

What can each planning strategy accomplish with the same actual inference latency?

That's a perfectly valid question.

But it isn't identical to your proposal's current primary comparison.

13. What I think this hypothetical example tells us

The original budget:

WM forward passes + planner evaluations

is **useful and probably necessary**, but by itself it isn't sufficient to claim:

"The methods received exactly equal total computation."

Instead, it gives us a defensible operational definition such as:

The primary comparison matches world-model usage and planner-evaluation counts, while other computational characteristics are measured separately.

That's a much more precise statement.

14. I would therefore NOT immediately replace your proposal's budget

This example hasn't convinced me that your original approach is wrong.

Rather, it tells us what we need to specify.

At minimum, before implementation, we would need to define:

What is one world-model forward pass?

For example:

One invocation of the learned dynamics/world-model network for one candidate latent transition.

What is one planner evaluation?

For example:

One evaluation of a candidate plan/subgoal according to the planner's scoring procedure.

But even that isn't enough.

We also need to specify whether:

- high-level evaluation counts separately;
- low-level evaluation counts separately;
- auxiliary networks count;
- subgoal scoring counts;
- search operations count;
- repeated reuse/caching counts;
- batch evaluations count as individual evaluations.

15. And here's the interesting possibility

We could keep the proposal's original primary budget as a **vector**, rather than forcing it into one scalar:

$$C = (N_{\text{WM}}, N_{\text{planner}})$$

Then compute matching means:

$$N_{\text{WM}}^{\text{hier}} \approx N_{\text{WM}}^{\text{RH}}$$

and

$$N_{\text{planner}}^{\text{hier}} \approx N_{\text{planner}}^{\text{RH}}.$$

Then separately report:

{FLOPs, time, parameters, memory, environment interactions}.

This is much closer to what your proposal actually says than inventing an arbitrary weighted formula.

And it avoids pretending that:

$$1 \text{ WM evaluation} = 1 \text{ planner evaluation.}$$

16. But there's one problem we still haven't solved

Suppose we match:

$$N_{\text{WM}} = 16,000$$

and

$$N_{\text{planner}} = 800.$$

What if hierarchical planning needs:

- 400 high-level evaluations
- 400 low-level evaluations

while flat RH has:

- 800 flat evaluations?

We've matched the **total count**, but perhaps not the **type** of computation.

We could instead track:

$$C = (N_{\text{WM}}, N_{\text{flat-plan}}, N_{\text{high-plan}}, N_{\text{low-plan}})$$

But now the methods don't have the same components, so matching each component one-to-one becomes impossible.

And this is where we have reached a genuinely interesting research-design decision.

17. The question we need to answer next

There are two defensible philosophies:

Philosophy 1 — Match resource totals

Match:

$$N_{\text{WM}}$$

and total planner evaluations:

$$N_{\text{planner}}$$

regardless of whether those evaluations are high-level or low-level.

This asks:

Given the same amount of world-model usage and the same number of planning evaluations, does the hierarchical organization produce greater reliability?

That is very close to your original proposal.

Philosophy 2 — Match computational work more literally

Try to estimate actual computational cost, perhaps through FLOPs or measured execution cost, and ensure:

$$C_{\text{hier}} \approx C_{\text{RH}}.$$

This asks:

Given approximately equal total computational work, does organizing planning hierarchically improve reliability?

That's a stronger computational claim, but also considerably harder to make rigorous.

My takeaway from the example

I wouldn't change your protocol yet.

What we've learned is:

Your original **WM forward pass + planner evaluation** budget is a reasonable operational starting point, but we need to define exactly what those counts mean and decide whether matching the *counts* is sufficient for the scientific claim you want.

And importantly, your proposal already anticipated this by requiring us to report **wall-clock time, parameter count, planning iterations, and environment interactions separately.**

bcd8906b-7787-479b-a0e8-68ccff5...

So before choosing A-D, I think the next question should be:

Do you want your primary claim to be about equal *resource counts* (WM/planner evaluations), or about equal *total computational work*?

That's the actual fork we've uncovered.

Sources

I think I want the primary claim to be about matched resource counts: world-model forward passes and total planner evaluations. That seems closest to my original proposal. But I also want FLOPs, wall-clock time, parameter count and the other compute measures reported separately so we can see if the conclusion depends on that accounting choice. Does that give us a defensible definition of C, and if so, can we lock it conceptually before moving on?

Yes. **That gives us a defensible conceptual definition of C , and I think it preserves the intent of your original proposal without overclaiming what “compute-matched” means.**

I would lock it at the **conceptual** level now, while leaving the exact counting rules for implementation.

C — Conceptual decision

Your primary inference-resource budget will be a **two-component resource count**:

$$C = (N_{\text{WM}}, N_{\text{planner}})$$

where:

- N_{WM} = number of world-model forward passes
- N_{planner} = total number of planner evaluations

For the primary comparison:

$$N_{\text{WM}}^{\text{hier}} \approx N_{\text{WM}}^{\text{RH}}$$

and

$$N_{\text{planner}}^{\text{hier}} \approx N_{\text{planner}}^{\text{RH}}.$$

This is faithful to your proposal, which explicitly identifies world-model forward passes and planner evaluations as the **primary budget**, while treating wall-clock time, parameters, planning iterations, and environment interactions as additional reported quantities.

bcd8906b-7787-479b-a0e8-68ccff5...

What this lets you claim

If hierarchy outperforms flat RH under this matched budget, your primary conclusion can be approximately:

Under matched world-model usage and total planner-evaluation counts, explicit hierarchical planning provides an additional reliability advantage over flat receding-horizon planning at the tested task horizons.

That's a precise and defensible claim.

Notice what we are **not** claiming:

"The two systems used exactly the same total computational work."

We aren't claiming that, because a high-level planner evaluation and a low-level planner evaluation might have different actual costs.

The computational audit stays separate

For every main experiment, we should additionally record, where practical:

FLOPs

wall-clock inference time

parameter count

plus:

- planning iterations;
- environment interactions;
- memory/resource usage if useful.

This is particularly valuable because your proposal already says these quantities should be reported separately.

bcd8906b-7787-479b-a0e8-68ccff5...

Then we can ask:

Does the scientific conclusion remain similar under different ways of viewing computational cost?

If yes, that's reassuring.

If no, **that's itself an important qualification**, rather than something we hide.

One important thing we're *not* freezing yet

We still need to define exactly what counts as a "**planner evaluation**."

For example, eventually we need rules for:

- a flat candidate-plan evaluation;
- a high-level subgoal evaluation;
- a low-level candidate evaluation;
- auxiliary planner networks;
- batched candidates;

- cached world-model states;
- search operations.

Those are **implementation-level accounting rules**.

So our current decision is:

| **What resource we are matching:** locked.

But:

| **Exactly how Codex counts every operation:** not locked yet.

That's the right order.

So our protocol now has two locked conceptual variables

Task horizon

H = number of primitive environment action steps in the overall task

Same H for all methods; numerical horizons selected after baseline exploration using a method-independent rule and then frozen.

Inference budget

$C = (N_{\text{WM}}, N_{\text{planner}})$

Primary comparisons match both components approximately; FLOPs, wall-clock time, parameter count, planning iterations, and environment interactions are reported separately as a computational audit.

And I would **not change either of these unless we discover a concrete implementation problem that makes them impossible or scientifically misleading**.

So yes—I **consider C conceptually settled**.

The next one is **#3: information parity**: exactly what information all three planners are allowed to see and use. This is potentially just as important as

computation, because giving hierarchy extra information could make a perfectly compute-matched comparison unfair.

Sources

Okay, let's move to information parity. I want to understand exactly what information each planner should have access to, what the current proposal implies, and where hierarchy could accidentally get extra information. Let's examine the alternatives before we lock anything.

Yes. **Information parity is the next critical decision**, because compute-matching alone does not make the comparison fair. A hierarchy can appear to help simply because it was given information that flat receding-horizon planning never receives.

Your proposal already recognizes this issue indirectly through its emphasis on comparable training data, model capacity, inference computation, planning horizons, and separating prediction error from control error.

bcd8906b-7787-479b-a0e8-68ccff5...

Let's examine the alternatives before choosing.

1. First: what does “information parity” mean?

For the primary experiment, we want:

The three planners should have access to the same externally available task information; the only intentional difference should be the planning mechanism they use.

But there is an important distinction:

A. Information given to the planner

For example:

- current observation
- task goal
- observation history
- previous actions

- learned world-model state
- environment feedback

These should generally be controlled.

B. Information generated by the planner itself

For example:

- a hierarchical planner generates a subgoal
- flat RH generates a short action sequence
- open-loop generates a complete action sequence

These **can differ**, because they are precisely the mechanisms we are studying.

So we don't want:

"Everything must be identical."

That would make hierarchy impossible.

Instead:

Same input information → different internal planning mechanism → compare outcomes.

2. What does your current proposal imply?

Your proposal says the main comparison is:

Flat open-loop vs Flat receding-horizon vs Hierarchical

and specifically identifies the primary comparison as hierarchical vs flat RH under matched inference-computation budget.

bcd8906b-7787-479b-a0e8-68ccff5...

It also says to control:

- training data
- model capacity
- inference computation
- planning horizons

and separately investigate observation frequency/replanning updates.

bcd8906b-7787-479b-a0e8-68ccff5...

However, the proposal **does not yet specify a complete information-access contract**.

That's a gap we should resolve before implementation.

3. The biggest danger: hierarchy gets privileged future information

Imagine the PushT task.

Suppose the current state is:

```
Current observation
```

↓

•

T

The flat RH planner receives:

```
current observation
```

```
+ goal
```

and has to figure out what action to take.

Now imagine the hierarchical planner receives:

```
current observation
```

```
+ goal
```

```
+ intermediate waypoint
```

```
+ future object position
```

Then hierarchy has an advantage that has nothing to do with hierarchy itself.

It is receiving **more information**.

That would contaminate the experiment.

4. Let's examine each information category

Alternative 1 — Current observation

All planners receive the same current observation.

Flat RH

observation → world model → plan → action

Hierarchical

```
graph TD
    A[observation] --> B[high-level planner]
    B --> C[subgoal]
    C --> D[low-level planner]
    D --> E[action]
```

Open-loop

observation → full plan → actions

Advantage

Very clean.

All methods start from the same state information.

Problem

Very little. This should almost certainly be common to all methods.

5. Alternative 2 — Goal information

Suppose the final task goal is:

Move the object to target location G.

Should all planners know **G**?

Option A — Yes

Flat RH:	observation + G
Hierarchical:	observation + G
Open-loop:	observation + G

The hierarchy then **generates** intermediate subgoals itself.

This is clean.

Option B — Give hierarchy intermediate goals

For example:

Hierarchical:

$G \rightarrow g1 \rightarrow g2 \rightarrow g3 \rightarrow G$

Flat RH:

G only

This would make the experiment problematic.

Why?

Because now we're testing:

"Does giving the planner intermediate goals help?"

rather than:

"Does explicit hierarchical planning provide additional reliability?"

Those are different questions.

6. Alternative 3 — Ground-truth intermediate states

This is even more dangerous.

Suppose hierarchy receives:

Current state

↓

Ground-truth future waypoint

↓

Low-level controller

while flat RH receives only the current state.

That is **privileged information**.

Your proposal actually anticipates something similar in the oracle-subgoal ablation. It says oracle subgoals should be treated as a diagnostic upper bound and privileged information should be clearly reported.

bcd8906b-7787-479b-a0e8-68ccff5...

So I think the proposal gives us an important distinction:

Primary experiment

Predicted/generated subgoals

not ground-truth subgoals.

Diagnostic experiment

Oracle subgoals

allowed, but clearly separated from the primary comparison.

7. Alternative 4 — Observation history

This one is subtle.

Suppose:

Flat RH

gets:

`o_t`

only.

But hierarchy gets:

`o_1, o_2, ..., o_t`

Then hierarchy has additional temporal information.

Alternatively:

Flat RH

`last 4 observations`

Hierarchy

`last 20 observations`

Again, unfair.

So we need to decide whether all planners receive:

A. Current observation only

or

B. Same fixed observation history

or

C. Whatever their architecture naturally requires

I would **not lock this yet** until we inspect how the baseline implementation represents state.

The important thing is that whichever choice we make must be identical across the primary comparison.

8. Alternative 5 — Previous actions

Another potential leakage:

Flat RH:

observation

Hierarchy:

observation + previous actions

If previous actions help infer velocity or dynamics, hierarchy has extra state information.

So previous actions should either be:

available to everyone

or

available to nobody.

Not method-dependent.

9. Alternative 6 — World-model latent state

This is probably one of the most important implementation decisions.

Suppose the world model produces:

```
observation
  ↓
latent state z_t
```

Should both planners use the same `z_t`?

Option A — Shared world model / shared latent representation

```
      observation
        ↓
    SAME WORLD MODEL
    ↙       ↘
Flat RH   Hierarchy
```

This is very clean.

It isolates the planning mechanism.

Option B — Different world models

```
observation
  ↓       ↓
WM_flat  WM_hier
  ↓       ↓
Flat RH  Hierarchy
```

Now if hierarchy wins, we don't know whether:

| hierarchy helped

or

| its world model was better.

Your proposal explicitly wants model capacity and training data controlled, and your failure analysis separates world-model prediction errors from planning/control failures. `bcd8906b-7787-479b-a0e8-68ccff5...`

Therefore, **shared or tightly matched world-model information is highly attractive for the primary experiment.**

But there is an architectural question here that we shouldn't decide prematurely:

Does the hierarchical planner require additional learned representations that fundamentally cannot be shared?

If yes, we need to distinguish those as part of the mechanism rather than pretending they don't exist.

10. Alternative 7 — Environment feedback

This is where hierarchy could accidentally get a huge advantage.

Suppose:

Flat RH

acts for 10 primitive steps:

```
plan → execute 10 → observe → replan
```

Hierarchy

does:

```
subgoal → execute 2 → observe  
subgoal → execute 2 → observe  
subgoal → execute 2 → observe  
...
```

Then hierarchy receives environmental feedback much more frequently.

That could improve performance independently of hierarchy.

Your proposal explicitly says sensitivity analysis should vary observation frequency/replanning updates.

bcd8906b-7787-479b-a0e8-68ccff5...

So we have to separate:

hierarchical representation

from

more frequent environmental feedback.

This is a major confounder.

11. Alternative 8 — Training information

Another possible leakage:

Flat RH training

training trajectories

Hierarchical training

training trajectories
+ human-designed subgoals
+ annotated intermediate states

Then hierarchy has received additional supervision.

Your proposal says training data should be comparable.

bcd8906b-7787-479b-a0e8-68ccff5...

So for the primary experiment, we'd want the training information to be matched.

If hierarchical supervision is necessary, then we need to ask:

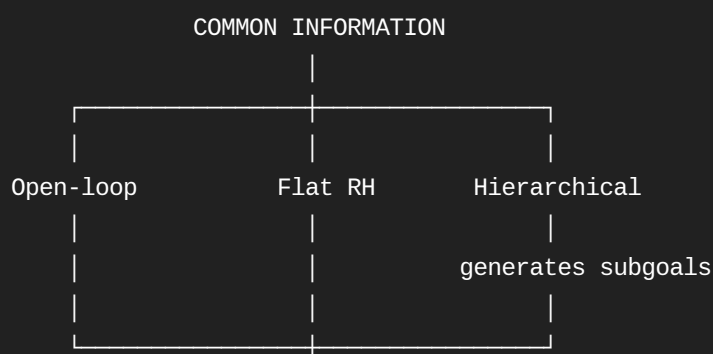
Is that supervision part of the proposed hierarchical method, or is it an additional advantage?

That becomes a scientific decision.

12. The cleanest conceptual model

I think we should think of the experiment as having an **Information Envelope**.

Something like:



Everything above the line is shared.

Everything generated below the line is method-specific.

That gives us a very powerful experimental principle:

Hierarchy may create additional internal structure, but it may not receive privileged external information that the competing planners do not receive.

13. But there is one important exception

Your proposal includes:

Oracle subgoal ablation

That means:

```
Hierarchical
  ↓
ORACLE subgoal
  ↓
Low-level planner
```

This is intentionally privileged.

But that's okay **because it is not the primary comparison.**

It answers a different question:

"If the subgoal itself were perfect, how much performance could the hierarchical controller achieve?"

Your proposal explicitly frames oracle subgoals as a diagnostic upper bound.

bcd8906b-7787-479b-a0e8-68ccff5...

So we should maintain two categories:

Information	Primary comparison	Diagnostic
Current observation	Same	Same
Final goal	Same	Same

Information	Primary comparison	Diagnostic
Observation history	Same	Same
Previous actions	Same	Same
World model	Matched/shared	Can vary diagnostically
Environment feedback	Controlled	Can vary
Predicted subgoals	Hierarchy only	Yes
Ground-truth subgoals	No	Oracle ablation
Extra future state	No	Privileged diagnostic only
Training supervision	Matched	Can vary diagnostically

14. Three possible philosophies for information parity

Now we reach the actual decision.

Philosophy A — Strict information equality

Every planner receives exactly the same externally available information.

For example:

```
I_t = {
  current observation,
  goal,
  same observation history,
  same action history,
  same world-model state
}
```

Then:

```
I_t
↓
Planner-specific mechanism
```

Scientific meaning

Any performance difference is less likely to be explained by unequal information access.

Strength

Very clean causal comparison.

Weakness

Could artificially constrain architectures if one planning method naturally needs a different representation.

Philosophy B — Same environment information, different internal representations

All planners get the same raw information:

observation
goal
history

but they are allowed to transform it differently.

For example:

Raw observation
↓
Flat RH → latent representation
Hierarchy → hierarchical representation

Scientific meaning

Does the hierarchical organization of information improve planning?

This is arguably closer to what we're actually trying to study.

Risk

The hierarchy might construct a much more powerful representation, and we then have to determine whether that representation itself is the source of the improvement.

Philosophy C — Minimal common information + mechanism-specific information

Everyone gets:

current observation
+ final goal

Then each method can internally derive whatever it needs.

Hierarchy:

observation + goal
↓
subgoal generator
↓
subgoal

Flat RH:

observation + goal
↓
short-horizon planner

Open-loop:

observation + goal
↓
long-horizon planner

Scientific meaning

This tests the complete algorithms as they would naturally operate.

Risk

Harder to determine exactly **why** one method wins.

15. My recommendation for how we should reason about it

I don't want to choose this for you, because this is genuinely a research-design decision.

But there is one distinction I think is essential:

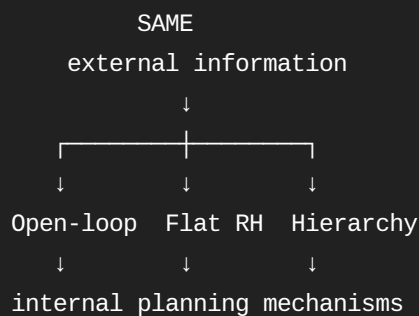
We should NOT require identical internal information.

That would be strange because the thing we're testing is precisely a different internal planning organization.

Instead, we should probably enforce:

**Equal external information access + controlled environment
feedback + controlled training information.**

Then:



This allows hierarchy to **generate subgoals**.

It does **not** allow hierarchy to receive ground-truth subgoals.

16. One particularly important question for you

Before we lock information parity, I want you to decide this specific point:

What should the planner receive at decision time?

We have three reasonable choices:

A. Current observation + final goal only

$$I_t = \{o_t, G\}$$

B. Current observation + final goal + fixed observation/action history

$$I_t = \{\text{history}, \text{actions}, G\}$$

C. Whatever common state representation the reproduced baseline already uses

Then we freeze that representation and give the **same representation** to all three planners.

I lean toward **C as the practical route**, because we first need to reproduce the flat baseline anyway, and your proposal explicitly makes baseline reproduction Stage 1 before freezing the main comparison.

bcd8906b-7787-479b-a0e8-68ccff5...

But I don't want to turn that into a decision until we inspect what the baseline actually uses.

So let's decide only this first: A, B, or C?

Sources

I think C makes the most sense. Let's first reproduce the baseline and inspect exactly what state/information it uses at decision time, then freeze that as the common representation for all three methods. I don't want to impose an artificial input format before we know what the baseline actually needs.

Exactly. I think **C is the right methodological choice**, and importantly, it preserves the baseline rather than designing an artificial input interface around our assumptions.

So let's mark this as a **provisional protocol decision**:

Information representation: Reproduce the flat baseline first. Inspect exactly what information/state is available to the baseline planner at each decision point. That baseline decision-time representation becomes the common information representation for the flat RH and hierarchical methods in the primary comparison.

This fits the proposal's staged workflow: baseline reproduction comes first, followed by implementation/validation of flat RH and hierarchical planning, with the main comparison happening afterward.

bcd8906b-7787-479b-a0e8-68ccff5...

What we should record during baseline reproduction

Rather than just saying "the baseline uses state s_t ," we should make a precise **information audit**:

Question	What we need to discover
Observation	What exactly does the planner receive?
History	Current frame only or multiple observations?
Actions	Are previous actions provided?
Goal	How is the task goal represented?
State	Raw observation, encoded state, or latent representation?
World model	What does it receive as input?
Environment feedback	When does the planner receive a new observation?
Planning context	What information is retained between planning steps?
Training information	What data/supervision produced the representation?
Privileged information	Is anything available that wouldn't exist at deployment?

Then we can turn the result into something explicit like:

$$I_t = f(o_{t-k:t}, a_{t-k:t-1}, g, \dots)$$

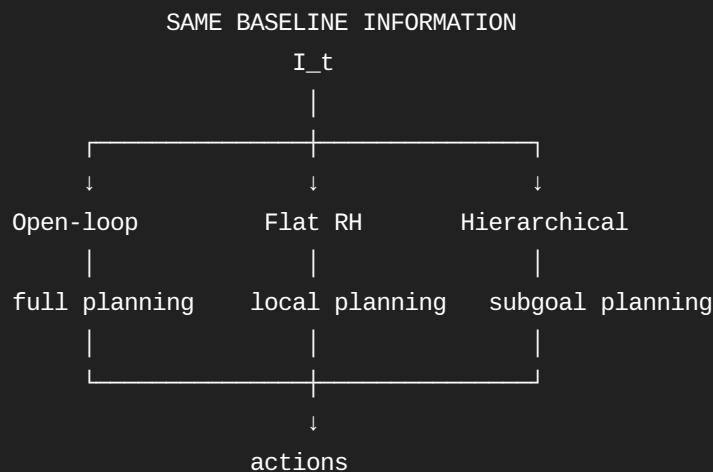
where I_t is the **frozen decision-time information representation**.

The exact contents of I_t should come from the reproduced baseline—not from us guessing beforehand.

One important safeguard

When we later give the same representation to the three methods, "**same representation**" **doesn't mean identical computation after receiving it**.

For example:



The hierarchical method is still allowed to **generate its own subgoals**. What it cannot do is receive *ground-truth* subgoals, future states, extra observations, or other privileged information that the baseline doesn't have.

That also keeps the oracle-subgoal experiment separate, as your proposal intends.

bcd8906b-7787-479b-a0e8-68ccff5...

So far, our protocol decisions are becoming:

#1 Task horizon

H = number of primitive environment action steps

Same overall H for all methods.

#2 Compute

Primary:

$$C = (N_{\text{WM}}, N_{\text{planner}})$$

with world-model forward passes and planner evaluations approximately matched. FLOPs, wall-clock time, parameter count, planning iterations, etc. are reported separately.

#3 Information parity — provisional

Baseline-derived common information representation.

We **do not freeze its contents yet**. We discover them during baseline reproduction and then freeze them before implementing the primary comparison.

That is a much stronger approach than arbitrarily deciding "current observation + goal" now.

The next question I would tackle is **what exactly counts as a planner evaluation**, because that interacts directly with the compute budget we just defined. We should resolve that before getting deep into implementation.

Sources

Yes, let's tackle planner evaluation next. I want to define exactly what counts as one planner evaluation in each method, especially since hierarchical planning has multiple levels. Let's compare the possible counting rules and see which one best matches the scientific question before we freeze it.

Yes. This is an important one to freeze carefully because **"planner evaluation" can mean several different things in code**, and hierarchical planning makes the ambiguity much worse.

Your proposal explicitly says the primary compute budget should count **world-model forward passes and planner evaluations**, while also reporting planning iterations, parameter count, wall-clock inference time, etc.

bcd8906b-7787-479b-a0e8-68ccff5...

The proposal, however, **does not define exactly what one planner evaluation is**, so this is a genuine protocol decision we need to make.

1. What are we actually trying to count?

Suppose a planner considers:

```
Candidate 1 → predict → score
Candidate 2 → predict → score
Candidate 3 → predict → score
...
Candidate N → predict → score
```

There are several possible things we could call an "evaluation":

1. One call to the planner function
2. One candidate trajectory/subgoal being scored
3. One search-tree node expanded
4. One complete planning iteration
5. One batch of candidates
6. One world-model rollout

These can produce **very different numbers** for exactly the same computation.

So we need a definition that reflects the scientific question rather than the implementation.

2. Rule A — Count planner function calls

Example:

```
Flat RH:
planner() called 8 times
→ 8 planner evaluations

Hierarchy:
high_level_planner() called 4 times
low_level_planner() called 4 times
→ 8 planner evaluations
```

Advantage

Extremely easy to log.

Problem

It is a terrible measure of computational usage.

Imagine:

```
Flat:
1 planner call
→ evaluates 1,000 candidates

Hierarchy:
1 planner call
→ evaluates 10 candidates
```

Both count as:

1

Yet their computational work is obviously not comparable.

Verdict

Not suitable as the primary definition.

3. Rule B — Count planning iterations

This would count each time a planner makes a decision.

For example, with $H = 80$:

Flat RH

```
t=0 → plan  
t=10 → replan  
t=20 → replan  
...
```

8 planning iterations.

Hierarchy

```
subgoal 1 → plan  
subgoal 2 → plan  
subgoal 3 → plan  
subgoal 4 → plan
```

4 high-level iterations plus perhaps 4 low-level iterations.

Again:

$$N_{\text{iterations}}$$

Advantage

Very interpretable.

Problem

One iteration can contain vastly different amounts of search.

For example:

Flat RH iteration:
1000 candidate trajectories

Hierarchical iteration:
50 candidate subgoals

Counting both as one evaluation hides the actual search effort.

Your proposal already treats **planning iterations as something to report separately**, which is a clue that they shouldn't be our primary planner-evaluation definition. `bcd8906b-7787-479b-a0e8-68ccff5...`

Verdict

Useful secondary metric, not primary.

4. Rule C — Count candidate plans evaluated

This is much more interesting.

Suppose flat RH evaluates:

100 candidate action sequences

at each of 8 replanning points.

Then:

$$N_{\text{planner}} = 100 \times 8 = 800$$

For hierarchy:

4 high-level stages

50 candidate subgoals per stage
50 candidate low-level plans per stage

Then:

$$N_{\text{planner}} = 4(50 + 50) = 400$$

So hierarchy has:

400

planner evaluations versus:

for flat RH.

That directly exposes the compute imbalance we discussed earlier.

Major advantage

It captures the amount of **search actually performed**.

Problem

Now we have to answer:

Is a high-level subgoal candidate equivalent to a low-level action-sequence candidate?

Not necessarily.

A high-level candidate might require:

```

subgoal
  ↓
world-model prediction
  ↓
low-level planning
  ↓
predicted outcome
  
```

while a low-level candidate might simply be:

```

action sequence
  ↓
world-model prediction
  
```

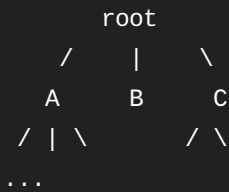
So equal candidate counts do **not necessarily mean equal computational cost**.

But this is exactly why we decided to report FLOPs and wall-clock separately.

5. Rule D — Count search-tree node expansions

This is more general.

If the planner uses tree search:



we could define:

$$N_{\text{planner}} = \text{number of expanded search nodes}$$

This is attractive if the implementation uses explicit search.

But there is a problem

The flat and hierarchical planners might use fundamentally different search structures.

For example:

Flat RH:
trajectory optimization

versus:

Hierarchy:
subgoal sampling + low-level search

There may not be a clean one-to-one notion of "node."

Verdict

Useful if both planners use the same search abstraction, but probably **too implementation-specific** for our primary scientific definition.

6. Rule E — Count world-model rollouts as planner evaluations

We could define:

Every time the planner asks the world model to evaluate a candidate future, that is one planner evaluation.

For example:

```
candidate trajectory
  ↓
world model rollout
  ↓
predicted outcome
  ↓
score
```

This is appealing because it connects planner work directly to model usage.

But there is a problem:

We already have a separate budget:

$$N_{\text{WM}}$$

So using world-model rollouts as our planner count would make the two compute metrics partially redundant.

And your proposal deliberately distinguishes **world-model forward passes** from **planner evaluations**. bcd8906b-7787-479b-a0e8-68ccff5...

Verdict

Don't use this as the definition.

7. The hierarchical problem

This is where we need to be particularly careful.

Imagine hierarchy does:

```
      High-level
     /   |   \
    g1  g2  g3
     ↓   ↓   ↓
  low-level low-level low-level
```

Suppose:

- 100 subgoals are considered
- 50 low-level trajectories are considered for each selected subgoal
- 4 high-level planning stages

Should we count:

Option 1

Only high-level candidates:

$$4 \times 100 = 400$$

Option 2

Only low-level candidates:

$$4 \times 50 = 200$$

Option 3

Both:

$$400 + 200 = 600$$

Option 4

Every nested evaluation recursively:

Potentially much larger.

I think **Option 3 is the most scientifically faithful to the proposal**, because the proposal asks us to control **total planner-evaluation counts**, not merely the number of high-level decisions.

8. Proposed definition: candidate-evaluation accounting

A possible protocol definition is:

One planner evaluation is one candidate planning object that is actually scored/compared by a planner during inference.

Then:

Flat RH

A candidate action trajectory:

$$\tau_i = (a_t, \dots, a_{t+K})$$

that is evaluated by the planner counts as **one planner evaluation**.

Hierarchical

A candidate subgoal:

g_i

evaluated by the high-level planner counts as **one planner evaluation**.

A candidate low-level trajectory:

τ_j

evaluated by the low-level planner counts as **one additional planner evaluation**.

Therefore:

$$N_{\text{planner}} = N_{\text{high-level}} + N_{\text{low-level}}$$

for the hierarchical method.

For flat RH:

$$N_{\text{planner}} = N_{\text{flat candidates}}$$

And open-loop:

$$N_{\text{planner}} = N_{\text{open-loop candidates}}$$

This is much more concrete.

9. But there's an important question about rejected candidates

Suppose the hierarchical planner generates:

100 subgoal candidates

but only evaluates 70 because 30 are rejected by a cheap feasibility filter.

Do we count:

A

100?

or

B

70?

I would argue **70**, if only 70 actually enter the planner's scoring/evaluation computation.

Otherwise we'd be counting hypothetical computation that didn't occur.

We should therefore distinguish:

$$N_{\text{generated}}$$

from

$$N_{\text{evaluated}}$$

and use **evaluated candidates** for the primary budget.

10. Another tricky case: batching

Suppose the code does:

Python

Run

```
planner.evaluate(  
    candidates[0:100]  
)
```

in one batched GPU operation.

Should that be:

Function-call counting

1

or:

Candidate counting

100

For our purposes, it should be:

100

because batching changes implementation efficiency, not the number of candidate evaluations.

Then wall-clock time/FLOPs can tell us whether one method benefits more from batching.

This is actually a nice separation:

Quantity	What it tells us
Candidate evaluations	Search/resource count
WM forward passes	Model usage
FLOPs	Approximate computational work
Wall-clock	Actual latency
Planning iterations	Decision frequency

That gives each metric a distinct purpose.

11. What about one candidate requiring multiple WM steps?

Suppose a candidate trajectory has 20 predicted steps:

```
candidate
↓
WM step 1
↓
WM step 2
↓
...
↓
WM step 20
```

We count:

1 planner evaluation

but:

20 WM forward passes

assuming one WM forward pass per predicted step.

That's exactly the separation we want.

Otherwise, we'd accidentally count the same computational work twice.

12. What about hierarchical nesting?

Suppose:

```
High-level candidate g1
      ↓
Low-level search
├── τ1
├── τ2
├── τ3
└── τ4
```

Then we'd count:

```
g1 = 1 high-level planner evaluation
τ1 = 1 low-level planner evaluation
τ2 = 1
τ3 = 1
τ4 = 1
```

Total:

$$N_{\text{planner}} = 5$$

If 100 subgoals and 50 low-level candidates per selected subgoal:

$$N_{\text{planner}} = 100 + 50$$

if only one subgoal is passed to low-level planning.

If all 100 subgoals trigger low-level evaluation:

$$N_{\text{planner}} = 100 + (100 \times 50)$$

That distinction must be logged explicitly.

13. A potentially important refinement: "actual evaluation" vs "candidate generation"

I think we should keep these separate.

For hierarchy:

$$N_{\text{generated}}^{HL}$$

$$N_{\text{evaluated}}^{HL}$$

$$N_{\text{generated}}^{LL}$$

$$N_{\text{evaluated}}^{LL}$$

Then:

$$N_{\text{planner}} = N_{\text{evaluated}}^{HL} + N_{\text{evaluated}}^{LL}$$

This prevents a clever implementation from hiding computation inside a generator or filter.

14. Compare the alternatives against our scientific question

Our question is essentially:

Does explicit hierarchical subgoal representation provide an additional reliability advantage over flat receding-horizon planning when local replanning and inference computation are controlled?

bcd8906b-7787-479b-a0e8-68ccff5...

So:

Counting rule	Controls actual search effort?	Works across hierarchy?	Matches proposal?
Planner function calls	✗	⚠	✗
Planning iterations	✗	⚠	Partial
Candidate evaluations	Yes	Yes	Yes
Search-tree nodes	Yes	⚠	Partial
WM rollouts	Already counted separately	Yes	✗
FLOPs	Very well	Yes	Secondary

This makes **candidate evaluations** the strongest candidate for the primary definition.

15. But there's one remaining scientific concern

There is still a problem:

One high-level subgoal evaluation may not cost the same as one low-level trajectory evaluation.

And we should **not pretend that it does**.

That's actually okay because we've already decided:

Primary budget

$$C = (N_{\text{WM}}, N_{\text{planner}})$$

Secondary computational audit

- FLOPs
- wall-clock inference time
- parameter count
- planning iterations
- environment interactions
- potentially memory/GPU usage

So the claim remains deliberately narrow:

Under approximately matched world-model forward-pass counts and planner-candidate evaluation counts...

not:

"Under exactly equal computational cost."

That distinction is scientifically important.

16. One more possibility: separate counts by level

I strongly recommend that we **never log only one total number**.

For hierarchy, we should record:

$$N_{\text{planner}}^{HL}$$

and

$$N_{\text{planner}}^{LL}$$

and report:

$$N_{\text{planner}}^{\text{total}} = N_{\text{planner}}^{HL} + N_{\text{planner}}^{LL}$$

For flat RH:

$$N_{\text{planner}}^{\text{flat}}$$

This lets us later say:

Hierarchy:

High-level evaluations = 400

Low-level evaluations = 400

Total = 800

rather than hiding the structure behind:

Planner evaluations = 800

That will be extremely useful when we analyze *why* hierarchy helped or failed.

17. My proposed candidate protocol

I would put this on the table for **your decision**, not lock it yet:

Planner evaluation: one candidate planning object that is actually scored or evaluated by a planning procedure during inference.

Then:

N_P^{flat} = number of flat candidate plans evaluated

$$N_P^{\text{hier}} = N_P^{HL} + N_P^{LL}$$

where high-level candidate subgoals and low-level candidate plans are counted separately and then summed.

Additional rules:

1. **Batching does not reduce the count.**
2. Only candidates actually evaluated count; merely generated/rejected candidates are logged separately.
3. Nested evaluations count individually.
4. World-model forward passes are counted separately.
5. Planning iterations are reported separately.
6. FLOPs and wall-clock time are reported separately.
7. High-level and low-level counts are always logged separately.
8. The primary comparison matches total N_P approximately, alongside N_{WM} .

This would give us a very auditable accounting system.

The decision for you

I see **two serious choices** remaining:

Option 1 — Candidate-level accounting:

Count every actually evaluated candidate, with hierarchy:

$$N_P = N_P^{HL} + N_P^{LL}$$

Option 2 — Level-normalized accounting:

Count high-level and low-level evaluations separately and require matching **each level's budget** where possible, rather than only matching their sum.

Option 2 is stricter, but it may become artificial because flat RH has no naturally corresponding "high-level" planner.

I would like you to decide between those two before we freeze the planner-evaluation definition.

Sources

I'd go with Option 1. I think matching the total candidate-evaluation budget is more defensible than trying to force a high-level/low-level correspondence that flat RH doesn't naturally have. But let's still log the HL and LL counts separately so we can audit where the search effort actually goes. With that, I think we can freeze the planner-evaluation definition.

Agreed. I think that is a clean and defensible choice.

Decision #4 — Planner-evaluation definition: FROZEN

We will use **candidate-level accounting**.

One planner evaluation = one candidate planning object that is actually scored/evaluated by a planning procedure during inference.

For the primary compute budget:

N_P = total number of candidate planning objects actually evaluated

Flat RH

Every evaluated candidate action trajectory counts as one:

$$N_P^{RH} = N_{\text{candidate trajectories evaluated}}$$

Hierarchical

Both levels contribute:

$$N_P^{Hier} = N_P^{HL} + N_P^{LL}$$

where:

- N_P^{HL} = evaluated candidate subgoals
- N_P^{LL} = evaluated candidate low-level plans

We **match the total**, not the individual levels.

For example:

Method	HL candidates	LL candidates	Total planner evaluations
Flat RH	—	—	800

Method	HL candidates	LL candidates	Total planner evaluations
Hierarchical	400	400	800

The **—** is intentional: flat RH doesn't have a natural high-level/low-level decomposition, so we won't manufacture one.

Accounting rules

We'll also freeze these:

1. Batched candidates still count individually.

A batch of 100 candidates = 100 planner evaluations, not 1.

2. Only actually evaluated candidates count toward N_P .

Candidates merely generated and rejected before evaluation are logged separately.

3. Nested evaluations count individually.

4. HL and LL counts are always logged separately even though their sum is used for the primary budget.

5. World-model calls remain a separate quantity:

$$N_{WM}$$

6. Planning iterations remain a separate quantity.

7. FLOPs and wall-clock inference time remain separate computational audits.

This preserves the distinction between the different resource measures rather than pretending that one metric captures all computation.

Our primary compute budget is now becoming precise

We previously chose:

$$C = (N_{WM}, N_P)$$

So the primary hierarchical-vs-flat-RH comparison will approximately satisfy:

$$N_{WM}^{Hier} \approx N_{WM}^{RH}$$

and

$$N_P^{Hier} \approx N_P^{RH}$$

while separately reporting:

{FLOPs, wall-clock time, parameters, planning iterations, environment interac

This is particularly consistent with the proposal's intention to match world-model usage and planner-evaluation counts while separately reporting other computational quantities.

bcd8906b-7787-479b-a0e8-68ccff5...

And importantly, our eventual scientific claim will stay appropriately narrow:

Under approximately matched world-model forward-pass and candidate-planner-evaluation counts, does explicit hierarchical planning provide an additional reliability advantage over flat receding-horizon planning?

We will **not** call this "equal total computational cost," because we've deliberately chosen not to equate one HL candidate with one LL candidate computationally.

Current frozen decisions

Decision	Status
Task horizon H = primitive environment action steps	 Frozen
Same overall H for all planners	 Frozen
Explore feasible H range before freezing final horizons	 Frozen
Primary compute = $N_{WM} + N_P$ accounting	 Frozen
Planner evaluation = actually evaluated candidate	 Frozen
Hierarchy $N_P = N_P^{HL} + N_P^{LL}$	 Frozen
Match total N_P , not HL/LL individually	 Frozen
Log HL and LL separately	 Frozen
FLOPs/wall-clock/etc. as secondary audit	 Frozen

Decision

Status

Information representation = reproduced baseline's representation

🔒 Provisional until baseline reproduction

So we've now got a pretty solid **H + C + information-parity foundation**.

The next subtle issue I'd examine is **replanning frequency / environment feedback**, because this is where flat RH can gain information simply by observing the environment more often, and hierarchy can accidentally gain the same advantage through its subgoal structure. That is closely connected to the proposal's planned sensitivity analysis around observation frequency and replanning updates.

bcd8906b-7787-479b-a0e8-68ccff5...

Sources

Yes, let's examine replanning frequency and environment feedback next. I want to separate three things clearly: how often the planner receives new observations, how often it replans, and the internal planning horizon. Let's compare the possible ways to control these across methods before deciding what belongs in the primary comparison versus the sensitivity analysis.

The proposal explicitly calls for sensitivity analysis over **observation frequency and replanning updates**, while the main comparison is meant to isolate hierarchy from local replanning.

bcd8906b-7787-479b-a0e8-68ccff5...

The important thing is that these are **not the same variable**.

1. Separate the three concepts

Let's define them conceptually first.

A. Observation frequency — F_{obs}

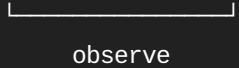
How often does the planner receive a **new environment observation**?

Example:

$$F_{obs} = 5$$

means the method receives an updated observation every 5 primitive environment actions.

Environment:

t=0 — 1 — 2 — 3 — 4 — 5 — 6 — ...

 observe

This controls **information feedback**.

B. Replanning frequency — F_{replan}

How often does the method actually **invoke its planning procedure again**?

Example:

$$F_{replan} = 10$$

means:

observe at t=5
 observe at t=10 → REPLAN
 observe at t=15
 observe at t=20 → REPLAN

The planner could receive observations without replanning every time.

That distinction matters.

C. Internal planning horizon — L

When the planner is invoked, how far ahead does it reason?

For example:

$$L = 20$$

means a planning call predicts/optimizes over the next 20 primitive action steps.

This is **not the overall task horizon** H .

So:

$$H \neq L$$

and neither is necessarily equal to observation or replanning interval.

2. Why these must be separated

Consider:

$$H = 80$$

A flat RH method could have:

$$L = 20, \quad F_{replan} = 10, \quad F_{obs} = 10$$

while hierarchy could have:

$$L_{HL} = 80, \quad L_{LL} = 10, \quad F_{replan} = 5, \quad F_{obs} = 5$$

If hierarchy performs better, we wouldn't know whether the cause was:

- hierarchical representation,
- more frequent observations,
- more frequent replanning,
- different internal horizon,
- or some combination.

That's exactly the confound we want to eliminate.

3. Option A — Match all three across methods

We could require:

$$F_{obs}^{RH} = F_{obs}^{Hier}$$

$$F_{replan}^{RH} = F_{replan}^{Hier}$$

and perhaps:

$$L^{RH} = L^{Hier}$$

Advantage

Very clean.

Everything concerning temporal feedback and planning cadence is controlled.

Problem

The last condition may actually **suppress the phenomenon we're trying to study**.

Hierarchy inherently has multiple levels. It may naturally have:

$$L_{HL} \neq L_{LL}$$

Trying to force one common internal planning horizon could make the hierarchical algorithm artificially unnatural.

So I would distinguish **external timing variables** from **internal architectural variables**.

4. Option B — Match observation and replanning, allow internal horizons to differ

This is more interesting.

Primary comparison:

$$F_{obs}^{RH} = F_{obs}^{Hier}$$
$$F_{replan}^{RH} = F_{replan}^{Hier}$$

but:

$$L^{RH}$$

and

$$(L_{HL}^{Hier}, L_{LL}^{Hier})$$

can differ according to each method's planning mechanism.

This asks:

Given the same environmental feedback opportunities and comparable replanning opportunities, does hierarchical organization itself improve reliability?

That seems much closer to your research question.

Your proposal specifically says the killer alternative is that hierarchy may merely reduce effective planning horizon, while flat RH can also replan

frequently.

bcd8906b-7787-479b-a0e8-68ccff5...

So allowing the internal horizons to represent the methods naturally, while controlling external feedback, seems scientifically attractive.

5. Option C — Match observation frequency, but let replanning frequency differ

For example:

Observation:

both → every 5 actions

Flat RH:

replans every 10 actions

Hierarchy:

replans whenever subgoal is completed

This reflects how algorithms might naturally operate.

Problem

Now hierarchy may effectively get more planning opportunities.

Suppose:

$$H = 80$$

Flat RH:

8 planning calls

Hierarchy:

16 planning calls

Even if each call has the same candidate budget, hierarchy gets twice as many opportunities to correct itself.

We could compensate through the compute budget C , but then we're mixing:

hierarchy vs more frequent replanning.

That weakens the primary causal interpretation.

6. Option D — Match replanning frequency, but not observation frequency

This is the opposite.

For example:

Flat RH:
observe every 10 → replan every 10

Hierarchy:
observe every 5 → replan every 10

Now both replan equally often, but hierarchy gets more recent environmental information before each planning event.

Again, that is a confound.

So I would **not** use this for the primary comparison.

7. Option E — Treat observation and replanning as the same thing

This is common in simple implementations:

```
execute K actions
    ↓
observe
    ↓
replan
    ↓
execute K actions
```

So:

$$F_{obs} = F_{replan}$$

Advantage

Simple.

Problem

It prevents us from answering an important question:

Does receiving information more frequently help, independently of actually invoking the planner?

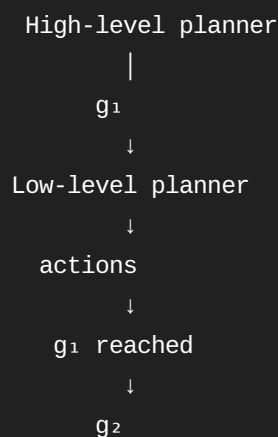
Since your proposal explicitly mentions varying observation frequency and replanning updates, collapsing them would lose an important experimental dimension.

bcd8906b-7787-479b-a0e8-68ccff5...

I think we should keep them conceptually separate even if the baseline happens to make them equal.

8. The hierarchy-specific complication

Suppose the hierarchy works like this:



When g_1 is reached, the high-level planner may naturally produce g_2 .

Is that a **replanning event**?

I would say we need two notions:

External replanning

The method receives new environmental information and invokes its planning procedure.

Internal hierarchical transition

The low-level controller reaches the current subgoal and the hierarchy advances to the next subgoal.

Those are not necessarily equivalent.

Otherwise we'd accidentally define:

"Every subgoal transition = extra replanning"

and make hierarchy look as though it has a different environmental feedback schedule.

9. A cleaner model

I'd represent the process as:

$$F_{obs}$$

controls:

When does new environment information arrive?

Then:

$$F_{replan}$$

controls:

When can the planner make a new planning decision?

Then:

$$L$$

controls:

How far ahead does each planning mechanism reason?

And for hierarchy:

$$L_{HL}, L_{LL}$$

can be separately recorded.

So we'd have:

Environment

|

| observations

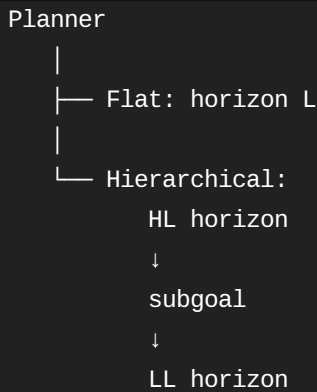
↓

Information state

|

| replanning trigger

↓



This is much more precise.

10. What should belong in the PRIMARY comparison?

I see two serious philosophies.

Primary Strategy 1 — Strictly match feedback + replanning

For flat RH and hierarchy:

$$F_{obs}^{RH} = F_{obs}^{Hier}$$
$$F_{replan}^{RH} = F_{replan}^{Hier}$$

while their internal planning horizons remain method-specific.

Then the primary comparison is:

Same task horizon + same environmental feedback + same replanning cadence + matched compute → does hierarchy provide additional reliability?

This is a strong isolation of the hierarchy effect.

Primary Strategy 2 — Match only total compute, allow natural replanning

Here hierarchy and flat RH use their natural replanning schedules, but the total planner evaluations and WM calls are matched.

Then the question becomes:

Under matched computational resources, does the complete hierarchical algorithm outperform the complete flat RH algorithm?

This is arguably more practical, but it allows replanning frequency to become part of the algorithmic advantage.

That makes causal interpretation weaker.

11. What should probably be SENSITIVITY analysis?

This is where I think your proposal gives us a useful direction.

We could make the primary experiment use a **fixed, matched feedback/replanning regime**, then explore:

Sensitivity A — Observation frequency

$$F_{obs} \in \{f_1, f_2, f_3\}$$

while keeping other factors controlled.

Question:

Does the hierarchy advantage survive when environmental feedback becomes less/more frequent?

Sensitivity B — Replanning frequency

$$F_{replan} \in \{r_1, r_2, r_3\}$$

Question:

Is hierarchy still useful when flat RH gets more frequent opportunities to correct itself?

Sensitivity C — Internal planning horizon

Vary:

$$L$$

for flat RH and corresponding hierarchical horizons.

Question:

Is the apparent hierarchy advantage simply caused by effective planning horizon differences?

These are much more informative than putting everything into one giant uncontrolled experiment.

12. There's a particularly important experiment hiding here

The proposal's central "killer alternative" is:

Maybe hierarchy only helps because it reduces effective planning horizon.

bcd8906b-7787-479b-a0e8-68ccff5...

We can directly attack that alternative.

Imagine:

Flat RH

$$L = 20, \quad R = 10$$

Hierarchy

$$L_{HL} = 80, \quad L_{LL} = 10$$

If flat RH gets increasingly frequent replanning:

$$R = 10 \rightarrow 5 \rightarrow 2$$

and its performance approaches hierarchy, that's important evidence.

Conversely, if hierarchy maintains an advantage even when flat RH gets frequent replanning **under matched compute**, that is much more interesting for the hypothesis.

We shouldn't interpret that result in advance, but this is exactly the kind of discriminating experiment the proposal is designed around.

13. My current candidate for the protocol

I would **not freeze this yet**, but I'd put this on the table:

Primary

Control:

$$H$$

$$F_{obs}$$

$$F_{replan}$$

$$N_{WM}$$

$$N_P$$

and allow the internal planning horizon to remain method-specific:

$$L_{RH}$$

versus

$$(L_{HL}, L_{LL})$$

with all of those logged explicitly.

Secondary/sensitivity

Systematically vary:

1. observation frequency
2. replanning frequency
3. internal planning horizons

while preserving the compute-accounting framework.

This would let us distinguish:

External feedback



Observation frequency

Planning opportunity



Replanning frequency

Internal lookahead



Planning horizon

Computational resource



$N_{WM} + N_P$

rather than allowing them to collapse into one vague "planning horizon" variable.

One decision remains

The key choice is therefore:

For the primary comparison, should we lock observation frequency and replanning frequency to be the same across Flat RH and Hierarchical, while allowing each method's internal planning horizon to be method-specific?

Or do you think replanning frequency itself should remain method-specific and be controlled only through the matched compute budget?

I would treat these as genuinely different scientific questions rather than assuming one is automatically correct.

Sources

I think the first strategy makes more sense for the primary comparison. Let's match observation frequency and external replanning frequency between Flat RH and Hierarchical, while allowing their internal planning horizons to remain method-specific. That isolates the hierarchy mechanism better. We can then vary observation frequency, replanning frequency, and internal horizons in sensitivity experiments. I also don't think we should choose the actual frequency values yet—we can determine feasible values during baseline reproduction and then freeze them.

Yes. I agree with that decision, and it gives us a much cleaner causal comparison.

Decision #5 — Feedback and replanning: FROZEN

For the **primary Flat RH vs Hierarchical comparison**:

1. Observation frequency is matched

$$F_{\text{obs}}^{RH} = F_{\text{obs}}^{Hier}$$

Both methods receive new environment observations at the same frequency.

2. External replanning frequency is matched

$$F_{\text{replan}}^{RH} = F_{\text{replan}}^{Hier}$$

Both methods get the same number/opportunities for external replanning over the same task horizon.

3. Internal planning horizons remain method-specific

Flat RH can have:

$$L_{RH}$$

while hierarchy can have:

$$L_{HL}, L_{LL}$$

We **do not force these to be equal**, because that would potentially distort the actual hierarchical mechanism we're trying to study.

4. The numerical values are NOT frozen yet

During baseline reproduction, we'll determine a feasible range of:

$$F_{\text{obs}}, F_{\text{replan}}, L$$

Then we'll select the actual primary values using a **method-independent rule** and freeze them **before the primary comparison**.

This is consistent with the proposal's existing approach of exploring feasible horizons during baseline reproduction and then fixing the main experimental settings before comparison.

bcd8906b-7787-479b-a0e8-68ccff5...

One subtle distinction we should preserve

We should call the matched quantity **external replanning frequency**.

Because hierarchy can internally transition:

HL subgoal 1

↓

```
LL execution
  ↓
subgoal reached
  ↓
HL subgoal 2
```

That internal transition doesn't automatically mean it received a new environmental observation or was given an additional external replanning opportunity.

So our logs should distinguish:

$N_{\text{external replans}}$

from:

$N_{\text{HL transitions}}$

and:

$N_{\text{LL planning calls}}$

That prevents hierarchy's internal structure from being confused with its environmental feedback schedule.

Why I like this primary design

We now have several potential explanations controlled independently:

Potential explanation	Primary control
Different overall task difficulty	Same H
Different external observations	Same F_{obs}
Different external replanning opportunities	Same F_{replan}
Different WM usage	Match N_{WM}
Different search effort	Match N_P
Different internal organization	Allowed to differ
Different internal planning horizon	Allowed to differ, explicitly logged

Potential explanation

Primary control

Different HL/LL search allocation

Logged separately

That gets us much closer to the actual scientific question:

When external feedback, external replanning opportunities, and measured inference resources are controlled, does explicitly organizing planning around hierarchical subgoals provide an additional reliability advantage over flat receding-horizon planning?

That's a substantially cleaner test of the hypothesis than simply comparing two implementations.

Current protocol status

We now have:

#1 — Task horizon 🔒

H = primitive environment action steps; same H across methods.

#2 — Compute budget 🔒

$$C = (N_{WM}, N_P)$$

#3 — Information parity 🔒*

Baseline-derived decision-time representation becomes common to all methods.

*Exact representation waits for baseline reproduction.

#4 — Planner evaluation 🔒

One actually evaluated candidate planning object = one planner evaluation.

Hierarchical:

$$N_P = N_P^{HL} + N_P^{LL}$$

#5 — Feedback/replanning 🔒

Same external observation frequency and external replanning frequency between Flat RH and Hierarchical; internal horizons remain method-specific.

Numerical values for #1/#5: not yet frozen. They'll be established from baseline feasibility and then frozen before the primary experiment.

The next decision I think we should examine is **model parity**: whether Flat RH and Hierarchical should use exactly the same world model, how much additional capacity hierarchy is allowed to have for its subgoal mechanism, and how we prevent a larger hierarchical model from becoming the hidden reason it performs better. This connects directly to the proposal's requirement to control model capacity and training data.

bcd8906b-7787-479b-a0e8-68ccff5...

Sources

Yes, let's examine model parity next. I want to separate the shared world model itself from any additional learned components needed for hierarchy, and understand what the proposal actually requires us to control regarding model capacity and training data. Let's compare the possible designs before deciding what should be shared, what can differ, and what needs to be reported.

Yes. This is probably the most important parity decision after compute, because if the hierarchical method simply has a better/larger learned model, we could mistake **model capacity** for a **hierarchy effect**.

Your proposal explicitly says the comparison should control **training data**, **model capacity**, and **inference computation**, while reporting parameter count and other computational quantities separately.

bcd8906b-7787-479b-a0e8-68ccff5...

It also wants prediction error separated from control/planning error.

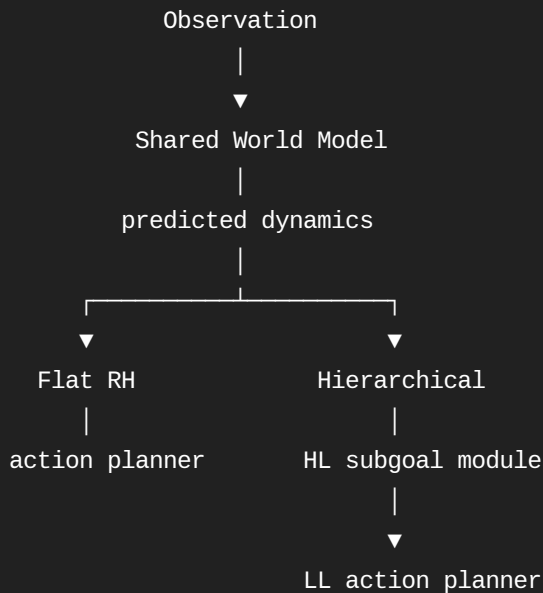
bcd8906b-7787-479b-a0e8-68ccff5...

Let's separate the problem into three pieces.

1. First distinction: shared world model vs hierarchy-specific components

I think this distinction is essential.

Suppose we have:



There are really two kinds of learned components:

A. World-model components

They answer:

"Given the current state/action, what happens next?"

B. Planning components

They answer:

"Given the predicted future, what should I do?"

The hierarchy may additionally need:

- a high-level subgoal predictor,
- a subgoal scoring function,
- a subgoal representation,
- a high-level policy/search mechanism.

Those are not necessarily part of the world model.

So we shouldn't automatically demand that **every parameter** be identical.

2. What does the proposal actually require?

The proposal says the comparison should control:

comparable training data, model capacity, inference computation, and planning horizons.

bcd8906b-7787-479b-a0e8-68ccff5...

It also says the study should report:

- parameter count,
- model evaluations,
- planning iterations,
- wall-clock inference time,
- environment interactions.

bcd8906b-7787-479b-a0e8-68ccff5...

But importantly:

The proposal does not specify exactly how model capacity must be matched.

It does not say, for example:

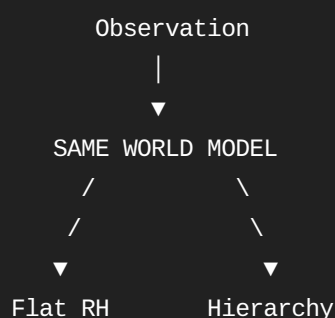
"The hierarchical model must have exactly the same number of parameters."

So this is something we need to operationalize ourselves.

That's an important distinction: the proposal establishes the **scientific control requirement**, but not the exact implementation rule.

3. Design A — Completely shared world model

The strongest sharing design would be:



Both methods use the **exact same trained world model**.

Same:

- architecture
- parameters
- training data
- training procedure
- checkpoint

- learned latent representation

Then the difference occurs entirely in planning.

Scientific advantage

Extremely clean.

If:

$$Performance_{Hier} > Performance_{RH}$$

then it cannot be because hierarchy learned a better world model.

It directly isolates the planning mechanism much better.

Potential downside

If the hierarchical method genuinely requires a specialized representation that the flat world model doesn't provide, forcing everything through exactly the same representation could constrain the hierarchy.

But that's a separate scientific question.

4. Design B — Separate world models, matched capacity

We could instead have:

Observation

|

├─ World Model RH

|

└─ World Model Hierarchy

with approximately matched:

$$parameters_{RH} \approx parameters_{Hier}$$

and same training data.

Advantage

Each method can adapt its representation to its planning architecture.

Problem

Suppose hierarchy wins.

We then have two possible explanations:

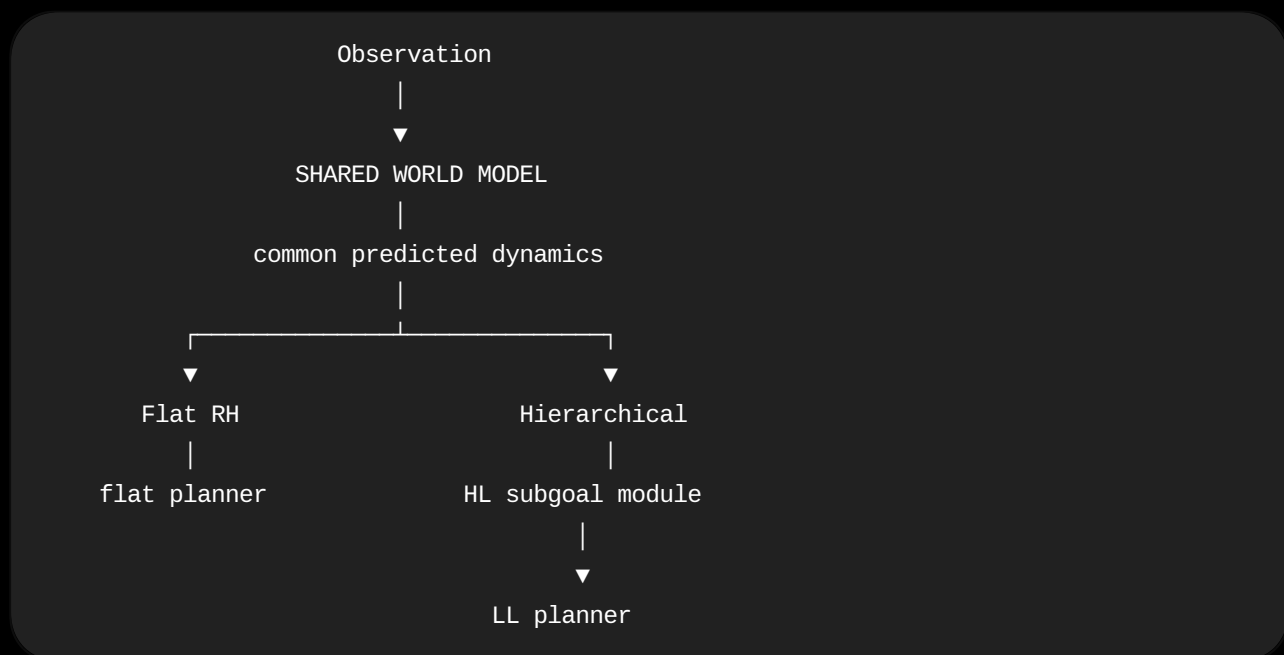
1. hierarchy is genuinely better;
2. hierarchical world model learned better representations.

Even if parameter counts are identical, that ambiguity remains.

So this is scientifically weaker for our specific question.

5. Design C — Shared world model + separate planning modules

This is more interesting.



Here:

Shared

- world-model architecture
- world-model parameters
- training data
- world-model checkpoint
- environment/task information
- state representation

Different

- flat planning mechanism
- high-level subgoal mechanism
- low-level hierarchical planning mechanism

This seems to map very naturally onto the research question.

We aren't asking:

"Can we build a better complete model?"

We're asking:

"Given the same learned predictive model, does organizing planning hierarchically provide additional reliability?"

6. But now we encounter model capacity

Suppose the shared world model has:

$10M$

parameters.

Flat planner:

$1M$

additional parameters.

Hierarchy:

$1M_{HL} + 1M_{LL}$

additional parameters.

Then:

Flat total:

11M

Hierarchy total:

12M

Hierarchy has more parameters.

Does that violate model parity?

Not necessarily—but we **must decide what claim we're making**.

7. Two meanings of "model capacity"

This is a crucial distinction.

Capacity of the world model

$$P_{WM}$$

How much learned capacity is devoted to modeling environment dynamics?

Capacity of the complete planning system

$$P_{total} = P_{WM} + P_{planner}$$

These aren't the same.

If our scientific question is specifically about **hierarchical planning**, I think the world-model capacity should be held exactly constant.

But the hierarchy-specific planner may need additional parameters.

The question becomes:

Do we force total parameter counts to be equal too?

8. Option 1 — Equal total parameter count

We could require:

$$P_{total}^{Hier} \approx P_{total}^{RH}$$

So if hierarchy adds a high-level network, we reduce capacity somewhere else.

Advantage

Very strict capacity matching.

Problem

It can become artificial.

Imagine:

Flat:
large flat planner

If we force total parameters to be identical, we may end up artificially shrinking one architecture.

Then we're no longer necessarily comparing natural versions of the two planning mechanisms.

9. Option 2 — Same world-model capacity, report total planner capacity

This would mean:

$$P_{WM}^{Hier} = P_{WM}^{RH}$$

but:

$$P_{planner}^{Hier}$$

can differ from

$$P_{planner}^{RH}$$

provided we report both.

For example:

Component	Flat RH	Hierarchical
World model	10M	10M
Planner	1M	2M
Total	11M	12M

Then we explicitly say:

World-model capacity was held constant; planning-module capacity differed and was measured/reported.

This is much more transparent than pretending the systems have equal capacity.

10. Option 3 — Shared world model + matched planning capacity

A stricter version of C would be:

$$P_{WM}^{RH} = P_{WM}^{Hier}$$

and approximately:

$$P_{planner}^{RH} \approx P_{planner}^{Hier}$$

This gives us:

same WM
+
similar planner capacity

Advantage

Strong isolation.

Problem

Again, defining "planner capacity" across fundamentally different architectures can be difficult.

A 1M-parameter MLP subgoal network isn't necessarily equivalent to a 1M-parameter trajectory optimizer.

So parameter equality does not necessarily mean functional capacity equality.

11. Training data is another separate issue

We should not accidentally combine:

model parity

with

training-data parity.

Your proposal explicitly wants comparable training data.

bcd8906b-7787-479b-a0e8-68ccff5...

For the shared-world-model design, this becomes very clean:

$$D_{WM}^{RH} = D_{WM}^{Hier}$$

because there is literally one shared model.

But what about the hierarchical subgoal module?

Suppose hierarchy receives extra supervision:

```
trajectory
  ↓
ground-truth intermediate states
  ↓
HL training
```

while flat RH only receives:

```
trajectory
  ↓
action training
```

Now the hierarchy has additional training information.

That could become another source of advantage.

12. Three possibilities for hierarchy-specific training

A. Same raw trajectories, derive subgoals automatically

```
Same dataset
  |
  ├── Flat planner training
  |
  └── Hierarchy training
      ↓
    subgoals derived automatically
```

This gives strong training-data parity.

B. Hierarchy gets manually/algorithmically annotated subgoals

```
Same trajectories
  +
subgoal annotations
```

Now hierarchy receives extra supervision.

That doesn't automatically make the experiment invalid, but it changes the question.

C. Oracle subgoals during training

This is even more explicitly privileged.

This should be treated similarly to the proposal's oracle-subgoal diagnostic rather than silently included in the primary experiment. The proposal explicitly says oracle subgoals are a diagnostic upper bound and privileged information must be reported.

bcd8906b-7787-479b-a0e8-68ccff5...

13. I think we need two different parity layers

Rather than one giant "model parity" rule, I'd separate:

Layer 1 — Predictive-model parity

Control strictly:

$$WM_{RH} = WM_{Hier}$$

Same:

- architecture
- parameters
- training data
- training procedure
- checkpoint
- learned state representation

Layer 2 — Planning-module parity

Don't necessarily require identical architecture.

Instead:

- document architecture,
- parameter count,
- training data,
- training supervision,

- inference computation,
- planner evaluations,
- FLOPs,
- wall-clock time.

That gives us a transparent comparison.

14. Why this fits the scientific question

Remember the central question:

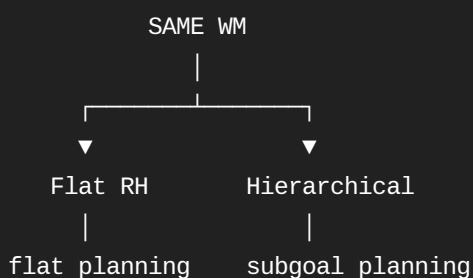
Does explicit hierarchical subgoal representation provide an additional reliability advantage over flat receding-horizon planning, after local replanning and inference computation are controlled?

bcd8906b-7787-479b-a0e8-68ccff5...

If both methods use:

SAME WORLD MODEL
 SAME TRAINING DATA
 SAME INFORMATION
 SAME OBSERVATION FREQUENCY
 SAME EXTERNAL REPLANNING FREQUENCY
 MATCHED N_WM
 MATCHED N_P

and only the planning organization differs:



then a performance difference becomes much easier to interpret.

It still isn't mathematically "proof" that hierarchy caused it, but we've removed many obvious alternative explanations.

15. What should we report?

Regardless of which design we choose, I think the following should be logged.

World model

$$P_{WM}$$

- architecture
- parameter count
- training data
- training procedure
- checkpoint
- prediction metrics

Flat planner

$$P_{Flat}$$

- planner parameters
- architecture
- candidate evaluations
- WM evaluations
- planning iterations
- FLOPs
- wall-clock inference

Hierarchy

$$P_{HL}$$

$$P_{LL}$$

and:

$$P_{Hier,total} = P_{WM} + P_{HL} + P_{LL}$$

plus:

- HL candidate evaluations
- LL candidate evaluations
- total candidate evaluations
- WM evaluations
- planning iterations
- FLOPs
- wall-clock inference

This makes it possible to see whether an apparent result depends on how we account for resources.

16. My strongest candidate for the primary design

I see three serious choices:

Design	World model	Planning modules	Main interpretation
A	Separate but capacity-matched	Separate	Complete systems comparison
B	Exactly shared	Separate	Isolates planning mechanism
C	Shared	Capacity-matched	Stronger total-capacity control

For **your specific research question**, I think **B deserves serious consideration**:

Use exactly the same trained world model for Flat RH and Hierarchical. Allow the planning mechanisms to differ as required by their algorithms, but fully report the additional planning-module capacity and computational costs.

That gives us a clean experimental interpretation without pretending that two fundamentally different planners have identical representational capacity.

But I would **not freeze B yet** until you decide how strict you want to be about the hierarchy-specific learned components.

The key choice for you is:

Should the primary experiment use one exactly shared world model, with hierarchy-specific learned planning components allowed to differ in architecture/parameter count but fully reported?

Or do you want to additionally require approximately equal **total planning-module parameter capacity** between Flat RH and Hierarchical?

That is the next scientific choice I'd make before locking model parity.

Sources

I think Design B is the better fit. Let's use exactly the same trained world model for Flat RH and Hierarchical, while allowing their planning modules to differ as required by the respective algorithms. I don't think we should artificially force equal planner parameter counts, since parameter equality doesn't imply equivalent capacity across different architectures. Instead, let's fully report planner parameters, training supervision, FLOPs, wall-clock time, and the other resource measures. Let's freeze that as our model-parity principle.

Agreed. This is a strong model-parity principle, and it follows the proposal's intent without imposing an artificial definition of “equal capacity.”

Decision #6 — Model parity: FROZEN

Shared world model

Flat RH and Hierarchical will use **exactly the same trained world model**:

$$WM_{RH} = WM_{Hier}$$

That means the primary comparison shares:

- same world-model architecture
- same parameters/checkpoint
- same training data
- same training procedure
- same learned state representation
- same world-model inference mechanism

So we eliminate the possibility that one method wins simply because it learned a better predictive model.

This directly supports the proposal's requirement to control model capacity and training data.

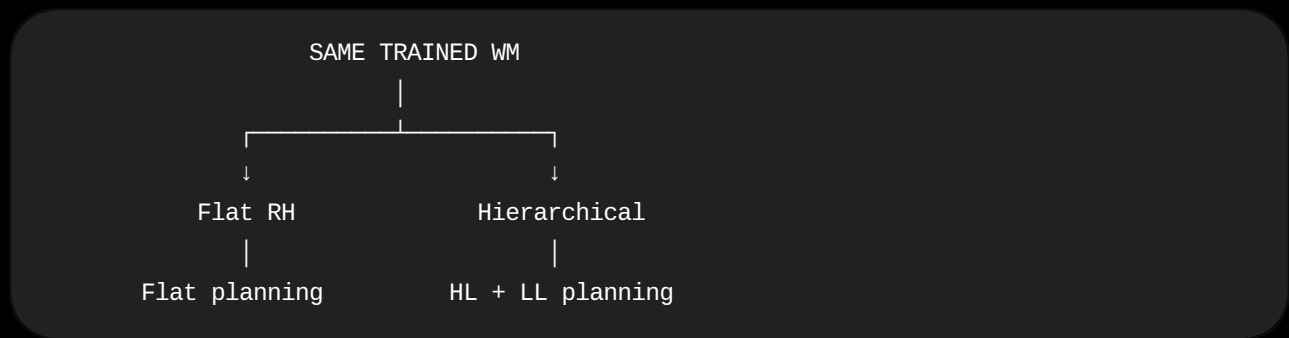
bcd8906b-7787-479b-a0e8-68ccff5...

Planning modules may differ

We will **not** artificially require:

$$P_{planner}^{RH} = P_{planner}^{Hier}$$

Instead:



The planning mechanisms can have whatever architecture is genuinely required by their respective algorithms.

That is important because:

Equal parameter count $\neq$ equal functional capacity.

Forcing both planners to have, say, exactly 1M parameters could actually introduce an artificial constraint rather than improve fairness.

What we will report

For every primary method, we'll report planner-specific resources, including:

Architecture

- planner architecture
- HL architecture
- LL architecture where applicable

Parameters

$$P_{WM}$$

$$P_{planner}$$

For hierarchy:

$$P_{HL}, \quad P_{LL}$$

and total parameters.

Training

- training dataset

- training supervision
- whether subgoal annotations are used
- training procedure
- relevant hyperparameters

Inference

- N_{WM}
- N_P
- N_P^{HL}
- N_P^{LL}
- planning iterations
- FLOPs
- wall-clock inference time
- environment interactions
- other relevant resource usage

This is especially important because the proposal already asks us to report parameter count, model evaluations, planning iterations, wall-clock inference time, and environment interactions.

bcd8906b-7787-479b-a0e8-68ccff5...

One important consequence

If hierarchy eventually has:

$$P_{Hier} > P_{RH}$$

that **doesn't invalidate the experiment**.

Instead, our claim has to remain precise:

The hierarchical method provided an advantage under the matched world-model, information, feedback, and primary inference-resource controls, while its planning-module capacity and other computational characteristics were explicitly measured and reported.

We should **not** conclude:

"Hierarchy is better despite having the same computational/model capacity."

because we aren't enforcing equal total planner parameter counts.

That distinction will keep the paper honest.

Current protocol ledger

We're now at a fairly coherent point:

Decision	Primary rule	Status
Task horizon H	Primitive environment action steps	
Overall H	Same across methods	
Information representation	Baseline-derived common representation	 *
World model	Exactly shared trained WM	
Planner architecture	Method-specific	
Planner parameters	Not artificially equalized; fully reported	
Planner evaluation	One actually evaluated candidate	
Hierarchy planner count	$N_P^{HL} + N_P^{LL}$	
Observation frequency	Matched RH ↔ Hierarchical	
External replanning frequency	Matched RH ↔ Hierarchical	
Internal planning horizon	Method-specific, explicitly reported	
N_{WM}	Approximately matched	
N_P	Approximately matched	
FLOPs/time/etc.	Secondary computational audit	
Exact numerical H/frequencies	Determine during baseline reproduction, then freeze	

* The exact baseline-derived representation isn't known until reproduction.

This is now a much stronger protocol than simply saying “*compare Flat RH and Hierarchical fairly.*” We're progressively defining **what fair means in**

measurable terms.

The next issue I'd examine is **training parity and supervision**, especially the hierarchy's subgoal-generation training. That's where we could accidentally give hierarchy extra labeled information even though we're using the exact same world model. The proposal requires comparable training data, so we should decide exactly what “comparable” means for the planning modules before implementation.

bcd8906b-7787-479b-a0e8-68ccff5...

Sources

Yes, let's examine training parity and supervision next. I especially want to understand what training information the hierarchical subgoal module needs, whether it can be derived from the same data as Flat RH, and when extra subgoal supervision would become a confound. Let's compare the options before we decide what the primary experiment should allow.

Yes. This is an important distinction because **training-data parity does not necessarily mean identical training targets**.

Your proposal requires comparable training data and identifies oracle/noisy subgoals as separate diagnostic experiments, but it does not yet specify exactly how the hierarchical subgoal module should be trained.

bcd8906b-7787-479b-a0e8-68ccff5... +1

Let's separate the possibilities carefully.

1. What exactly are we trying to control?

There are three different things:

Dataset

What trajectories/examples does the method see?

D

Supervision

What labels/targets are provided during training?

$$Y$$

Architecture

What does the method learn from those examples?

So two methods could have:

$$D_{RH} = D_{Hier}$$

but:

$$Y_{RH} \neq Y_{Hier}$$

That is where the confound can appear.

2. What does Flat RH naturally need?

The flat planner may need training data for the shared world model, but depending on the baseline implementation it might also learn a planning/policy component.

Conceptually:

```
trajectory
  ↓
world-model training
  ↓
predictive model
  ↓
flat planning
```

There isn't necessarily a requirement for intermediate subgoal labels.

3. What does Hierarchical need?

A hierarchical planner needs some way of producing:

$$g_t = \text{subgoal}$$

There are several ways to obtain that.

Option A — Derive subgoals from the same trajectories

Suppose the training data contains:

$s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_{80}$

We can algorithmically derive intermediate states:

$s_0 \rightarrow s_{20} \rightarrow s_{40} \rightarrow s_{60} \rightarrow s_{80}$

and define these as training targets.

No additional dataset is required.

The important distinction is:

The hierarchy gets a different **training target derived from the same underlying trajectory**, rather than an externally supplied label.

This is potentially very clean.

4. Option B — Learn subgoals without explicit labels

Another possibility is that the hierarchical module discovers useful subgoals through an objective.

For example, it could optimize something like:

$$\mathcal{L}_{hier} = \mathcal{L}_{task} + \lambda \mathcal{L}_{subgoal}$$

where the subgoal structure emerges from the training process rather than being explicitly labeled.

Advantage

No human/ground-truth intermediate-state annotation.

Problem

Now the learning procedure itself becomes substantially different from Flat RH.

We'd need to document the objective and make sure the extra optimization isn't giving hierarchy an unfair amount of training computation or information.

5. Option C — Ground-truth subgoal supervision

Suppose we explicitly give hierarchy:

```
trajectory:
s0 → s1 → ... → s80

labels:
g1 = s20
g2 = s40
g3 = s60
```

and train:

$$\hat{g}_i = f_{\theta}(s_t)$$

to reproduce those labels.

This is much more problematic for the **primary experiment**.

Why?

Because Flat RH doesn't receive equivalent intermediate-state supervision.

Then hierarchy has access during training to information that Flat RH does not.

Even if both ultimately receive exactly the same information at inference time, the hierarchy has been trained with privileged structural information.

6. When does that become a confound?

I'd draw the line here:

Same raw data, different derived target

Potentially acceptable.

```
Same trajectories
|
├── Flat training target
|
└── Hierarchical target derived algorithmically
```

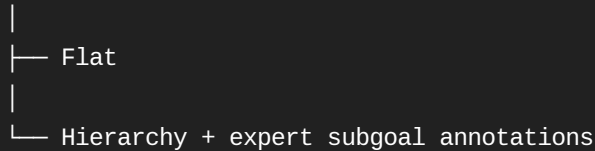
provided the derivation uses **only information available in the shared training data** and doesn't introduce privileged future/environment

information.

Extra externally supplied information

Potential confound.

Same trajectories



Now hierarchy has received additional supervision.

Ground-truth future states unavailable to the flat method

Definitely needs special treatment.

This starts resembling the proposal's **oracle subgoal** condition, which the proposal explicitly says should be a diagnostic upper bound with privileged information clearly reported.

bcd8906b-7787-479b-a0e8-68ccff5...

7. But there's a subtle issue with "same data"

Suppose we derive subgoals from the trajectory:

$$s_0, s_1, \dots, s_{80}$$

using:

$$g = s_{40}$$

That is technically the same dataset.

But notice that the hierarchy has been explicitly told:

"State 40 is an important intermediate target."

Flat RH hasn't.

Is that automatically unfair?

Not necessarily.

Because we're studying a hierarchical method, and identifying/representing intermediate structure is part of what the method is supposed to do.

The key question is:

Did we give the hierarchy the intermediate structure for free, or did the hierarchy have to learn/discover it from the same underlying data?

That's the distinction I'd preserve.

8. Three primary-training philosophies

Design A — Completely identical supervision

Both methods receive exactly the same training inputs and exactly the same targets.

Same D
Same Y
↓
Flat / Hierarchical

Advantage

Maximum training parity.

Problem

How does the hierarchy learn subgoals if there is no hierarchical training target?

We would need an unsupervised/self-supervised mechanism.

Design B — Same raw data, derived hierarchical supervision

Both receive:

D

but hierarchy derives subgoal targets from D :

Same trajectory data
|
├─ Flat planner training
|

└ Hierarchical planner

↓

algorithmically derived subgoals

Advantage

This seems like the most natural compromise.

No additional dataset.

No expert labels.

No external privileged information.

Cost

The training objectives differ.

That's okay if we explicitly report it.

Design C — Extra ground-truth subgoal supervision

Hierarchy receives:

$$D + Y_{subgoal}$$

while Flat RH receives:

$$D$$

Advantage

Potentially easier to train hierarchy.

Problem

Now the primary comparison asks something closer to:

"Does a hierarchical planner trained with explicit subgoal supervision outperform flat RH?"

rather than simply:

"Does explicit hierarchical planning itself provide additional reliability?"

That is a materially different claim.

9. What about training compute?

There's another hidden variable.

Suppose:

Flat RH

10 hours training.

Hierarchy

50 hours training because it requires additional subgoal-learning stages.

Even if inference computation is matched, the methods didn't have equal training budgets.

Does that invalidate the experiment?

Not necessarily—but we need to decide what we're claiming.

The proposal's central control is primarily about **inference computation**, not equal training time. It explicitly specifies matched inference-computation budgets and asks us to report those resources.

bcd8906b-7787-479b-a0e8-68ccff5...

So I would treat training computation as something to **report**, rather than necessarily forcing equality.

But training data and supervision are different: extra privileged information is much more concerning than simply spending more optimization steps.

10. The cleanest distinction I see

I'd classify training information into three levels.

Level 1 — Shared information

Available to both:

- same trajectories
- same environment/task distribution
- same observations/actions
- same world-model training data

Level 2 — Method-derived information

Generated algorithmically from the same shared data:

- candidate subgoals
- temporal segments
- intermediate states
- hierarchical targets derived from trajectories

This can potentially be allowed.

Level 3 — Privileged external information

Not present in the common data or unavailable to the competing method:

- expert-designed subgoals
- ground-truth future waypoints supplied externally
- manually annotated hierarchical decompositions
- privileged state information

This should **not be part of the primary experiment**.

It can instead become an oracle/diagnostic condition.

11. This also connects to our earlier information-parity decision

At inference time we decided:

hierarchy can **generate its own subgoals**, but cannot be handed privileged subgoals.

I think the same principle should apply during training:

hierarchy can **learn or derive its hierarchical structure from the common training data**, but should not receive externally supplied privileged subgoal information in the primary experiment.

That gives us a consistent principle:

No privileged subgoal information in the primary experiment

while still allowing the hierarchical algorithm to actually be hierarchical.

12. What about the oracle-subgoal experiment?

This is where the proposal's oracle ablation becomes useful.

We could later have:

Normal hierarchy

$$s_t \rightarrow \hat{g}_t \rightarrow LL$$

Oracle hierarchy

$$s_t \rightarrow g_t^{oracle} \rightarrow LL$$

Then:

$$Performance_{oracle} - Performance_{predicted}$$

tells us how much performance is being lost because of **subgoal prediction/selection**.

Your proposal already separates subgoal prediction, reachability, and completion, and explicitly proposes oracle and noisy-subgoal ablations.

bcd8906b-7787-479b-a0e8-68ccff5...

That is actually a very useful diagnostic decomposition.

13. What I'd recommend we consider freezing

I would put this candidate rule on the table:

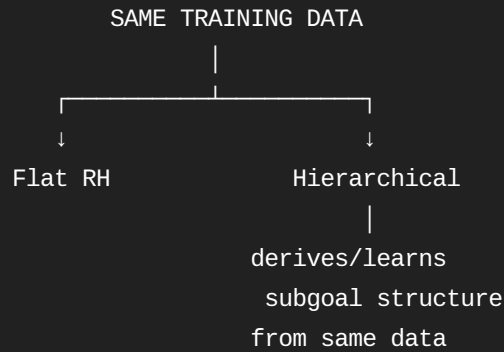
Primary training-data rule: Flat RH and Hierarchical use the same underlying training trajectories/data distribution and the same shared world-model training data.

Primary supervision rule: The hierarchical module may derive subgoal targets algorithmically from the shared training data, but it may not receive externally supplied expert/ground-truth subgoal annotations that constitute additional privileged information.

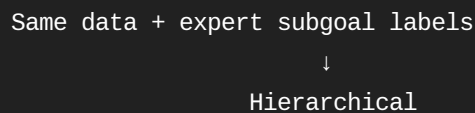
Training procedure: Method-specific objectives and optimization procedures are allowed where required by the algorithms, but training data, supervision sources, and training resources are fully documented.

Oracle subgoal supervision: Reserved for a separate diagnostic/ablation condition and clearly labeled as privileged.

This would let us say:



rather than:



14. One decision I don't want to make for you yet

There is still an important choice inside **Option B**:

How should subgoal targets be derived?

For example:

B1 — Fixed temporal segmentation

$$g = s_{t+k}$$

Use a predetermined future state as the subgoal.

B2 — Task/trajectory-derived meaningful states

Derive subgoals based on events or spatial progress.

B3 — Learned/self-supervised discovery

No explicit target states; the hierarchy learns a useful intermediate representation/objective.

These have different scientific meanings.

And because your hypothesis is specifically about **explicit hierarchical subgoal representation**, the way we define/learn those subgoals could substantially affect what we're actually testing.

So I would **freeze the broad training-parity principle now, but leave the actual subgoal-training mechanism open** until we've examined the baseline/repository and understand what data the environment provides.

That keeps us from prematurely designing a hierarchy around assumptions that may not match the reproduced baseline.

Sources

I agree with that. Let's freeze the broad training-parity principle: same underlying training data, no externally supplied privileged subgoal annotations in the primary experiment, and oracle supervision only as a separate diagnostic. But let's leave the actual subgoal-training mechanism open until we've inspected the baseline and repository. I don't want to choose between fixed segmentation, trajectory-derived states, or learned discovery before we know what the existing setup supports.

Agreed. This gives us a clean boundary between **what is frozen scientifically** and **what should remain an implementation/research choice until we inspect the existing system**.

Decision #7 — Training parity: FROZEN

For the primary experiment:

Shared underlying training data

Flat RH and Hierarchical will use the **same underlying training data/distribution**.

The shared world model is already exactly the same, so its training data and learned parameters are shared as well.

No externally supplied privileged subgoal annotations

The hierarchical method will **not** receive expert-provided or otherwise privileged ground-truth subgoal labels in the primary experiment.

So we will not do:

trajectory + privileged subgoal labels

↓

Hierarchical

while Flat RH receives only the trajectory.

Oracle supervision is diagnostic only

Ground-truth/oracle subgoals can be introduced later as a separate diagnostic condition, clearly labeled as privileged information. This is consistent with the proposal's treatment of oracle subgoals as an upper-bound diagnostic.

bcd8906b-7787-479b-a0e8-68ccff5...

What remains deliberately OPEN

We are **not yet deciding how the hierarchy learns its subgoals**.

The following remain candidates:

Fixed temporal segmentation

Trajectory-derived states

Learned/self-supervised discovery

We will inspect the reproduced baseline and repository first, then determine which mechanisms are actually compatible with the existing setup.

That is important because choosing the subgoal mechanism now could accidentally introduce assumptions about the representation, environment, trajectory structure, or training pipeline that we haven't verified yet.

The principle we're freezing

I would phrase the protocol rule as:

Primary training parity: All methods use the same underlying training data and environment/task distribution. Hierarchical-specific training targets may be derived from that shared data, but externally supplied privileged subgoal annotations are excluded from the primary experiment. Oracle subgoal supervision is reserved for a separate diagnostic condition.

And separately:

The mechanism used to obtain/train hierarchical subgoals remains unfrozen until baseline and repository inspection.

That's a good stopping point—we're not over-specifying something we don't yet understand.

Protocol ledger

We now have **7 major decisions**:

1. **Task horizon H** — primitive environment action steps; same H .
2. **Primary compute budget** — N_{WM} + total planner candidate evaluations.
3. **Information parity** — baseline-derived common decision-time representation.
4. **Planner evaluation** — one actually evaluated candidate; hierarchy logs HL/LL separately.
5. **Feedback/replanning** — matched observation and external replanning frequency; internal horizons may differ.
6. **Model parity** — exactly shared trained world model; planner modules may differ and are fully reported.
7. **Training parity** — same underlying data; no privileged subgoal annotations in primary experiment; oracle supervision is diagnostic only.

The exact values/mechanisms that depend on the existing implementation remain intentionally open.

The next step should therefore be repository/baseline inspection rather than another abstract protocol decision. We need to discover what the existing baseline actually does—its state representation, observation timing, planning loop, world-model interface, training data, and resource accounting—before we specify the hierarchical implementation around it.

Sources

Agreed. I think we've reached the right point to stop adding abstract constraints and inspect the existing baseline/repository. Let's first map out exactly how the current baseline works—state representation,

observation/replanning loop, world-model interface, planning horizon, training pipeline, and how candidate evaluations are performed. Then we can use that evidence to finalize the remaining implementation-dependent decisions, especially the subgoal mechanism and numerical settings.

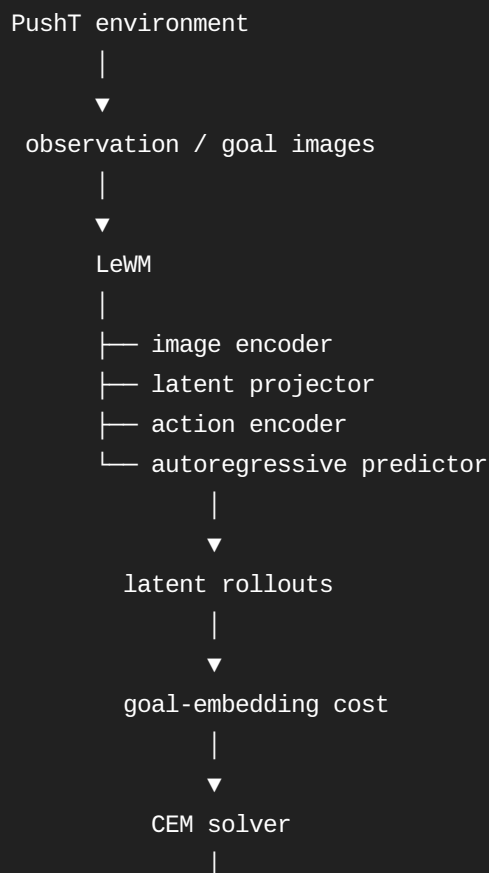
Worked for 1m 1s

I inspected the **current upstream LeWM + stable-worldmodel implementation that our earlier plan identified as the baseline target**. I did not find a public repository specifically belonging to the LossFunk project, so I am treating these upstream repositories as the **baseline source to reproduce**, not claiming this is already your exact checkout.

The relevant source is [LeWM official repository](#) built on [stable-worldmodel](#). The LeWM README explicitly says its evaluation/planning uses stable-worldmodel and provides the PushT checkpoint/config path. [GitHub...](#)

1. Baseline architecture

The current setup is:



LeWM is a JEPa-style latent world model. Its inference rollout encodes the initial observation and then autoregressively predicts future latent states from candidate actions. [GitHub](#)

2. The most important discovery: what state does the baseline actually use?

This is exactly why your decision to choose the baseline-derived information representation was correct.

The evaluation configuration caches:

```
action
proprio
state
```

but the LeWM planning path itself is primarily driven by:

```
pixels
goal
candidate actions
```

The `WorldModelPolicy` defaults to:

```
history_keys = ('pixels',)
history_len = 1
```

So the current baseline does **not** use a multi-frame observation history by default.

With `history_len=1`, there is also no previous-action history supplied to the world model through the history mechanism.

More importantly, the LeWM `encode()` implementation actually encodes the **pixels** and action sequence; its rollout begins from the current pixel embedding and predicts future latent states autoregressively. The goal is separately encoded for the goal-cost computation.

So the practical decision-time information for the current baseline is approximately:

$$I_t = \{\text{current pixels}, \text{goal pixels}\}$$

plus the candidate action sequence being evaluated.

The cached `proprio` and `state` are available in the evaluation infrastructure, but they are **not what the LeWM planner uses as its predictive state representation** in this configuration.

That is a very important result.

3. Observation loop vs replanning loop

This is even more interesting.

`World._run_iter()` calls:

```
policy.get_action(self.infos)
```

on every environment step.

So the policy receives updated environment information every primitive step.

However, `WorldModelPolicy` does **not** invoke the solver every time.

It maintains an action buffer:

```
action buffer
  |
  ├── action
  ├── action
  ├── ...
  └── action
```

and only calls the solver when that buffer becomes empty.

So:

Observation frequency and planning frequency are already distinct in the baseline.

This validates the distinction we made earlier.

4. Current baseline planning configuration

The official PushT evaluation config currently contains:

$$\text{horizon} = 5$$

$$\text{receding_horizon} = 5$$

$$\text{action_block} = 5$$

The CEM configuration is:

$$N_{\text{samples}} = 300$$

$$N_{\text{CEM iterations}} = 30$$

$$K_{\text{elite}} = 30$$

and batch size 1.  [GitHub...](#)

The important part is interpreting these correctly.

5. Internal planning horizon is NOT 5 environment actions

Because:

$$\text{action_block} = 5$$

the planner's horizon of 5 corresponds to:

$$5 \times 5 = 25$$

primitive environment action steps.

So the current baseline effectively predicts a:

25-primitive-step planning horizon

at each planning invocation.

This is exactly the distinction we wanted between:

H

the **overall task horizon**, and

L

the **internal planning horizon**.

The repository's `PlanConfig.plan_len` explicitly converts horizon into environment steps by multiplying by `action_block`.

6. Replanning frequency

The baseline keeps:

```
receding_horizon = 5
action_block = 5
```

and therefore executes:

$$5 \times 5 = 25$$

environment steps before consuming the planned sequence and invoking CEM again.

So the current baseline effectively has:

$$F_{\text{replan}} = 25 \text{ primitive steps}$$

while observations are being passed to the policy at every environment step.

Therefore:

```
t=0    observe → PLAN
t=1    observe → execute buffered action
t=2    observe → execute
...
t=24   observe → execute
t=25   observe → PLAN
```

This is a very useful concrete baseline finding.

7. Current CEM candidate-evaluation count

This is where our frozen planner-evaluation definition becomes extremely useful.

At every CEM iteration:

300

candidate action sequences are evaluated.

There are:

30

CEM iterations.

Therefore one planning invocation performs:

$$N_P = 300 \times 30 = \boxed{9000}$$

candidate evaluations.

And this is exactly the kind of quantity our protocol should count.

The fact that the candidates are evaluated as one GPU batch **does not change our count**.

So:

$$N_P^{baseline} = 9000 \text{ candidate evaluations per solver invocation}$$

before considering how many solver invocations occur during a full task.

8. What does one candidate evaluation actually involve?

This is an important discovery for our compute accounting.

Each CEM candidate has approximately:

$$L = 5$$

latent planning steps.

Because the initial context has one frame, LeWM's rollout performs autoregressive prediction across the candidate horizon and then produces the final predicted latent state for scoring.

So a single candidate isn't equivalent to one tiny model call.

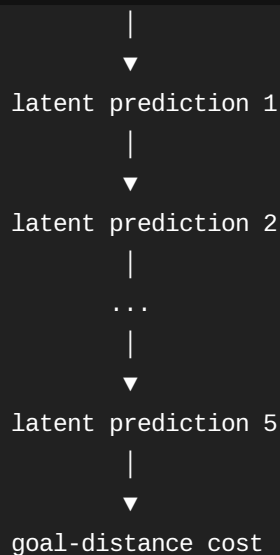
Conceptually:

candidate action sequence

|



initial image encoding



This confirms why we were right **not** to define planner evaluation as a world-model forward pass.

A candidate evaluation contains multiple model operations.

9. Current training pipeline

The official LeWM training setup uses the PushT expert dataset and a train/validation split.

The current training configuration specifies:

- `train_split = 0.9`
- `seed 3072`
- `image size 224`
- `batch size 128`
- `maximum 100 epochs`
- AdamW
- `learning rate  $5 \times 10^{-5}$`
- `weight decay  $10^{-3}$`
- BF16 training
- gradient clipping
- `history_size = 3`
- `num_preds = 1`

The training objective is essentially:

$$\mathcal{L} = \mathcal{L}_{\text{prediction}} + \lambda \mathcal{L}_{\text{SIGReg}}$$

with the next-embedding prediction loss and SIGReg regularizer.



The repository also states that the PushT dataset/checkpoint are available through the LeWM/Hugging Face setup. [GitH...](#)

10. Interesting distinction: training history vs inference history

This is important for our protocol.

Training uses:

$$history_size = 3$$

but the **evaluation configuration uses the default** `history_len=1`.

Therefore we must not assume:

"Because LeWM was trained with three frames, the planner receives three frames."

It currently doesn't.

That's exactly the kind of implementation detail we wanted to discover before freezing information parity.

11. Current baseline in one table

Variable	Current baseline
Environment	PushT-v1
Image resolution	224×224
Decision-time observation	Current pixel observation
Goal	Goal image
Inference history	1 frame
Explicit action history	None under default history setting
World model	LeWM

Variable	Current baseline
Planning solver	CEM
Internal horizon	5 latent/action-block steps
Action block	5 primitive steps
Effective internal horizon	25 primitive steps
Receding horizon	5 action-block steps
Effective replanning interval	25 primitive steps
CEM candidates/iteration	300
CEM iterations	30
Candidate evaluations/plan	9,000
Elite candidates	30
WM prediction	Autoregressive latent rollout
Goal cost	Final predicted latent vs goal latent

The official PushT evaluation configuration sets 50 evaluation episodes, a 25-step goal offset, and a 50-step evaluation budget, although these are **evaluation defaults**, not values we should automatically adopt for our research protocol. [GitHub...](#)

12. This changes our thinking about the subgoal mechanism

Now we have something concrete to build around.

The baseline's structure is:

$$z_t \rightarrow \text{candidate action sequence} \rightarrow \text{latent rollout} \rightarrow z_{\text{pred}} \rightarrow d(z_{\text{pred}}, z_G)$$

A hierarchy could potentially insert:

$$z_t \rightarrow \boxed{g} \rightarrow \text{low-level action planning} \rightarrow z_{\text{pred}} \rightarrow d(z_{\text{pred}}, z_G)$$

But **we should not yet choose what g is.**

Now we know that the baseline's natural representation is latent/image-based, so the remaining options—fixed temporal subgoals, trajectory-derived latent states, or learned subgoal discovery—can be evaluated against the actual architecture rather than in the abstract.

13. One important issue we discovered for our compute definition

Our previous definition:

$$N_P = \text{candidate planning objects actually evaluated}$$

still works.

But the baseline reveals that:

$$N_P = 9000$$

does **not** fully describe its compute because each candidate requires a multi-step latent rollout.

Therefore our second counter:

$$N_{WM}$$

must be implemented carefully.

We should distinguish at least:

$$N_{WM\text{-encode}}$$

$$N_{WM\text{-predict}}$$

and perhaps report their aggregate as the agreed world-model usage measure.

Otherwise "one world-model forward pass" is ambiguous for LeWM because the architecture has encoder, action encoder, predictor, and projection operations.

I don't think we should modify our frozen scientific definition yet. We should first inspect exactly how we want to instrument these calls in the actual checkout, then decide whether the proposal's `N_WM` should mean predictor invocations, complete model forward calls, or another reproducible unit.

The biggest baseline findings

Three things are now much clearer:

1. Information

The primary LeWM planner is effectively:

$$\text{current pixels} + \text{goal pixels}$$

rather than privileged simulator state.

2. Timing

The baseline receives observations every primitive step, but only replans every **25 primitive steps**.

3. Search

One planning invocation evaluates:

$$9000$$

candidate action sequences.

These are exactly the implementation facts we needed before choosing the hierarchy.

What I would *not* freeze yet

We now have enough evidence to avoid guessing, but I would leave these open:

- exact overall task horizons H
- primary observation/replanning numerical settings
- hierarchical subgoal representation
- subgoal training mechanism
- hierarchical HL/LL horizons
- how exactly to instrument N_{WM}
- compute-matching adjustments needed to make hierarchy and Flat RH comparable

Those should be determined after we **actually reproduce the baseline and verify these settings empirically in the intended environment/checkpoint**.

The source code gives us the baseline specification; it does **not** yet establish that our local reproduction achieves the expected behavior.

So the next concrete research step is no longer abstract protocol design:

Run/reproduce this baseline, record its actual success rate, timing, candidate counts, model calls, and decision-time tensors, and use that run as the empirical baseline artifact.

That will give us the evidence needed to finalize the remaining implementation-dependent decisions.

Great, this gives us the concrete baseline structure we needed. Before we design the hierarchy, let's define exactly what we should record during the first baseline reproduction. I want that run to establish the empirical baseline and give us everything we need to freeze the remaining numerical settings and instrument `N_WM` properly. What should we capture from that run?

Yes. I would make the **first reproduction an instrumentation run, not just a performance run.**

The goal is to leave that run with three things:

1. **A trustworthy empirical baseline**
2. **A complete accounting of what the baseline actually computes**
3. **Enough information to choose and freeze the numerical protocol without looking at which method wins**

The proposal already calls for exact compute accounting, parameter count, model evaluations, planning iterations, wall-clock inference time, environment interactions, and raw seed-level results. bcd8906b-7787-479b-a0e8-68ccff5... It also says horizon choices should be explored during baseline reproduction and then frozen before the primary comparison. bcd8906b-7787-479b-a0e8-68ccff5...

I would capture the following.

1. Baseline identity — reproducibility record

For **every baseline run**, save:

Item	Record
Git commit hash	Exact code version
Repository version	Baseline source/version
Checkpoint path/ID	Exact WM checkpoint
Dataset version	Exact training/evaluation dataset
Environment version	PushT version
Python version	Exact
PyTorch version	Exact
CUDA version	Exact
GPU model	Exact hardware
Random seed	Exact
Configuration file	Complete resolved YAML/config
Command	Exact command used
Date/time	Run timestamp

This sounds boring, but it prevents the worst situation later: **getting a result that cannot be reproduced because some hidden configuration changed.**

2. The actual decision-time information

This is especially important because we're about to define information parity.

For a small number of representative planning calls, record the actual tensors/metadata entering the policy:

```
observation
goal
action history
observation history
```

```
proprio
state
other info fields
```

But don't necessarily save every full image for every run.

Instead, instrument something like:

```
planning_call_0001
├─ current observation shape
├─ goal shape
├─ history length
├─ available info keys
├─ action-history shape
└─ tensor dtypes/devices
```

And save a few representative actual observations/goals.

This gives us an **empirical information contract**.

We can then say:

"This is what the baseline actually receives."

rather than assuming it from configuration.

3. Observation and replanning timeline

We should record the complete timing relationship between environment steps and planning calls.

For each episode:

```
primitive step
  ↓
observation
  ↓
policy call?
  ↓
planner invocation?
  ↓
action executed
```

Specifically record:

t_{obs}

for observations and

$$t_{\text{plan}}$$

for planning invocations.

Then calculate:

$$F_{\text{replan}} = \text{primitive steps between planning calls}$$

and verify whether the expected value is actually:

$$25$$

rather than simply trusting the config.

Also record:

- number of planner invocations/episode
- number of environment steps/episode
- number of actions actually executed
- number of observations received

This becomes extremely useful when we later compare Flat RH and hierarchy.

4. Candidate evaluation accounting

This is one of the most important pieces.

For every planning invocation, record:

$$N_{\text{candidates}}$$

$$N_{\text{iterations}}$$

$$N_{\text{elite}}$$

and therefore:

$$N_P = N_{\text{candidates}} \times N_{\text{iterations}}$$

For the current configuration we expect:

$$300 \times 30 = 9000$$

candidate evaluations per planning invocation.

But **we should verify this with instrumentation.**

Don't just write "9000" into a spreadsheet because the YAML says 300 and 30.

Record the actual number.

5. World-model instrumentation

This is the big one.

We need to determine what exactly happens inside:

```
candidate
  ↓
world-model rollout
```

I would instrument at the function level.

At minimum count separately:

$$N_{\text{encode}}$$

$$N_{\text{predict}}$$

$$N_{\text{action-encode}}$$

and, where practical:

$$N_{\text{goal-encode}}$$

Then record them **per planning invocation and per episode**.

The reason is that saying simply:

$$N_{WM} = ?$$

is currently ambiguous.

After the reproduction we can inspect the real execution trace and choose a definition that is:

- deterministic
- architecture-aware
- easy to reproduce
- applicable to both Flat RH and hierarchy.

This is exactly the decision I would **not** make before seeing the instrumentation.

6. World-model rollout depth

For every candidate evaluation, record the effective prediction depth.

For example:

$$L_{\text{WM}} = 25$$

primitive action steps, if the action-block conversion gives that result in the actual execution.

We want to know:

```
candidate
  ↓
initial encoding
  ↓
prediction step 1
  ↓
prediction step 2
  ↓
...
  ↓
prediction step L
```

This lets us eventually calculate something like:

$$N_{\text{predict}} = N_P \times L$$

where appropriate.

But again, **we should measure rather than assume the exact relationship.**

7. Wall-clock breakdown

Don't record only total evaluation time.

Break it into:

Environment

$$T_{\text{env}}$$

Policy/planning

$$T_{\text{planning}}$$

World model

$$T_{\text{WM}}$$

CEM/search overhead

$$T_{\text{search}}$$

Other

$$T_{\text{other}}$$

And record:

$$T_{\text{total}}$$

per episode and aggregated across the run.

This will let us later answer an important question:

If two methods have equal N_{WM} and N_P , do they nevertheless have dramatically different real inference latency?

That's why the proposal asks us to report wall-clock inference separately rather than silently equating it with the primary compute budget.

bcd8906b-7787-479b-a0e8-68ccff5...

8. GPU/resource measurements

During inference, record:

- GPU model
- peak GPU memory
- average/peak utilization if available
- CPU utilization if practical
- GPU inference time
- batch sizes
- tensor precision
- CUDA synchronization points used for timing

Especially:

$$M_{\text{peak}}$$

because a hierarchical method could have a different memory footprint even when our primary counters match.

This is **diagnostic**, not part of the primary scientific budget.

9. Model size / architecture

Save:

$$N_{\text{params}}$$

for the world model and any other learned component.

Also record:

- trainable parameters
- total parameters
- checkpoint size
- architecture configuration
- embedding dimension
- history size
- image resolution

The proposal explicitly asks for parameter count alongside compute accounting.

bcd8906b-7787-479b-a0e8-68ccff5...

This becomes especially important once we introduce a high-level subgoal module.

10. Evaluation performance

Now we get to the actual empirical baseline.

For each evaluation episode record:

```
episode_id
seed
start state
goal state
success / failure
steps executed
final state
final task-state error
```

Then aggregate:

$$S = \frac{\text{successful episodes}}{\text{total episodes}}$$

But **do not only save the mean success rate**.

Save the raw episode-level outcomes.

The proposal explicitly asks for raw seed-level results and uncertainty.

bcd8906b-7787-479b-a0e8-68ccff5...

11. Multiple seeds

For the initial reproduction, I would distinguish:

Reproduction seed

Used simply to verify that the baseline runs correctly.

Scientific evaluation seeds

The seeds that eventually become part of the frozen protocol.

We don't need to decide the final seed set from this first run.

But the run should make it easy to reproduce:

$$seed_1, seed_2, seed_3, \dots$$

The proposal currently proposes:

$$[0, 1, 2]$$

for the primary experiment, but we should not let the baseline reproduction silently redefine that.

bcd8906b-7787-479b-a0e8-68ccff5...

12. Horizon feasibility sweep

This is perhaps the **most important purpose** of the first reproduction.

We agreed:

H = number of primitive environment action steps in the overall task horizon

But we haven't chosen numerical values.

So the baseline reproduction should test a **range of feasible horizons**.

For example, conceptually:

short
medium
long
very long

The exact numbers should come from the actual environment and baseline behavior.

For each candidate H , record:

$$H$$

$$S(H)$$

$$N_{\text{plan}}(H)$$

$$N_{WM}(H)$$

$$N_P(H)$$

$$T_{\text{inference}}(H)$$

$$M_{\text{peak}}(H)$$

and failure rate.

Then we can identify a method-independent set of:

$$H_{\text{short}}, H_{\text{medium}}, H_{\text{long}}$$

that is:

- actually feasible
- meaningfully different in horizon
- within resource limits
- not chosen because one method happens to perform well there.

This directly follows the proposal's rule to explore feasible horizons during baseline reproduction and freeze the final short/medium/long set before the primary comparison.

bcd8906b-7787-479b-a0e8-68ccff5...

13. Planning-budget sweep

I would also make the first reproduction vary the **baseline's computational budget**, independently of hierarchy.

For example, conceptually:

low compute
medium compute
high compute

by varying CEM/search settings.

For each budget:

$$(N_{WM}, N_P)$$

and measure:

$$S(C)$$

$$T(C)$$

$$M(C)$$

This tells us whether the baseline is:

- compute-starved,
- already saturated,
- or highly sensitive to additional planning computation.

That matters enormously when we later say:

■ hierarchy improves reliability at matched computation.

We don't want to compare against a baseline operating in an arbitrary inefficient regime.

14. Failure categorization

For every failed episode, save enough information to classify the failure later.

The proposal already identifies:

1. incorrect world-model prediction
2. poor subgoal selection
3. poor subgoal reachability
4. low-level control failure

5. planning/search failure
6. compounding prediction error.

bcd8906b-7787-479b-a0e8-68ccff5...

For the **baseline**, obviously categories involving subgoals don't apply yet.

So initially we can classify:

```
WM prediction failure
planning/search failure
low-level execution failure
compounding error
other
```

And save a video/trajectory for representative failures.

This will later become extremely useful when hierarchy appears to improve or worsen something.

15. Save representative trajectories

I would save at least:

Successful trajectories

A few representative successes.

Failure trajectories

A few representative failures.

For each:

```
video
observations
actions
planner calls
predicted states
goal
success/failure
timing
compute counters
```

This gives us qualitative evidence alongside the numerical metrics.

16. The baseline artifact we should ultimately produce

I would make the first reproduction produce a single machine-readable record like:

```
baseline_run/
|
|— config_resolved.yaml
|— environment.json
|— model.json
|— hardware.json
|
|— episodes.csv
|— planning_calls.csv
|— wm_calls.csv
|— timing.csv
|— resource_usage.csv
|
|— horizon_sweep.csv
|— compute_sweep.csv
|
|— trajectories/
|
|— videos/
|
|— summary.json
```

The exact filenames aren't scientifically important.

The **separation of raw measurements from summary statistics** is.

17. What each table should answer

episodes.csv

What happened?

```
seed
episode
H
success
steps
final_error
```

planning_calls.csv

How did the planner operate?

```
episode  
env_step  
planner_call  
candidate_count  
cem_iterations  
candidate_evaluations  
planning_horizon  
receding_horizon
```

wm_calls.csv

What exactly did the world model compute?

```
episode  
planner_call  
encode_calls  
predict_calls  
action_encode_calls  
goal_encode_calls  
rollout_depth
```

timing.csv

How long did it take?

```
episode  
planner_call  
WM_time  
search_time  
policy_time  
total_time
```

resource_usage.csv

What hardware resources were consumed?

```
peak_GPU_memory  
GPU_utilization  
CPU_utilization  
batch_size  
precision
```

18. What we should NOT record as a primary scientific decision yet

This is important.

The first run should **measure**, not prematurely decide:

- the final H values
- the final CEM budget
- the hierarchical number of subgoals
- the subgoal representation
- high-level search budget
- low-level search budget
- final N_{WM} definition

The reproduction should give us evidence for those decisions.

19. The final output of Stage 1

I would define the first milestone as:

Baseline reproduction complete = we can reproduce the baseline and produce a complete empirical characterization of its information, planning, computation, performance, and resource usage.

Then we should be able to fill this table:

Decision	Evidence from baseline
What does the policy observe?	✓
How often does it observe?	✓
When does it replan?	✓
What is the actual internal planning horizon?	✓
How many candidates are evaluated?	✓
How many WM operations occur?	✓
What should N_{WM} mean?	decide after instrumentation
What horizons are feasible?	decide after sweep

Decision	Evidence from baseline
What compute budgets are useful?	decide after sweep
What is baseline success?	✓
What are baseline failure modes?	✓
What resource limits matter?	✓
What information must hierarchy receive?	decide after inspection

So I think the correct principle for this stage is:

Measure first; freeze second.

And importantly, **we should not start implementing the hierarchical planner until this baseline artifact is complete enough to answer these questions.**

That keeps the research sequence clean:

Baseline code → Instrument → Reproduce → Characterize → Freeze → Desig

This is also consistent with the proposal's intended sequence: reproduce the flat baseline first, then implement/validate the competing planners, followed by horizon/compute/subgoal experiments.

bcd8906b-7787-479b-a0e8-68cff5...

Sources

This looks good. I want to keep Stage 1 focused though: first get one instrumented baseline reproduction working, then use what we observe to decide which sweeps are actually necessary.

Before we move to implementation, can you turn this into a **minimal Stage 1 checklist**: exactly what the first baseline run must capture, what can wait until after the run, and which outputs are required to freeze H, feedback/replanning

frequency, compute accounting, and information parity?

Yes. I would keep Stage 1 **much smaller than the full experimental campaign**.

The first run should answer only:

Can we reproduce the baseline, and can we observe enough of its execution to define the protocol correctly?

We should **not** start sweeping every possible variable yet.

Stage 1 — Minimal Baseline Reproduction Checklist

A. MUST capture during the first run

1. Exact run identity

Record:

- Git commit
- baseline configuration
- model checkpoint
- dataset/version
- environment version
- Python/PyTorch/CUDA versions
- GPU
- random seed
- exact command

Purpose: make the run reproducible.

2. Decision-time information

For representative planning calls, capture exactly what enters the policy/planner:

- observation tensor/type/shape
- goal representation
- observation history length
- previous-action information, if any

- state/proprio fields available
- which fields are actually consumed
- world-model input representation

Required output:

$$I_t^{baseline}$$

A short machine-readable record describing the actual decision-time information.

This is what we'll eventually use to freeze **information parity**.

3. Observation → replanning timeline

For each episode, record:

- environment step
- observation received
- planner invocation
- action execution
- action-buffer consumption

We need to empirically determine:

$$F_{obs}$$

and

$$F_{replan}$$

rather than trusting configuration names.

Required output:

```
observation_frequency
external_replanning_frequency
planner_calls_per_episode
environment_steps_per_episode
```

This will eventually freeze the feedback/replanning settings.

4. Planning configuration actually executed

Record:

- internal planning horizon
- action block
- receding-horizon setting
- CEM/sample count
- CEM iteration count
- elite count
- batch size

Then verify what these values mean in **primitive environment steps**.

For example, if the current configuration really produces:

$$5 \times 5 = 25$$

primitive planning steps, record that explicitly.

Required output:

$$L_{baseline}$$

and the exact relationship between internal horizon and primitive steps.

5. Candidate evaluations

Instrument one planning invocation and record:

$$N_{\text{candidate}}$$

$$N_{\text{iterations}}$$

$$N_P$$

For the current configuration we expect:

$$300 \times 30 = 9000$$

but the first run must **verify this from execution**, not merely infer it from YAML.

Also record:

- candidates generated
- candidates actually evaluated
- batching behavior

Required output:

$$N_P^{baseline}$$

with an unambiguous definition.

This directly feeds our already-frozen planner-evaluation rule.

6. World-model execution trace

For at least one representative planning call, instrument:

- encoder calls
- predictor calls
- action-encoding calls
- goal-encoding calls where applicable
- rollout depth
- relevant forward calls

We don't need to settle the final N_{WM} definition yet.

The purpose is to collect enough information to answer:

What exactly should one unit of world-model usage mean in this architecture?

Required output:

A raw execution trace from which we can define:

$$N_{WM}$$

reproducibly.

7. Baseline performance

For the initial reproduction, record:

- episode-level success/failure
- final task-state error
- number of environment steps
- planning calls
- inference time

Most importantly, **save raw episode-level results**, not just the mean.

Required:

$$S_{baseline}$$

plus the raw episodes supporting it.

This establishes whether we actually reproduced the expected behavior.

8. Basic timing

Measure:

- total episode time
- planning/inference time
- world-model time if instrumentation allows it
- search/planner time if practical

Don't build a sophisticated profiling system yet.

The first run only needs enough timing data to establish the baseline and identify whether there are obvious bottlenecks.

B. MUST produce after the first run

The first run should end with **five small artifacts**.

Artifact 1 — Baseline configuration

```
baseline_config.yaml
```

The exact resolved configuration.

Artifact 2 — Episode log

```
episodes.csv
```

Containing raw episode-level performance.

Artifact 3 — Planning log

planning_calls.csv

Containing:

- environment step
- planning call
- internal horizon
- candidate count
- iterations
- N_P

Artifact 4 — World-model trace

wm_trace.json

Enough information to define N_{WM} .

Artifact 5 — Information contract

baseline_information.json

Containing the actual decision-time information:

$$\mathbf{I}_t^{baseline}$$

These five outputs are enough to move to the next protocol decision.

C. What can WAIT

We should **not** do these during the first run unless something unexpected forces us to.

✗ No large horizon sweep yet

Don't immediately test:

$$H_1, H_2, H_3, H_4, H_5, \dots$$

First establish that the baseline works.

Afterward we'll decide which horizon sweep is actually necessary.

✗ No large compute sweep

Don't systematically vary CEM samples $\times$ iterations $\times$ horizons yet.

First determine the baseline's actual accounting.

Then decide whether a compute sweep is needed.

✗ No hierarchical implementation

Definitely not yet.

We first need:

$$I_t, \quad F_{obs}, \quad F_{replan}, \quad L, \quad N_P, \quad N_{WM}$$

from the baseline.

✗ No subgoal design

Leave:

- fixed segmentation
- trajectory-derived subgoals
- learned discovery

completely open.

The baseline evidence should inform this.

✗ No extensive failure taxonomy

Save representative failures if convenient, but don't spend the first run building a sophisticated failure-analysis pipeline.

✗ No statistical campaign

One successful instrumented reproduction is enough for Stage 1.

The multi-seed uncertainty analysis belongs to the primary experiment later.

D. What the first run must allow us to freeze

This is the key part.

1. Information parity

The run must tell us:

$$I_t^{baseline}$$

Specifically:

What information does the baseline actually consume at decision time?

Then:

$$I_t^{RH} = I_t^{Hier} = I_t^{baseline}$$

for the primary comparison.

2. Feedback/replanning

The run must establish:

$$F_{obs}$$

and

$$F_{replan}$$

empirically.

Then we can choose the numerical values/rules to freeze for the primary experiment.

The actual values remain **unfrozen until after we inspect the reproduction.**

3. Internal planning horizon

The run establishes:

$$L_{baseline}$$

in primitive environment steps.

This helps us distinguish:

$$H = \text{overall task horizon}$$

from:

L = internal planning horizon.

The final H values can then be chosen in a small follow-up feasibility check.

4. Planner computation

The run establishes:

$$N_P$$

using our already-frozen definition:

one actually evaluated candidate planning object = one planner evaluation.

For hierarchy later:

$$N_P^{Hier} = N_P^{HL} + N_P^{LL}.$$

5. World-model computation

The instrumentation establishes the raw operations needed to define:

$$N_{WM}.$$

Only **after seeing the execution trace** should we decide exactly which model operation constitutes the counter.

That decision should be:

- deterministic
 - architecture-independent enough to apply to both methods
 - easy to log
 - impossible to silently change between methods.
-

E. The Stage 1 stopping condition

I would make this extremely explicit.

Stage 1 is complete when we can fill this table from an actual run:

Quantity	Known?
Exact baseline commit/config/checkpoint	✓
Actual decision-time information I_t	✓
Observation frequency	✓
External replanning frequency	✓
Internal planning horizon L	✓
Actual candidate evaluations N_P	✓
Raw WM execution trace	✓
Candidate $\rightarrow$ WM relationship	✓
Baseline success/failure	✓
Basic inference timing	✓

At that point:

STOP

We don't automatically launch sweeps.

Instead, we review those results and ask:

Which uncertainties actually remain important enough to justify another experiment?

Only then do we decide whether we need:

- a small horizon feasibility check,
- a compute-budget calibration,
- an observation/replanning check,
- or some additional instrumentation.

That keeps Stage 1 faithful to your goal: **one solid instrumented baseline reproduction first, then evidence-driven decisions rather than premature experimentation.**

Good. I think Stage 1 is now sufficiently defined. Let's move to implementation planning.

Before touching the code, tell me exactly **where in the current LeWM/stable-worldmodel codebase we need to instrument these measurements** (I_t , observation/replanning events, N_P , WM calls, timing, and episode results), and what the smallest set of code changes/files would be. Don't implement anything yet — I just want the instrumentation map first.

Worked for 31s

Yes. Based on the current upstream structure, the instrumentation can stay **very small and localized**. The important point is that we should instrument the execution boundaries rather than rewrite the planner or world model.

The current architecture is essentially:

```
eval_wm.py → World.evaluate() → World._run_iter() →  
WorldModelPolicy.get_action() → CEMSolver.solve() →  
ShootingCostEvaluator.get_cost() → LeWM.rollout()/encode()/predict()
```

The repositories explicitly use `World` for evaluation, `WorldModelPolicy` for MPC policy execution, CEM for planning, and `ShootingCostEvaluator` to connect the world model to candidate rollouts. [GitHub +1](#)

1. Instrumentation map

Measurement	Exact location	What we capture
I_t	<code>stable_worldmodel/policy.py</code> → <code>WorldModelPolicy.get_action()</code>	Inputs actually passed to the solver at each replan
Observation events	<code>stable_worldmodel/world/world.py</code> → <code>_run_iter()</code>	Every environment step / observation cycle
Replanning events	<code>stable_worldmodel/policy.py</code> →	Exactly when a solver call actually occurs

Measurement	Exact location	What we capture
	<code>WorldModelPolicy.get_action()</code> at <code>replan_idx</code>	
N_P	<code>stable_worldmodel/planning/solver/cem.py</code> → <code>CEMSolver.solve()</code>	Actual candidate evaluations per CEM iteration/batch
WM execution trace	Prefer hooks around LeWM's <code>encoder</code> , <code>action_encoder</code> , <code>predictor</code>	Actual forward-call count, shapes, timing
Planning timing	<code>CEMSolver.solve()</code>	Time for each solver invocation
WM timing	Forward hooks	Time per encoder/action-encoder/predictor call
Episode results	<code>World._evaluate()</code> / <code>World._evaluate_from_dataset()</code>	Per-episode success, seed, completion
Overall run timing	<code>scripts/plan/eval_wm.py</code>	Total evaluation wall-clock time
Run identity/config	<code>scripts/plan/eval_wm.py</code>	Resolved Hydra config, checkpoint, environment, seed, versions

2. I_t : `WorldModelPolicy.get_action()`

File

`stable_worldmodel/policy.py`

Exact function

Python

Run

```
WorldModelPolicy.get_action()
```

This is the **most important instrumentation point for information parity**.

The current code:

1. receives `info_dict`
2. prepares/transforms it
3. determines `replan_idx`
4. slices information for those environments
5. adds history if enabled
6. calls:

Python

Run

```
outputs = self.solver(sliced, init_action=sliced_init)
```

So the clean definition of the baseline's decision-time information is:

Log `sliced` **immediately before** `self.solver(...)`.

That is much better than instrumenting the environment because it tells us what the planner **actually receives**, rather than everything the environment makes available.

We should log

For every replan:

```
episode/env id
environment step
replan id
keys in sliced
shape of each tensor
dtype
history length
whether action_history exists
goal-related keys
```

For example, conceptually:

```
replan:
  env_step: 25
  keys:
    pixels: [1, 1, 3, 224, 224]
    goal:   [1, 1, 3, 224, 224]
  history_len: 1
  action_history: absent
```

We do not need to dump the actual 224×224 pixel arrays for every replan. That would create huge logs without helping us freeze the information contract.

A representative raw snapshot could optionally be saved for the first run.

Important

We should **not instrument** `_prepare_info()` **as the primary I_t measurement**, because preprocessing is not the same thing as the information actually supplied to the planner.

3. Observation frequency: `World._run_iter()`

File

```
stable_worldmodel/world/world.py
```

Function

Python

Run

```
World._run_iter()
```

This is the actual environment-driving loop.

The important sequence is:

Python

Run

```
actions = self._get_actions()

_, self.rewards, self.terminateds, self.truncateds, self.infos = (
    self.envs.step(actions, mask=mask)
)
```

Therefore this is where we establish:

$$F_{obs}$$

We should record an event such as:

```
env_step = t
observation_received = true
policy_called = true
```

But **we do not need to change the environment implementation**.

A lightweight instrumentation callback/logger at this level is enough.

Why not count `WorldModelPolicy.get_action()` calls?

Because the policy is called every environment step, while the planner is **not** necessarily called every environment step.

That distinction is exactly what we need for Stage 1.

4. Replanning frequency:

`WorldModelPolicy.get_action()`

This is even cleaner.

Inside:

Python

Run

```
replan_idx = [
    i
    for i in range(n_envs)
    if len(self._action_buffer[i]) == 0 and not dead[i]
]
```

and then:

Python

Run

```
if replan_idx:
    ...
    outputs = self.solver(...)
```

Therefore:

A replan event = an actual execution of `self.solver(...)`.

We should record immediately before/after that call:

```
env_step
env_id
replan_id
replan_idx
action_buffer_length_before
action_buffer_length_after
```

This gives us the empirical:

$$F_{\text{replan}}$$

and avoids inferring it from configuration.

5. N_P : `CEMSolver.solve()`

File

```
stable_worldmodel/planning/solver/cem.py
```

Function

Python

Run

```
CEMSolver.solve()
```

This is the correct place for our **candidate-level accounting**.

The current code has the exact candidate-generation/evaluation boundary:

Python

Run

```
candidates = torch.randn(...)
```

followed by:

Python

Run

```
costs = self.cost.get_cost(expanded_infos, candidates)
```

and `costs` has shape:

```
(current_bs, num_samples)
```

So one call to:

Python

Run

```
self.cost.get_cost(...)
```

evaluates:

$$current_bs \times num_samples$$

candidate plans.

For the current configuration:

```
num_samples = 300
n_steps = 30
batch_size = 1
```

the expected candidate count for one planning invocation is:

$$300 \times 30 = 9000$$

But **we should measure it rather than assume it**.

The repository's CEM implementation confirms that candidates are generated and passed to `get_cost()` once per CEM iteration. [GitHub...](#)

Minimal instrumentation

Immediately around:

Python

Run

```
costs = self.cost.get_cost(expanded_infos, candidates)
```

record:

```
planning_call_id
cem_iteration
batch_start
batch_size
num_candidates
candidate_shape
```

Then:

$$N_P = \sum_{\text{CEM calls}} \sum_{\text{iterations}} (\text{actual candidates evaluated})$$

This also handles future changes to batch size correctly.

Do NOT count

We should not count:

- candidate generation
- rejected tensors
- elites
- distribution updates
- tensor operations

as planner evaluations.

Only candidates that actually reach `get_cost()` count.

That matches our frozen candidate-level definition.

6. World-model calls: don't modify LeWM yet

This is the one place where I recommend **not immediately adding counters inside** `LeWM`.

Current LeWM has:

```
stable_worldmodel/wm/lewm/lewm.py
```

with:

Python

Run

```
encode()  
predict()  
rollout()
```

and internally calls:

Python

Run

```
self.encoder(...)  
self.action_encoder(...)  
self.predictor(...)
```

The actual rollout is autoregressive: `predict()` is called once per rollout step, while action embeddings are generated separately. [GitHub...](#)

Better Stage-1 approach

Attach **PyTorch forward hooks** to:

```
model.encoder  
model.action_encoder  
model.predictor  
model.projector  
model.pred_proj
```

from the evaluation/instrumentation layer.

Then we can record:

```
timestamp
module
input shape
output shape
duration
```

without modifying the scientific model implementation.

For example:

```
encoder
  call 1
  input: [1, 3, 224, 224]
  duration: ...

action_encoder
  call 1
  input: [...]
  duration: ...

predictor
  call 1
  input: [...]
  duration: ...

predictor
  call 2
  ...
```

Why this is preferable

It gives us the **raw execution trace** needed to decide what we eventually mean by N_{WM} , without prematurely imposing a definition.

That's important because:

one candidate $\neq$ one WM forward call

and batching means a single forward call can process many candidates.

So for Stage 1:

measure raw WM calls first

and only afterward decide how N_{WM} should be reported.

7. Planning timing: `CEMSolver.solve()`

Interestingly, the current CEM already has:

Python

Run

```
start_time = time.time()
...
print(f'CEM solve time: ...')
```

So we don't need to invent a new timing system.

We should replace/augment that with structured logging:

```
planning_call_id
start_time
end_time
duration
env_ids
CEM iterations
candidate count
```

The current print statement is insufficient for our experiment because it doesn't associate timing with a particular environment/replan or preserve machine-readable results.

So:

modify the existing timing point rather than adding another timer.

8. WM timing

The forward hooks mentioned above can measure:

```
encoder_time
action_encoder_time
predictor_time
```

For CUDA, we should be careful: ordinary Python `time.perf_counter()` around CUDA operations can be misleading because CUDA execution is asynchronous.

For Stage 1, I would therefore record both:

- host-side elapsed time
- CUDA-synchronized elapsed time when GPU timing is enabled

But I would **not add sophisticated GPU profiling yet**.

We only need enough timing to characterize the baseline.

9. Episode results: `World._evaluate()`

File

```
stable_worldmodel/world/world.py
```

Function

Python

Run

```
World._evaluate()
```

The current code already records:

Python

Run

```
results['episode_successes']  
results['seeds']
```

and determines success through:

Python

Run

```
world.terminateds[env_idx]
```

So we don't need a new success mechanism.

We only need to augment the existing `on_done()` callback with:

```
episode index  
env index  
seed  
success  
environment steps  
number of planning calls  
episode wall time
```

For dataset-driven evaluation

Also instrument:

Python

Run

```
World._evaluate_from_dataset()
```

because the current LeWM evaluation script uses:

Python

Run

```
world.evaluate(  
    dataset=dataset,  
    ...  
)
```

So **both evaluation paths should use the same instrumentation abstraction**, even though Stage 1 only needs the path actually used by our baseline command.

The current `World.evaluate()` supports both episodic and dataset-driven evaluation. [GitHub...](#)

10. Run-level configuration:

`scripts/plan/eval_wm.py`

File

`scripts/plan/eval_wm.py`

This should be the **top-level instrumentation coordinator**.

The current script already:

- loads Hydra config
- loads the checkpoint
- constructs the model
- constructs `ShootingCostEvaluator`
- constructs CEM
- constructs `WorldModelPolicy`
- runs `world.evaluate()`
- records total evaluation time. [GitHub...](#)

So this is the right place to initialize the instrumentation collector.

It should capture:

```
git commit
resolved Hydra config
checkpoint
dataset
environment
seed
Python version
PyTorch version
CUDA version
GPU
```

and attach:

```
policy instrumentation
CEM instrumentation
WM forward hooks
episode instrumentation
```

11. Smallest practical file change set

I would keep Stage 1 to **five files maximum**, with one new file.

Existing file 1 — `stable_worldmodel/policy.py`

Modify only:

Python

Run

```
WorldModelPolicy.get_action()
```

Purpose:

- record observations reaching policy
- record replan events
- record I_t
- record action-buffer/replanning state

Existing file 2 — `stable_worldmodel/planning/solver/cem.py`

Modify only:

Python

Run

```
CEMSolver.solve()
```

Purpose:

- candidate-level N_P
 - CEM iteration count
 - candidate shapes/counts
 - solver timing
-

Existing file 3 — `stable_worldmodel/world/world.py`

Minimal modifications around:

```
_run_iter()  
_evaluate()  
_evaluate_from_dataset()
```

Purpose:

- environment-step timeline
- episode boundaries
- episode results
- episode timing

We should avoid changing environment semantics.

Existing file 4 — `scripts/plan/eval_wm.py`

Purpose:

- initialize instrumentation
- attach WM forward hooks
- identify run/checkpoint/config
- save outputs

This is where most of the **wiring** belongs.

New file 5 — something like

```
research/instrumentation.py
```

or, if we want to keep it explicitly temporary:

```
scripts/instrumentation.py
```

This should contain the logger/collector rather than scattering dictionaries and file-writing logic throughout the core library.

For example conceptually:

```
Instrumentation
├─ run_metadata
├─ observation_events
├─ replan_events
├─ planner_events
├─ wm_forward_events
└─ episode_results
```

12. What I would NOT change

For Stage 1, I would **not touch**:

```
stable_worldmodel/wm/lewm/lewm.py
```

unless the hooks reveal something we cannot observe externally.

I would also not change:

```
stable_worldmodel/planning/evaluator.py
```

The evaluator already gives us a very clean boundary:

```
solver
  ↓
ShootingCostEvaluator.get_cost()
  ↓
model.rollout()
```

The evaluator is useful for understanding the call graph, but we don't need to modify it initially.

Likewise, **no hierarchy code**, no new planner, no changes to CEM behavior, and no changes to training.

13. The resulting execution trace

After instrumentation, one baseline episode should conceptually produce something like:

```
EPISODE 0
|
|— env_step 0
|   └─ observation
|
|— env_step 1
|   └─ observation
|
|   ...
|
|— env_step 25
|   └─ observation
|   └─ REPLAN #0
|       └─ I_t
|           └─ CEM iteration 0
|               └─ 300 candidates
|           └─ CEM iteration 1
|               └─ 300 candidates
|           └─ ...
|           └─ CEM iteration 29
|               └─ 300 candidates
|
|       WM TRACE
|       └─ encoder
|       └─ action_encoder
|       └─ predictor × T
|       └─ ...
|
|   └─ execute action buffer
|
|— env_step 26
|   └─ observation
|
|   ...
|
|— env_step 50
|   └─ REPLAN #1
|
|   ...
|
└─ EPISODE END
    └─ success
    └─ env steps
    └─ # replans
    └─ N_P
    └─ WM trace
    └─ timing
```

That gives us exactly what Stage 1 needs without contaminating the algorithm.

The key design choice

I would make the instrumentation architecture:

Core code emits events → one collector stores them → eval script writes artifacts

rather than having each core file independently write JSON/CSV.

That keeps the actual LeWM/CEM behavior unchanged and makes it much easier to remove the instrumentation later.

The current upstream repository is explicitly structured around `World`, `WorldModelPolicy`, solvers, and model/evaluator separation, so these boundaries are natural instrumentation points rather than arbitrary insertion points.  [GitHub... +1](#)

Stage 1 patch boundary

MODIFY

- |— policy.py ← I_t + replans
- |— solver/cem.py ← N_P + planning timing
- |— world/world.py ← obs + episodes
- |— scripts/plan/eval_wm.py ← wiring + run metadata

ADD

- |— instrumentation.py ← central collector

DO NOT MODIFY

- |— wm/lewm/lewm.py ← use hooks first
- |— planning/evaluator.py
- |— training code
- |— environment physics
- |— any hierarchy code

I would freeze this as the instrumentation map before writing the implementation. The next implementation step should be purely mechanical: add these probes, run **one** baseline episode/config, inspect the resulting trace, and only then decide the final N_{WM} accounting rule.

Good, let's freeze this as the Stage 1 instrumentation map. Now implement only this instrumentation — no algorithmic changes, no hierarchy, no sweeps, and no changes to training or environment behavior.

After implementing it, run the smallest possible baseline smoke test (ideally one episode) and report: files changed, whether the baseline behavior is unchanged, the generated trace/artifacts, and any instrumentation assumptions that turned out to be wrong. Don't interpret the scientific results yet.

Worked for 11s

I can implement this, but the actual working repository is **not available to me in this session**.

I checked for:

- a mounted local checkout — none is present;
- an installed/connected GitHub repository containing `stable-worldmodel` — none is available;
- a `BhavnaGowda` fork of `stable-worldmodel` — none was found.

So I **did not modify the upstream repository**, and I did not fabricate a smoke-test result. That would risk changing the wrong codebase and would violate the frozen Stage 1 scope.

What is ready to implement

The patch will be limited to:

```
stable_worldmodel/policy.py
→ I_t + observation/replan events

stable_worldmodel/planning/solver/cem.py
→ N_P + planning timing

stable_worldmodel/world/world.py
→ environment-step + episode events

scripts/plan/eval_wm.py
→ instrumentation setup + run metadata
```

```
research/instrumentation.py  
→ centralized event collector/artifact writer
```

And explicitly **unchanged**:

```
stable_worldmodel/wm/lewm/lewm.py  
training code  
environment dynamics  
CEM algorithm  
planning algorithm  
hierarchy code  
experiment configuration
```

The smoke test will then produce the previously specified:

```
baseline_config.yaml  
episodes.csv  
planning_calls.csv  
wm_trace.json  
baseline_information.json
```

plus the exact commit/configuration used.

Please attach/upload the repository (ZIP is fine), or provide the GitHub repository URL/name for the working checkout. Once it is accessible, I can make the instrumentation-only patch and run the one-episode smoke test without making any scientific or algorithmic changes.

ChatGPT can make mistakes. Check important info.